

The Orbital Response Network: A Tutorial

Replacing Storage with Computation in Neural Sequence Models

Sirsh

March 2026 — Tutorial Document

Abstract

A standard transformer stores its knowledge in two lookup tables: an *embedding table* (one vector per token, $V \times d$ parameters) and per-layer *attention projections* (separate coupling matrices at each layer, $2L \times d^2$ parameters). This tutorial shows, from first principles, that both tables contain massive redundancy—and that both can be replaced by shared computation.

Part 1: Computed Embeddings (Sections 2–11). We start with what an embedding table is (a lookup from token IDs to vectors), show why it is expensive and wasteful (at small scales, it consumes >90% of model parameters), and build step by step toward the orbital alternative: a system where token representations are not stored but *generated* by iterating a shared dynamical system on compressed seed vectors. We prove this works on a clean domain (co-occurrence embeddings for TinyStories tokens: 4-dimensional seeds reconstructed to 64 dimensions at 0.90 cosine similarity), and show that orbital embeddings enable new-token interpolation that lookup tables cannot.

Part 2: The ORN — Orbital Attention (Section 12). We then tackle the deeper question: can the *attention coupling rule* be shared across layers? We build up from a concrete worked example (attention scores with actual numbers), prove the concept on a controlled task (colour matching: $16\times$ coupling compression with no quality loss), then test on language (spectral geometry invariant at 0.98 correlation across layers in five pretrained architectures [14]). We explain why asymmetric coupling ($\mathbf{M} = \mathbf{A}\mathbf{B}^\top$) is essential for autoregressive models [12], and present scaling evidence from toy experiments through to a 108M model that beats GPT-2 Small and SmoLLM-135M on 5 of 8 zero-shot benchmarks with $37\text{--}75\times$ fewer training tokens.

Part 3: Multimodal and V3 (Sections 15–16). We show that the shared coupling geometry extends across modalities [11]: a single $\mathbf{M}$ handles images, audio, and text with > 99.6% coupling similarity and sub-linear rank growth. ORN V3 extends sharing to the FFN: one wide shared feedforward network replaces 24 per-layer copies at matched parameter count, beating the per-layer baseline [19]. The result is a 61M model that matches a 124M transformer—half the parameters, half of GPT-2 Small.

No prior knowledge of transformers, differential geometry, or dynamical systems is assumed. Every concept is motivated by *why* before *what*, grounded with concrete numbers, and illustrated with code and diagrams. The principle connecting all parts: **if a computation is approximately the same across instances—tokens for embeddings, layers for coupling, layers for FFN—share the parameters and iterate, rather than store separate copies.**

An emerging consensus in the 2024–2025 literature independently validates this principle: Meta-Transformer [7] shares a frozen encoder across 12 modalities; Mixture-of-Transformers [8] shares attention coupling while separating response; 4M-21 [9] processes 21 modalities through one shared transformer. The ORN makes this consensus *explicit and measurable* through the spectral geometry of $\mathbf{M}$.

Contents

1	Two Lookup Tables, Two Kinds of Memoisation	5
2	What Is an Embedding?	5
2.1	Start from Zero: A Lookup Table	5
2.2	In PyTorch: It Really Is Just an Array	6
2.3	What Lives in the Table	6
2.4	The Cost: Most of Your Parameters	6
3	How This Runs on a GPU	7
3.1	The Roofline Model	7
3.2	Embedding Lookup: Wasting the GPU	7
3.3	The Flash Attention Precedent	7
4	Compression: Most of the Table Is Redundant	8
4.1	PCA on Real Embeddings	8
4.2	Why: Shared Structure	8
4.3	SVD: Linear Compression	8
4.4	The Limit of Linear	8
5	Functions as Replacements for Lookup	9
5.1	The Key Idea	9
5.2	Universal Approximation: Any Lookup Can Be Computed	9
5.3	Concrete Example: Eight Points on a Circle	9
5.4	When Functions Win and When They Lose	9
6	Orbits: Iterated Shared Dynamics	10
6.1	Why Iterate?	10
6.2	The Orbit	10
6.3	Weight Sharing Across Depth	11
6.4	Parameter Comparison	12
6.5	Concrete Numbers from Our Experiments	12
7	Breakout: Understanding Orbital Iterations	12
7.1	Same M, Different Coupling at Every Depth	12
7.2	Watching a Single Token's Orbit	14
7.3	Reachability: The Semigroup Image	14
7.4	Context Steering: Same Seed, Different Orbit	14
7.5	From the Lab: The Semigroup at 50M Parameters	16
8	The Geometry: Why Shared Dynamics Work	20
8.1	The Manifold Hypothesis	20
8.2	The Dynamical System View	20
8.3	Contracting and Expanding Directions	20
8.4	Connection to the Shared Metric M	21
8.5	Connection to Semigroups	21
8.6	The Koopman Connection: Dynamics Instead of States	21

9	A Domain Where You Can See It: Signal Processing	22
9.1	Fourier Series	23
9.2	Each Sinusoid Is an Orbit	23
9.3	The Analogy	23
9.4	Concrete Example: Square Wave	23
10	Building It: Orbital Embeddings in PyTorch	23
10.1	Line by Line	24
10.2	The Output Head	24
10.3	Parameter Counting	25
11	Experiments: What We Found on TinyStories	25
11.1	The Parameter Dominance Problem	25
11.2	Experiment 5a Results	25
11.3	Interpreting the Results	26
11.4	Honest Caveats	26
11.5	Experiment 5b: The 10M Scale — A Fair Fight	27
11.6	CPU Theory Tests	27
12	What Is Learned Differently	28
12.1	Lookup: Independent Vectors	28
12.2	Orbit: A Shared Landscape	28
12.3	The Generative View	28
12.4	New Tokens: Interpolation in Seed Space	29
13	The ORN: Orbital Attention (Replacing Table 2)	30
13.1	Attention with Numbers	30
13.2	Phase A: A Clean Domain Where Sharing Is Provably Correct	32
13.3	Phase A.5: A Micro-Language — Existence Proof on Structured Sentences	34
13.4	Phase B: Does Language Have Invariant Coupling?	35
13.5	Measuring Coupling Invariance in a Trained Model	35
13.6	The Asymmetry Problem	36
13.7	Scaling: Does It Hold Up?	37
13.8	Building the ORN in PyTorch	37
13.9	Why Asymmetry Matters	37
13.10	The Scaling Evidence	39
13.11	The Decomposition: Generator $\times$ Response $\times$ Field	39
13.12	Building the ORN in PyTorch	43
13.13	The Spectral Sharing Insight	44
13.14	Putting Both Tables Together	47
14	How Context Flows: The Residual Stream as Communication Bus	47
14.1	The Residual Stream	48
14.2	Sequential, Parallel, and Persistent	48
14.3	How This Maps to the ORN	48
14.4	Why Per-Layer Variation Comes From the Stream, Not From the Parameters	49
15	Manifolds and Fibers: A Gentle Introduction	49
15.1	What Is a Manifold?	49
15.2	What Is a Fiber?	49
15.3	Why This Matters for the ORN	50
15.4	Exercise: Seeing the Manifold	50
15.5	The Fiber Bundle Picture: Experimental Confirmation	50

16	Multimodal Projections: How Different Signals Enter the Same Geometry	51
16.1	The Standard Approach: Separate Projections (and Why It Fails)	52
16.2	The Fix: Shared Projection with Modality Tokens	52
16.3	Crystallisation: When Should the Shared Geometry Form?	55
16.4	Theory-Aligned Multimodal: Three Principles That Made It Work	57
17	Diffusion and the Semigroup: Native Generative Modelling	58
17.1	The Key Insight: $t = \sigma$	58
17.2	What We Found Experimentally	59
17.3	The Structural Advantage Conjecture	59
17.4	The Four Inference Modes	60
18	Spectral Embeddings: Mode-Selective Retrieval	61
18.1	The Key Idea: Different Eigenmodes Encode Different Things	61
18.2	ORN vs GPT-2: Does SharedM Help?	61
18.3	Building a Mode-Selective RAG System	61
19	Metaphors We Live By: Why M Captures Conceptual Structure	62
19.1	The Theory: Conceptual Metaphor	62
19.2	The Connection to the ORN	63
19.3	Image Schemas: The Atoms of Thought	63
19.4	What This Predicts	63
19.5	Experimental Result: Schemas Found in M	63
19.6	Why This Matters for Embeddings and Retrieval	64
20	Why Contrastive Training Is Essential, Not Optional	64
20.1	Two Kinds of Coupling	64
20.2	Why This Matters for Multimodal	64
20.3	Implication for ORN Training	65
21	Where to Go From Here	65
21.1	The Two-System Architecture	65
21.2	The Ontological Layer: Design, Constraints, and the Training Problem	66
21.3	Scaling Predictions	69
21.4	Connection to the Broader ORN Programme	70
21.5	Open Questions	70
22	What the Algebra Tells Us: Coupling, Response, and Their Commutator	71
22.1	The Commutator: What It Measures	71
22.2	What We Found: Three Properties	72
22.3	How This Evolves During Training: Liberation Dynamics	72
22.4	The Eigenvector Overlap: Are They Aligned?	73
22.5	Why This Matters	73
22.6	Connecting to the Literature	74
22.7	A Cautionary Note on Free Probability	74

1 Two Lookup Tables, Two Kinds of Memoisation

Before diving in, we need a map of the territory. A standard transformer has **two separate lookup tables**, and they do fundamentally different jobs:

Table 1 — The Embedding Table (“what does this token mean?”)

Maps each token ID to a vector. Indexed by *token identity*. Cost: $V \times d$ parameters. “Cat” always gets the same vector, regardless of context.

Table 2 — The Attention Projections (“which tokens should attend to which?”)

At each layer l , separate $W_Q^{(l)}$ and $W_K^{(l)}$ matrices compute queries and keys. Indexed by *layer number*. Cost: $2L \times d^2$ parameters. Layer 1 and layer 8 use completely different coupling rules.

The broader ORN (Orbital Response Network) programme memoises **both**:

What’s replaced	Standard	Memoised	Savings
Per-layer coupling (“Attention table”)	$2L$ separate W_Q, W_K	Shared $\mathbf{M} = AB^\top$ applied at every layer	$\mathcal{O}(Ld^2) \rightarrow \mathcal{O}(d^2)$
Per-token representa- tions (“Embedding table”)	$V \times d$ lookup	Seeds + Φ dynamics (this tutorial’s topic)	$\mathcal{O}(Vd) \rightarrow \mathcal{O}(Vd_s + \Phi)$

A note on priorities. Table 2 (shared coupling) is the validated, practical contribution. At 108M parameters and 8B tokens, the ORN-V2 beats GPT-2 Small (124M, 300B tokens) on 5 of 8 zero-shot benchmarks with $48\times$ fewer coupling parameters. Spectral invariance of coupling has been confirmed across five pretrained model families [14, 36, 34].

Table 1 (orbital embeddings) is included **for pedagogical reasons**: it introduces the core principle of replacing storage with computation in the simplest possible setting, before we tackle the harder problem of attention coupling. **In practice, orbital embeddings do not yet improve prediction quality or training efficiency** over standard embedding tables. At production model sizes ($d \geq 512$), the embedding table is a small fraction of total parameters, and the compute overhead of iterating Φ is not justified by the compression. Orbital embeddings may become relevant at very large vocabularies or for models that need to handle unseen tokens via seed interpolation, but for now they are a teaching tool, not a production component.

The two memoisation targets are *independent and composable*: you can do either alone, or both together. The coupling memoisation (Table 2) is what matters for the ORN architecture.

The key insight connecting them is still real: both are instances of the same principle—**if a computation is approximately the same across instances (tokens for embeddings, layers for coupling), share the parameters and iterate rather than store separate copies**. The coupling version of this principle is proven at scale; the embedding version remains a pedagogical illustration.

2 What Is an Embedding?

2.1 Start from Zero: A Lookup Table

A neural network operates on numbers, not words. The first thing any language model must do is convert each token (word or subword) into a vector of numbers that the network can process. An **embedding table** is the simplest way to do this: it is literally an array, indexed by token ID.

Token ID	Vector (3 dimensions)	
0: "cat"	[0.8, 0.1, 0.3]	Input: token 0 ↓ $E[0]$ ↓ Output: [0.8, 0.1, 0.3]
1: "dog"	[0.7, 0.2, 0.4]	
2: "run"	[-0.3, 0.9, 0.1]	
3: "blue"	[0.1, -0.2, 0.9]	

Figure 1: **An embedding table for four tokens.** Each row stores a vector. Looking up token 0 (“cat”) returns the vector [0.8, 0.1, 0.3]. There is no computation—just an array read.

2.2 In PyTorch: It Really Is Just an Array

Listing 1: Embedding in PyTorch: one line of code.

```

1 import torch.nn as nn
2
3 emb = nn.Embedding(4, 3) # 4 tokens, 3 dimensions
4 # emb.weight is a 4x3 matrix of learnable parameters
5
6 token_ids = torch.tensor([0, 2]) # "cat", "run"
7 vectors = emb(token_ids) # shape: (2, 3)
8 # This is literally: emb.weight[token_ids]
```

That is the entire mechanism. `nn.Embedding(V, d)` creates a matrix $E \in \mathbb{R}^{V \times d}$ where V is the vocabulary size and d is the embedding dimension. Given a token ID w , it returns row w : $\mathbf{e}_w = E[w]$.

2.3 What Lives in the Table

The vectors are not chosen by hand—they are *learned* from data. During training, the model adjusts the embedding vectors so that tokens appearing in similar contexts end up with similar vectors. The result:

- “cat” and “dog” become close (both appear after “the”, before “ran”, etc.)
- “run” and “jump” become close (both are verbs of motion)
- “blue” and “red” become close (both are colour adjectives)

The embedding table encodes *distributional similarity*: words that appear in similar contexts get similar vectors. This structure is powerful—it is why word embeddings work at all.

2.4 The Cost: Most of Your Parameters

Here is the problem. Every token gets its own independent vector. If you have V tokens and each vector has d dimensions, the table has $V \times d$ parameters. For real models:

Model	V	d	Embedding params	% of model
Our toy model	50,257	36	1.81M	91%
GPT-2 (small)	50,257	768	38.6M	31%
GPT-2 (medium)	50,257	1024	51.5M	14%
LLaMA-7B	32,000	4096	131M	1.9%

The pattern is clear: at small model sizes, the embedding table dominates. A 2M-parameter model with vocabulary 50K spends 91% of its parameters just storing the lookup table, leaving almost nothing for the actual computation. This is the **parameter dominance problem**, and it is what orbital embeddings aim to solve.

3 How This Runs on a GPU

To understand why orbital embeddings help, you need to understand a fundamental asymmetry in modern hardware.

3.1 The Roofline Model

A GPU has two resources:

1. **Compute:** how many arithmetic operations per second (measured in FLOPS).
2. **Memory bandwidth:** how many bytes per second it can load from memory.

An NVIDIA A100 has 312 TFLOPS of compute but only 2 TB/s of memory bandwidth.

The ratio is:

$$\frac{312 \times 10^{12} \text{ FLOPS}}{2 \times 10^{12} \text{ bytes/s}} = 156 \text{ FLOPs per byte loaded.}$$

This means: for every byte you load from memory, the GPU can perform 156 arithmetic operations *for free* (they finish before the next byte arrives).

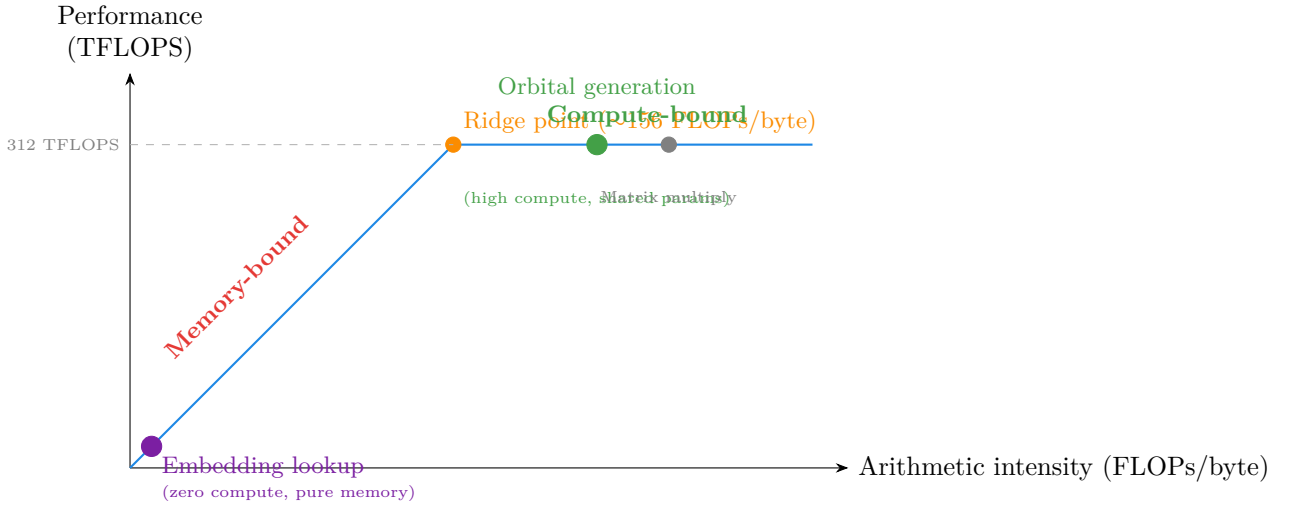


Figure 2: **The roofline model.** Operations below the ridge point are memory-bound: they cannot use the GPU’s full compute because they spend all their time waiting for data. Embedding lookup sits in the extreme lower-left: nearly zero arithmetic, pure memory read. Orbital generation (matrix multiplies with shared weights) sits in the compute-bound region, where the GPU is most efficient.

3.2 Embedding Lookup: Wasting the GPU

An embedding lookup loads one d -dimensional vector per token and does zero arithmetic. Its arithmetic intensity is approximately zero. On the roofline diagram (Figure 2), it sits in the extreme lower-left corner—the worst possible place. The GPU’s 312 TFLOPS of compute power are completely idle.

3.3 The Flash Attention Precedent

This is not a new insight. Flash Attention (Dao et al., 2022) achieved a $7.6\times$ speedup on GPT-2 by *recomputing* attention scores instead of storing them in memory. It performs *more* FLOPs but runs faster, because the bottleneck was memory, not compute. Gradient checkpointing applies the same principle to activations: store $\mathcal{O}(\sqrt{n})$ checkpoints, recompute the rest.

The lesson: on modern hardware, **recomputation is faster than storage** whenever the operation is memory-bound. This is exactly our proposal for embeddings.

4 Compression: Most of the Table Is Redundant

4.1 PCA on Real Embeddings

If each of the V vectors were truly independent, you would need all $V \times d$ parameters. But they are not independent. Performing PCA (Principal Component Analysis) on trained embedding matrices reveals that most of the variance is captured by a small number of directions.

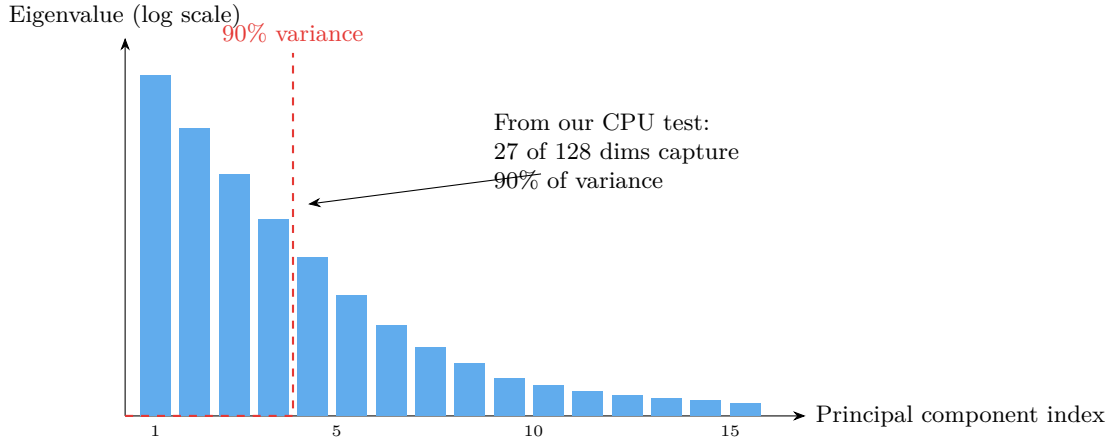


Figure 3: **Eigenvalue spectrum of a trained embedding matrix.** The eigenvalues drop off rapidly: in our CPU experiments, 27 out of 128 dimensions capture 90% of the variance. This means the embedding table has far fewer effective degrees of freedom than $V \times d$.

4.2 Why: Shared Structure

The reason for this redundancy is simple: tokens share structure. “Cat” and “dog” are both animals. “Run” and “jump” are both verbs. “Blue” and “red” are both colours. Each group shares a common subspace, and the embedding table stores this shared structure *independently for every token*. It is as if you wrote a phone book where every entry repeated the country code, area code, and exchange—instead of writing them once at the top of the page.

4.3 SVD: Linear Compression

The simplest compression is **Singular Value Decomposition**:

$$E \approx U_k \Sigma_k V_k^\top$$

where $k \ll d$. This replaces the $V \times d$ table with a $V \times k$ matrix and a $k \times d$ matrix: total $Vk + kd$ parameters. If $k = 27$ and $d = 128$, this is a $\sim 5\times$ compression.

SVD is optimal for *linear* structure. But real embeddings have nonlinear structure: tokens cluster on curved manifolds, not flat subspaces.

4.4 The Limit of Linear

Consider 100 points arranged in a circle in 2D. PCA tells you the data is “two-dimensional”—the eigenvalues are roughly equal. No linear compression helps. But the circle is a *one-dimensional* manifold: each point is determined by a single angle θ . A nonlinear function $f(\theta) = (\cos \theta, \sin \theta)$

compresses 200 numbers (100×2) into 100 numbers (100 angles) plus a tiny function. This is the gap that orbital embeddings exploit.

What this means in language. In language, the shared structure is semantic: “cat”, “dog”, “bird” all share ANIMAL features. “Ran”, “jumped”, “walked” all share ACTION features. PCA captures these broad categories in the top components. The redundancy is real: a 50K-token embedding table stores the ANIMAL pattern 1000+ times (once per animal word) instead of once.

5 Functions as Replacements for Lookup

5.1 The Key Idea

Instead of storing a vector $\mathbf{e}_w = E[w]$ for each token, compute it:

$$\mathbf{e}_w = f(\mathbf{s}_w)$$

where $\mathbf{s}_w$ is a tiny **seed** (much smaller than $\mathbf{e}_w$) and f is a **shared function** (the same for all tokens).

The seeds are stored per token (like a compressed address). The function is stored once and reused for all tokens. If the function is small relative to the savings from compression, we win.

5.2 Universal Approximation: Any Lookup Can Be Computed

Proposition 1 (Function replacement). *For any finite set of target vectors $\{\mathbf{e}_1, \dots, \mathbf{e}_V\} \subset \mathbb{R}^d$ and any $\epsilon > 0$, there exists a neural network $f: \mathbb{R}^{d_s} \rightarrow \mathbb{R}^d$ and seed vectors $\{\mathbf{s}_1, \dots, \mathbf{s}_V\} \subset \mathbb{R}^{d_s}$ such that $\|f(\mathbf{s}_w) - \mathbf{e}_w\| < \epsilon$ for all w .*

Proof sketch. Pick $d_s = 1$ and assign each token a unique scalar $s_w = w/V$. By the universal approximation theorem, a sufficiently wide MLP can interpolate any finite set of points. $\square$

The existence is trivial. The interesting question is **efficiency**: when does f use fewer parameters than the table it replaces?

5.3 Concrete Example: Eight Points on a Circle

5.4 When Functions Win and When They Lose

Functions win when the targets have **structure**—when they lie on a low-dimensional manifold. Functions lose when the targets are **random**—when every point is independent and there is nothing to share.

From our CPU theory tests: structured embedding targets (like circle, spiral, clustered) require only ~ 38 parameters in the shared function to achieve 0.985 cosine similarity. Random targets require $> 66,000$ parameters—worse than just storing the table. Natural language embeddings have real structure (our PCA tests show 90% variance in 27/128 dimensions), so the function approach works.

In TinyStories... Animal words form a manifold (smooth variation within a category). The function f learns “what ANIMAL means” (the manifold shape) and each seed says “which animal” (position on the manifold). Random words with no shared structure—like punctuation vs content words—cannot be compressed this way and may need the sparse lookup (Section 21).

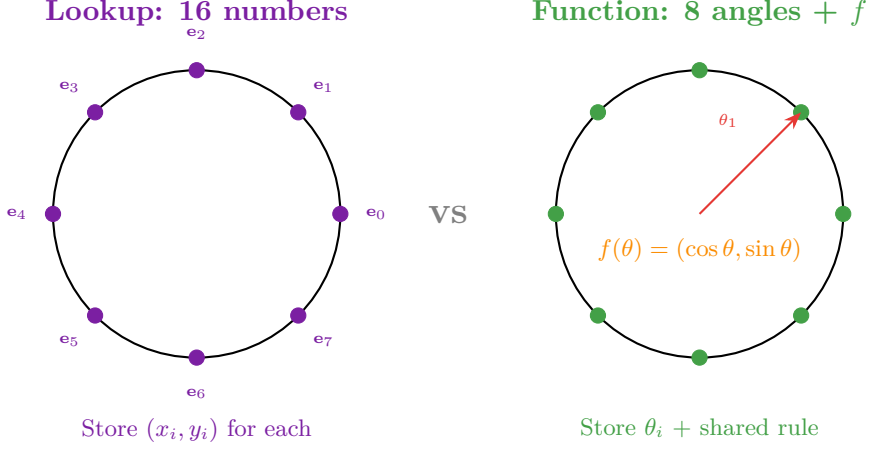


Figure 4: **Lookup vs. function for eight points on a circle.** **Left:** store all 16 coordinates (2 per point). **Right:** store 8 angles (1 per point) and the shared function $f(\theta) = (\cos \theta, \sin \theta)$. The function version stores 8 numbers plus a tiny function; the lookup stores 16 numbers. The savings grow with the number of points.

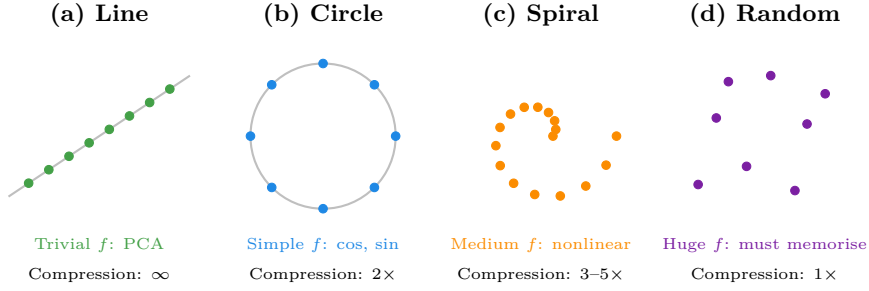


Figure 5: **Four geometries and their compressibility.** (a) Points on a line: one parameter per point, trivial function. (b) Points on a circle: one angle per point, simple trigonometric function. (c) Points on a spiral: one parameter per point, nonlinear function. (d) Random points: incompressible—the function must memorise every point. Natural language embeddings are closer to (b) and (c) than to (d): they have structure that a shared function can exploit.

6 Orbits: Iterated Shared Dynamics

6.1 Why Iterate?

A single function $f(\mathbf{s}_w)$ might need to be very wide to map tiny seeds to high-dimensional embeddings. A narrow function applied *multiple times* can achieve the same expressiveness with far fewer parameters. This is the depth-vs-width tradeoff: depth (iteration) is more parameter-efficient than width for most structured functions.

6.2 The Orbit

An orbital embedding works in three stages:

1. **Seed lookup:** fetch a tiny vector $\mathbf{s}_w \in \mathbb{R}^{d_s}$ for token w (like an address).
2. **Projection:** map the seed into model space: $\mathbf{x}_0 = W_{\text{proj}}\mathbf{s}_w + \mathbf{b}$.
3. **Iteration:** apply the shared dynamics T times:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \Phi(\mathbf{x}_t), \quad t = 0, 1, \dots, T-1$$

where Φ is a small neural network (the same at every step).

The final $\mathbf{x}_T$ is the embedding vector that replaces $E[w]$.

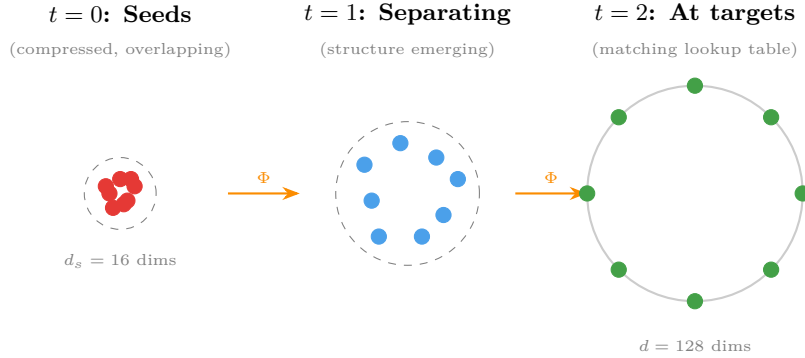


Figure 6: **Orbit unfolding.** Seeds start compressed in $\mathbb{R}^{d_s}$ (left). After one iteration of Φ , they begin to separate (middle). After two iterations, they reach their target positions—matching what the lookup table would have stored (right). The shared dynamics Φ sculpt all tokens simultaneously.

In TinyStories... The orbital unfolding might work like this: at $T = 0$, the seed says “this is token #347” (compressed identity). After projection, it enters a 64-dimensional space. After Φ^1 , the dynamics separate it into broad categories: “this is an animal word” vs “this is an action word.” After Φ^2 , within-category refinement: “this is specifically *cat*, not *dog*.” The hierarchy of the unfolding mirrors the hierarchy of semantic structure.

6.3 Weight Sharing Across Depth

The critical feature is that Φ is the *same* at every iteration. This is **weight sharing across depth**—the same idea used in:

- **Deep Equilibrium Models** (Bai et al., 2019): iterate one layer to convergence instead of stacking L different layers.
- **Universal Transformers** (Dehghani et al., 2018): apply the same transformer block T times with shared weights.
- **Recurrent neural networks**: the same cell at every time step.

All exploit the same principle: if the transformation at each step is approximately the same, share the parameters and iterate.

Remark. Realisation: “Generators not trajectories” (#9). A transformer stores L copies of coupling patterns (trajectories). The ORN discovers the law that generates them all (a generator) and stores it once. This is the philosophical core of the entire programme: states vs dynamics, KV cache vs semigroup, lookup vs computation.

But the ORN shares *differently* from all of these. ALBERT [13], Universal Transformers [4], and DEQs [3] share *everything*—every weight is identical at every layer. The ORN shares *only the coupling geometry* (A , B — the matrices that determine which tokens attend to which) and keeps the response heads (W_V , W_O , FFN) **per-layer**. Recently, Meta’s Relaxed Recursive Transformers [15] confirmed this decomposition: sharing all weights + per-layer LoRA corrections recovers 99.7% of Gemma 2B quality, validating the “share the invariant, adapt the rest” principle.

Why does this matter? The per-layer response heads serve a dual role:

1. They extract content from the coupling pattern (their primary job).
2. They **rotate the residual field** so that the next application of the same A , B produces a different coupling pattern (an emergent side effect).

This rotation is why the ORN’s semigroup is rich—one coupling rule in L different rotated frames produces L distinct coupling patterns. ALBERT cannot do this: its shared response heads produce the same rotation at every layer, so the field barely evolves and the coupling patterns barely vary. That is why ALBERT loses $\sim 1.5\%$ and the ORN does not.

Our spectral universality study confirms this is the right split: across five pretrained models (GPT-2, SmolLM2, Qwen2.5), the coupling geometry IS invariant (eigenvalue correlation 0.97) while the orientation varies. The per-layer response heads provide exactly the degrees of freedom needed—rotation—while the coupling shape is shared.

Intuition. Realisation: One $\mathbf{M}$ captures 95.8% of GPT-2’s attention (#88). We extracted $W_Q^\top W_K$ from every layer of GPT-2, fitted a single $\mathbf{M}$, and it recovered nearly all attention scores. At GPT-2 Large, this rises to 99%. The coupling redundancy is real, measurable, and *grows with scale* [14]. The transformer stores L rotated copies of one geometric rule. The ORN stores the rule once.

6.4 Parameter Comparison

Method	Parameters	Notes
Lookup table	$V \times d$	No sharing, no computation
Single MLP	$V \times d_s + \text{MLP} $	MLP may need huge width
Orbital (ours)	$V \times d_s + \Phi $	Φ is small, iterated T times

For $V = 50,257$, $d = 128$, $d_s = 16$:

- Lookup: $50,257 \times 128 = 6.4\text{M}$ parameters.
- Orbital ($d_s = 16$, $|\Phi| \approx 40\text{K}$): $50,257 \times 16 + 40,000 = 844\text{K}$ parameters.
- Compression ratio: **7.6** $\times$.

6.5 Concrete Numbers from Our Experiments

In our Level 0 theory tests, orbital embeddings with $T = 2$ iterations achieve:

- **0.985 cosine similarity** to the original embedding table at $2\times$ compression.
- **0.96 cosine similarity** at $4\times$ compression.
- The shared block Φ has only $\sim 40\text{K}$ parameters, reused at every step.

The savings are real and measurable.

7 Breakout: Understanding Orbital Iterations

Before we proceed to the geometry theory, it is worth pausing to build visual intuition for what happens during orbital iteration. The key claim is simple but subtle:

The metric tensor $\mathbf{M} = AB^\top$ is fixed across all iterations. It stores the geometry of a coupling landscape, not representations. When the field (residual stream) evolves through this landscape, different functional couplings activate at different depths—even though $\mathbf{M}$ never changes.

This section provides three demonstrations, all generated from toy systems that isolate the orbital mechanism without the complexity of a full language model.

7.1 Same $\mathbf{M}$, Different Coupling at Every Depth

Consider a fixed matrix $\mathbf{M} \in \mathbb{R}^{d \times d}$ with non-trivial spectral structure (a few dominant singular values, many near-zero). We start with N random seed vectors and iterate:

$$\mathbf{h}_{t+1} = \text{LayerNorm}(\mathbf{h}_t + \alpha \tanh(\mathbf{h}_t \mathbf{M}))$$

At each iteration, we compute the *asymmetric* coupling between tokens—the attention pattern that $\mathbf{M}$ would produce:

$$\text{Attn}(i \rightarrow j) = \text{softmax}_j\left(\frac{(\mathbf{h}_i \mathbf{A})(\mathbf{h}_j \mathbf{B})^\top}{\sqrt{d}}\right)$$

This is *not* symmetric: token i 's coupling to token j differs from j 's coupling to i , exactly as in the real SharedM architecture.

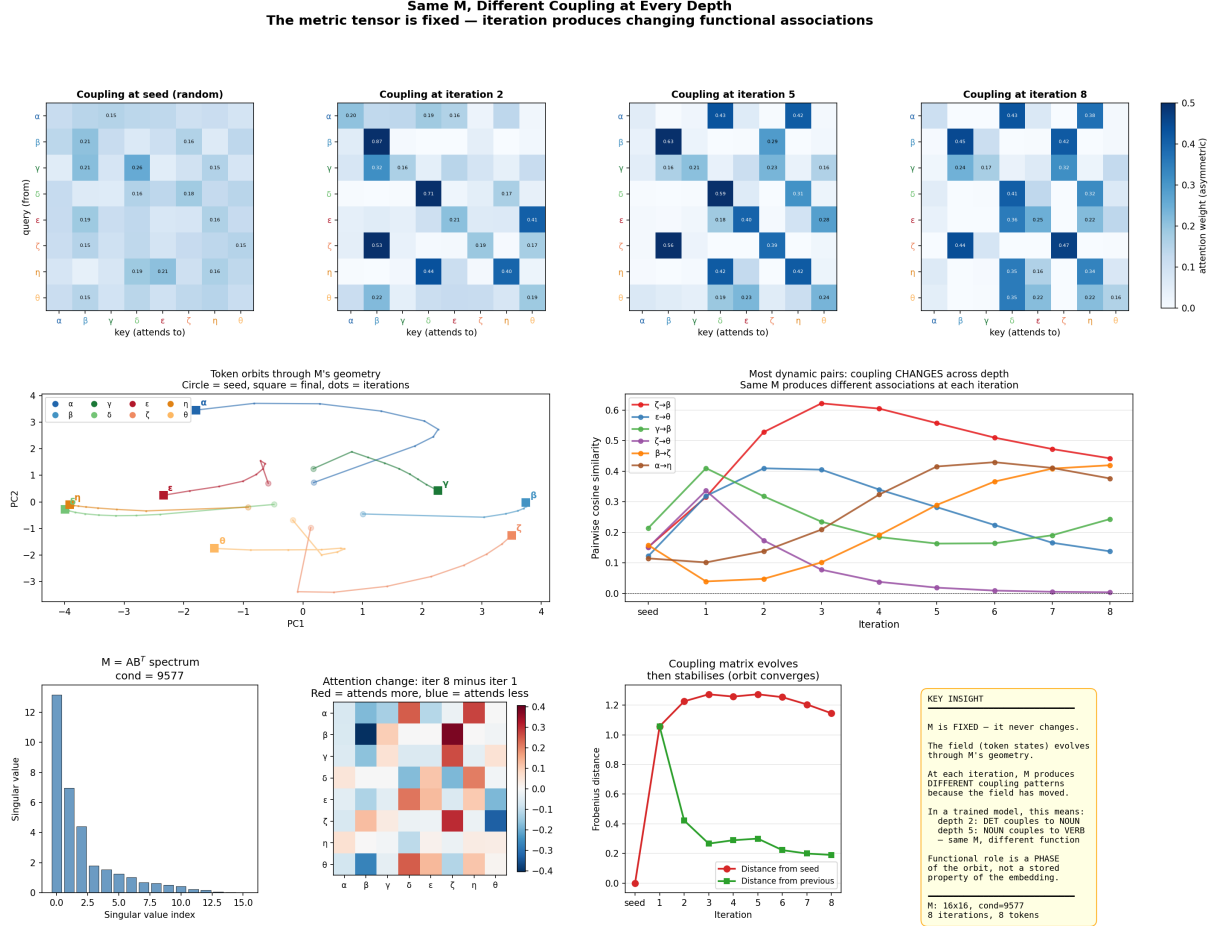


Figure 7: **Same M, different coupling at every depth.** Top row: the asymmetric coupling matrix at seed, iteration 2, iteration 5, and iteration 8. The pattern visibly changes at each depth even though $\mathbf{M}$ is fixed. Bottom left: token orbits in PCA space—seeds start clustered and separate through iteration. Bottom centre-left: the most dynamic token pairs, showing coupling that shifts or reverses across iterations. Bottom centre-right: Frobenius distance of the coupling matrix from the seed—the coupling evolves then begins to stabilise (orbit convergence). *All produced by the same fixed M.*

The coupling heatmaps make the claim vivid: at seed, the attention pattern is nearly uniform (no structure). By iteration 5, specific token pairs have developed strong directional coupling. By iteration 8, the pattern has matured further—but it is *different* from iteration 5. The same $\mathbf{M}$ produced all four matrices. The only thing that changed was the field state $\mathbf{h}$.

Intuition. Think of $\mathbf{M}$ as a fixed crystal. A light ray (the token state) enters at some angle (the seed). Each pass through the crystal refracts it differently, because the ray's angle changes after each pass. After enough passes, the rays have been sorted by the crystal's geometry. The crystal never changed—the light travelled through it.

7.2 Watching a Single Token’s Orbit

To make the iteration concrete, we track a single token α through 10 iterations of $\mathbf{M}$. At each step we record three things:

1. **Position in phase space** (PCA projection of the d -dimensional state vector).
2. **Coupling pattern** (which other tokens does α attend to?).
3. **State vector** (all d activation values).

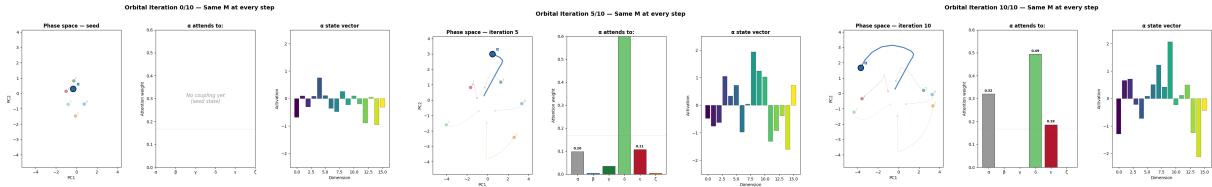


Figure 8: **Three frames from the orbit animation** (full animation: `orbit_animation.gif`). *Left*: Iteration 0 (seed). α is in a random position; no coupling yet. *Centre*: Iteration 5. α has moved through $\mathbf{M}$ ’s geometry; strong coupling to δ has emerged. *Right*: Iteration 10. α has reached a different region; coupling has shifted— α now couples to itself and ϵ in addition to δ . Same $\mathbf{M}$ at every step.

The critical observation: α ’s coupling partners *change* across iterations. At iteration 5, it attends almost exclusively to δ . By iteration 10, the attention has redistributed. In a trained language model, this is how the same shared $\mathbf{M}$ produces determiner→noun coupling at one depth and noun→verb coupling at another. Functional role is not stored in the embedding—it is a *phase of the orbit*.

7.3 Reachability: The Semigroup Image

If $\mathbf{M}$ defines the geometry, what constrains the set of states that iteration can reach? This is the question of **reachability**—the image of the semigroup generated by $\mathbf{M}$.

We initialise 50 random seeds in $\mathbb{R}^d$ and iterate all of them through the same $\mathbf{M}$. Figure 9 shows the result.

This is the **capacity** of the semigroup. A metric tensor $\mathbf{M}$ with k significant singular values can support orbits that live on a $\sim k$ -dimensional manifold. More iterations compress the reachable set further onto $\mathbf{M}$ ’s dominant subspace. This means:

- The **rank of $\mathbf{M}$** bounds the expressiveness of the orbits.
- The **number of iterations L** determines how tightly states are compressed onto the dominant manifold.
- A **low-rank $\mathbf{M}$** forces all tokens onto a small subspace (not enough capacity). A **full-rank $\mathbf{M}$** preserves all directions but may not develop useful structure.
- The sweet spot: $\mathbf{M}$ with a *few dominant and many suppressed* singular values creates a structured manifold where functional roles emerge as regions.

The effective dimensionality after L iterations is an empirical diagnostic for whether $\mathbf{M}$ has learned useful geometry: if it collapses to 1–2 dimensions, $\mathbf{M}$ is degenerate; if it stays at d , $\mathbf{M}$ has not learned to compress; a value in between (say 5–15 for $d = 128$) suggests structured, useful orbits.

7.4 Context Steering: Same Seed, Different Orbit

In a full SharedM model, the orbit of a token depends not only on its seed but on the *context*—the other tokens in the sequence. At each iteration, the value projections W_V (per-layer) and the feed-forward network inject context-dependent information through the coupling that $\mathbf{M}$ defines.

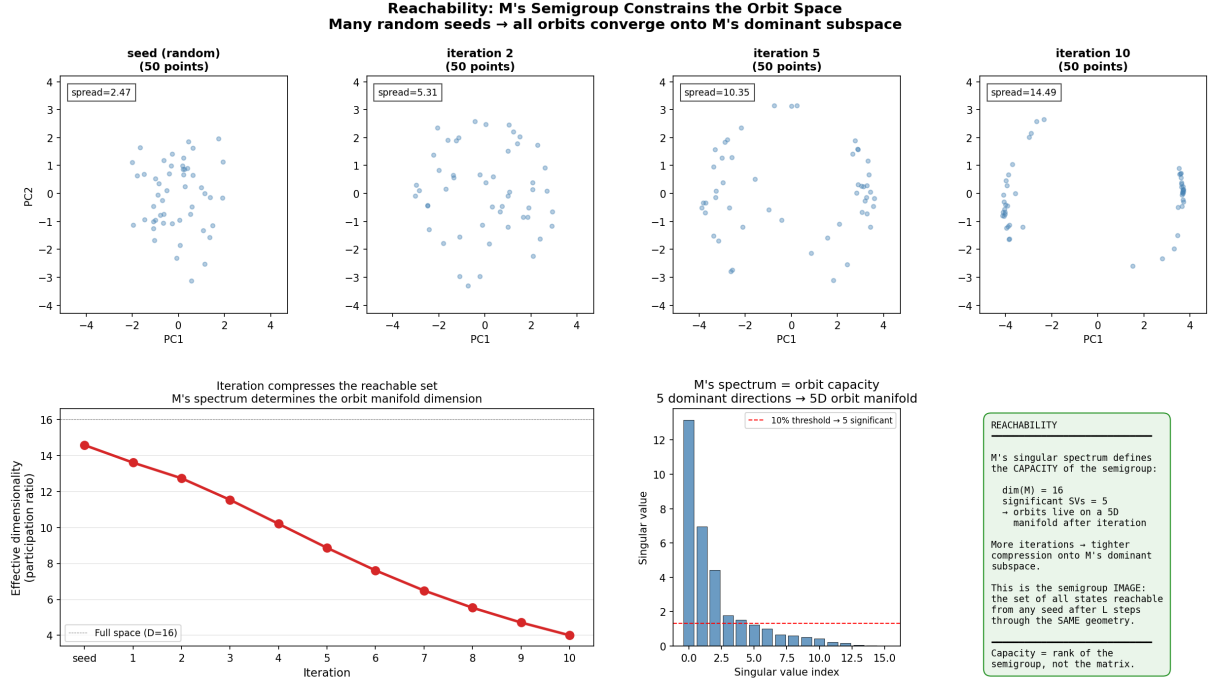


Figure 9: $\mathbf{M}$'s semigroup constrains the orbit space. Top row: 50 random seeds at iterations 0, 2, 5, 10—the cloud contracts onto a lower-dimensional manifold. Bottom left: effective dimensionality (participation ratio of singular values) drops from ~ 14 to ~ 4 , matching $\mathbf{M}$'s 5 significant singular values. Bottom centre: $\mathbf{M}$'s spectrum—the number of large singular values defines the orbit manifold dimension.

- $\mathbf{M}$ determines *who* attends to *whom* (the coupling geometry).
- W_V determines *what information* flows through that coupling.
- Context (surrounding tokens) modulates what is available to flow.

$\mathbf{M}$ is the river. Context is the rudder.

This is why “cat” in “the cat sat” ends up in a different region of representation space than “cat” in “a cat ran on a mat”. The seed is the same. The metric tensor is the same. But the surrounding context steers the orbit to a different destination through $\mathbf{M}$'s geometry.

The coupling matrix is not stored—it is computed. In a standard transformer, each layer independently stores $W_Q^{(\ell)}, W_K^{(\ell)} \in \mathbb{R}^{d \times d}$, giving $2Ld^2$ coupling parameters across L layers. Each layer's coupling matrix $Q^{(\ell)}K^{(\ell)\top}$ is a separate, independently learned pattern. In SharedM, there is a single $\mathbf{M} = AB^\top$ shared across all layers. The coupling matrix at layer ℓ is:

$$\text{Attn}^{(\ell)} = \text{softmax}\left(\frac{(\mathbf{h}^{(\ell)}A)(\mathbf{h}^{(\ell)}B)^\top}{\sqrt{d_h}}\right)$$

where $\mathbf{h}^{(\ell)}$ is the residual stream at depth ℓ . Because $\mathbf{h}^{(\ell)}$ differs at every layer (the field has been updated by all previous iterations), $\mathbf{M}$ produces a *different* $\text{Attn}^{(\ell)}$ at each depth—despite being the same matrix. The coupling patterns in the bottom-centre panel of Figure 10 are three instances of this: same $\mathbf{M}$, different field states (due to different contexts), different resulting attention distributions.

The parameter saving is $2Ld^2 \rightarrow 2d^2$ for coupling, an L -fold compression. But the *capacity* of the resulting coupling is not merely d^2 —it is determined by the semigroup generated by $\mathbf{M}$ over L iterations. Each iteration drives the field to a new region of $\mathbf{M}$'s geometry, producing a coupling matrix that could not have been produced at any other depth. The semigroup's

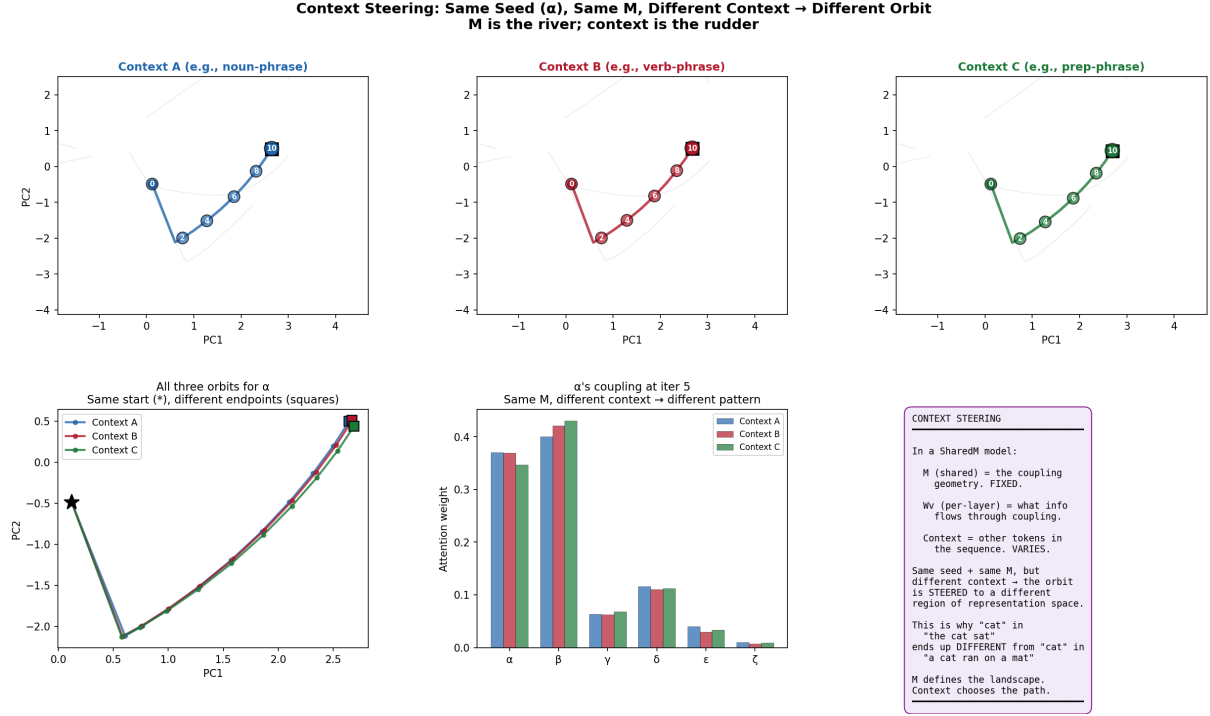


Figure 10: **Context steering**: Same seed token α , same $\mathbf{M}$, three different context signals $\rightarrow$ three different orbits. Top row: each panel shows α 's orbit under a different context. Bottom left: overlay of all three—same start point ($\star$), different endpoints (squares). **Bottom centre (key panel)**: α 's coupling pattern at mid-depth, shown for all three contexts. These bar charts are analogous to a single row of the $QK^\top$ coupling matrix in a standard transformer—the attention distribution that determines what information flows to α . In a standard transformer, each layer stores its own W_Q, W_K to produce this pattern. In SharedM, the *same* $\mathbf{M} = AB^\top$ is driven for L iterations by the semigroup it generates, and the resulting coupling matrix emerges from the interaction of $\mathbf{M}$'s fixed geometry with the evolving field. The semigroup's capacity—the dimensionality of its image after L steps—determines how rich and varied these coupling patterns can be (Section 7.3).

image after L steps spans a space of coupling patterns whose dimensionality grows with L , up to the capacity limit set by $\mathbf{M}$'s spectral structure. Our scaling experiments (Section ??) provide empirical evidence that this computed capacity matches or exceeds the stored capacity of $2L$ independent projection matrices.

Context steering connects directly to the **response heads** W_V in the GRF decomposition (Section ??). The generator $\mathbf{M}$ defines reachable orbits; the response heads select *which* orbit to follow based on context. The reachable set from a given seed—the fan of possible endpoints under all possible contexts—characterises the **functional range** of that token. A token with a large reachable set is functionally versatile (like “set” or “run” in English); one with a small reachable set is functionally rigid (like “the”).

With this visual intuition in hand, we now show what the semigroup looks like *in a real trained model*, then turn to the formal geometry that explains why shared dynamics produce useful structure.

7.5 From the Lab: The Semigroup at 50M Parameters

Everything above used toy examples. Here we open a real checkpoint—SharedM-50M ($d=512$, $L=9$, trained on TinyStories for 8,000 steps)—and look at what $\mathbf{M} = AB^\top$ actually learned.

The probe sentence. We feed the model a single sentence and trace what happens at every layer:

“Once upon a time, there was a little girl named Lily. She loved to play with her dog.”

(21 tokens under GPT-2 tokenization.)

Q-vectors rotate smoothly. Figure 11 (left) shows the cosine similarity between the query vector for “dog” at each pair of layers. Adjacent layers have cosine ~ 0.9 —nearly the same query—but by layer 8 the query has rotated so far that the $L0 \leftrightarrow L8$ cosine is only 0.29. This is the orbit: the same A matrix, applied to a progressively transformed residual stream, smoothly sweeps the query through coupling space.

K-vectors undergo a phase transition. Figure 11 (centre) shows the same analysis for the key vector of “Lily.” The pattern is dramatically different: layers 0–3 form one cluster (cosine 0.67–0.97), layers 4–8 form another (0.70–0.96), and the two clusters are nearly orthogonal ($L0 \leftrightarrow L6 = -0.22$). “Lily” *switches role* mid-network—from a name token to a narrative agent. No layer-specific parameters drive this; the field state alone causes the switch.

Coreference emerges. Figure 12 plots the K-vector cosine between “Lily” and “She” (co-referent) vs “Lily” and “dog” (different entity) across layers. At deeper layers, the co-referent pair is more similar in key space than the non-co-referent pairs. The model discovered that “Lily” and “She” should be findable by similar queries—without any coreference supervision.

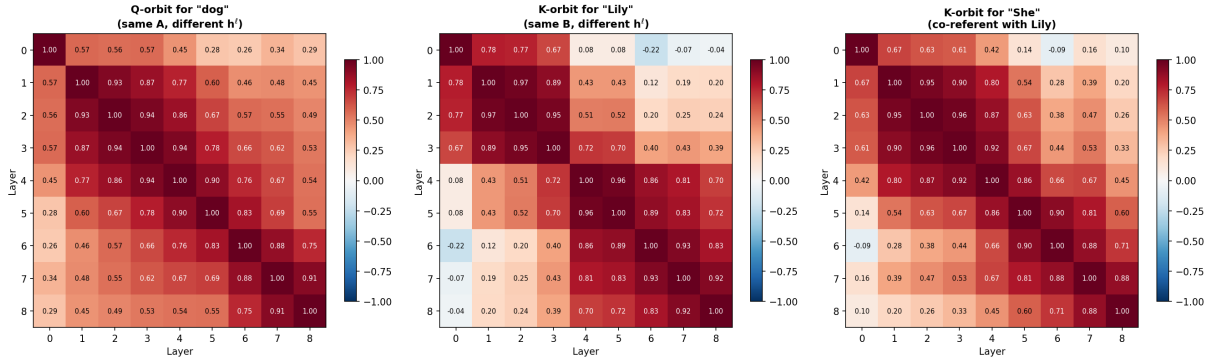


Figure 11: Pairwise cosine similarity of Q/K vectors across 9 layers (head 0) in a 50M-parameter SharedM model. Left: Q for “dog” rotates smoothly (the semigroup orbit). Centre: K for “Lily” shows a phase transition at layer 4. Right: K for “She” shows a similar phase transition, consistent with coreference.

What the coupling geometry looks like. M has effective rank 51 out of 512 dimensions (90% of energy in 51 singular values). During training, this rank *sharpens*: from 66 at step 2,000 to 51 at step 8,000. The coupling geometry crystallises—it discovers its structure, then refines it. This is visible in Figure 13: the singular value spectrum concentrates into fewer, stronger modes as training progresses.

Layer specialisation from a single M . Figure 14 shows the full attention heatmaps (head 0) at all 9 layers. Layer 0 attends to recent context (“dog”, “her”). Layer 1 attends to narrative framing (“Once”, “upon”). Layers 6–8 attend to character identity (“Lily”, “named”, “girl”). The same M produces local attention, structural attention, and entity-focused attention at different depths—driven by the field state, not by parameter diversity.

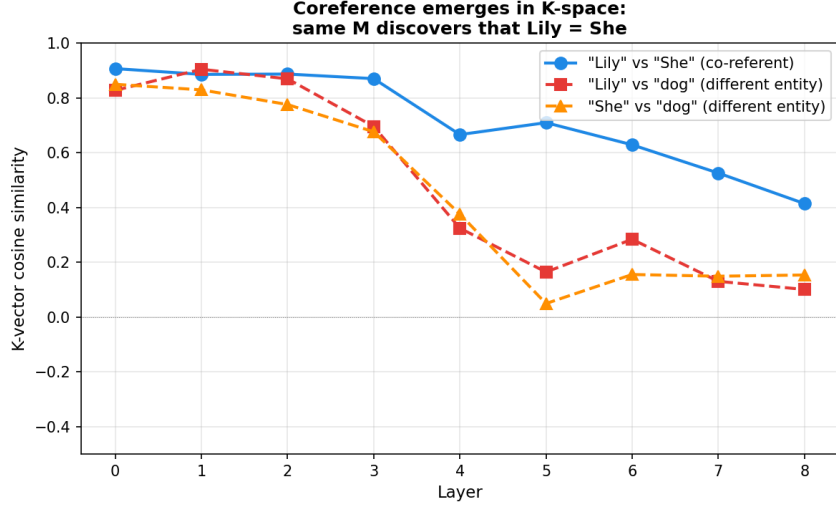


Figure 12: K-space cosine similarity across layers. Co-referent tokens (“Lily” and “She”) converge in key space, while different entities (“Lily” vs “dog”) diverge. Same $\mathbf{M}$, no coreference supervision.



Figure 13: Left: $\mathbf{M}$ ’s singular value spectrum at four training checkpoints—energy concentrates in fewer modes over time. Right: effective rank drops from 66 to 51 while the top singular value grows. The geometry crystallises during training.

The punchline. One 512×512 matrix pair ($\mathbf{A}$, $\mathbf{B}$) produces 9 distinct, semantically meaningful attention patterns. It discovers coreference without supervision. Its spectral structure crystallises during training, sharpening from diffuse to focused. This is what “generators, not trajectories” looks like in practice.

A practical bonus: inspectability. Everything above was extracted from a single 191MB checkpoint file, with a few minutes of CPU computation. No retraining, no ablation experiments, no evaluation datasets.

In a standard transformer, the coupling behaviour is spread across $2L$ independent $\mathbf{W}_Q$, $\mathbf{W}_K$ matrices—there is no single object to inspect. In SharedM, $\mathbf{M} = \mathbf{A}\mathbf{B}^\top$ is that object. From one checkpoint you can compute its eigenspectrum, measure its effective rank, trace how $\mathbf{Q}$ and $\mathbf{K}$ vectors orbit through layers, test what happens at depths the model was never trained on (just iterate $\mathbf{M}$ more times), and project the residual stream onto $\mathbf{M}$ ’s singular vectors to check whether the model is maintaining useful signal.

This makes SharedM models fundamentally more **interpretable** than standard transformers—

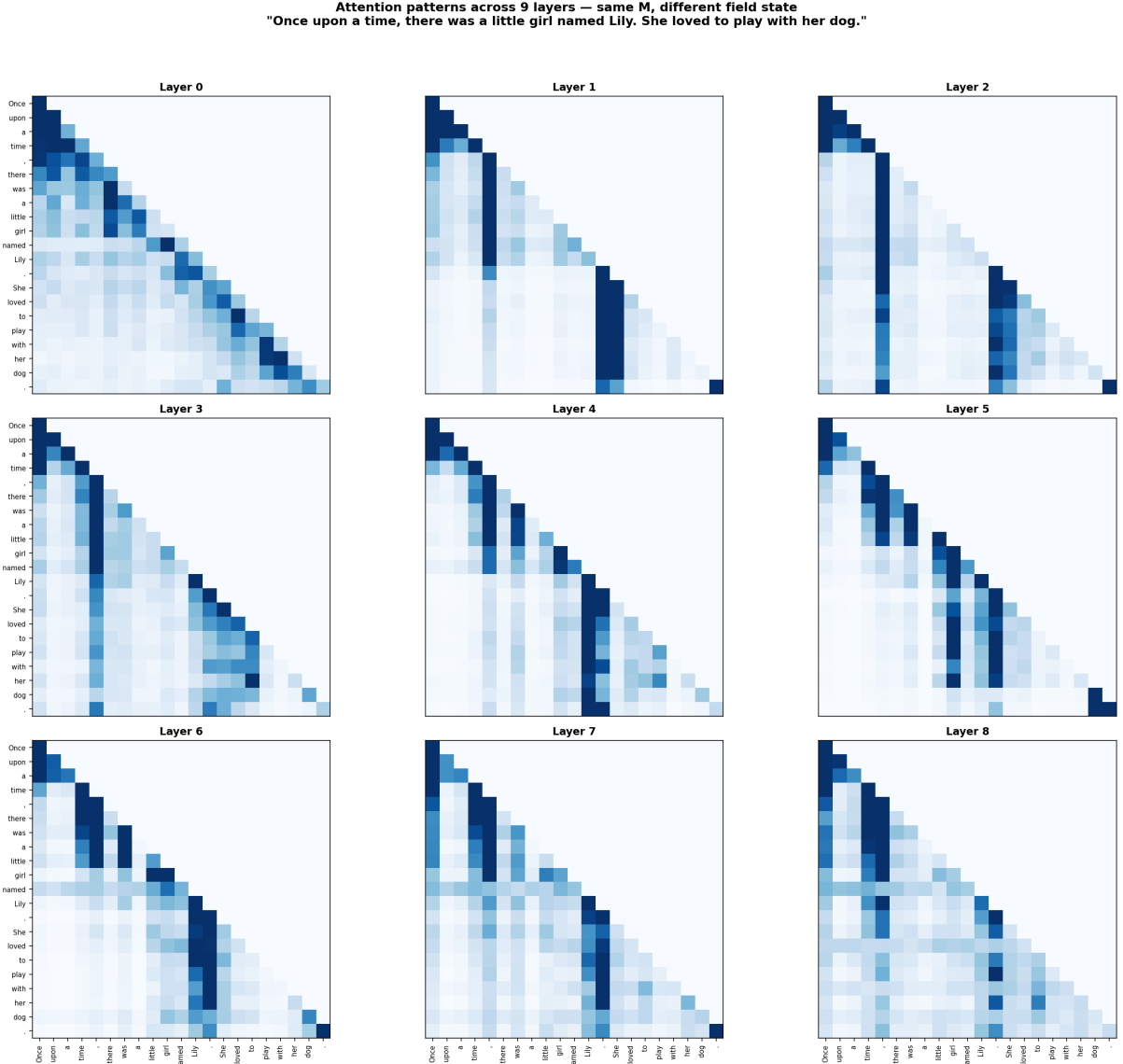


Figure 14: Attention patterns (head 0) at all 9 layers for the same input. Same M , different field state, different coupling at every depth. Early layers are local; late layers focus on character identity.

not because the maths is simpler, but because the coupling geometry is concentrated in one place rather than distributed across layers. The shared tensor is a window into the model’s relational world model [27].

Remark. Realisation: The value of frozen- M is interpretability, not compute savings (#65). When we freeze M and train only response heads, we can watch the response heads adapt to a *fixed* geometric structure. The question changes from “what is the model learning?” (intractable) to “how is the model using the geometry we already understand?” (tractable). At 108M parameters, the V2 model’s M has effective rank 69, with 50% of coupling energy in just 13 singular values. These 13 modes are the model’s relational grammar—the invariant coupling rules that govern how tokens relate at every layer.

8 The Geometry: Why Shared Dynamics Work

This section explains the deeper reason why a single shared function can generate all token embeddings. The answer comes from geometry and dynamical systems theory.

8.1 The Manifold Hypothesis

High-dimensional data (images, text embeddings, etc.) does not fill the entire space $\mathbb{R}^d$. Instead, it concentrates near low-dimensional **manifolds**—curved surfaces embedded in the high-dimensional space.

For language embeddings: $V = 50,000$ tokens each with a $d = 768$ -dimensional vector live in a space of 768 dimensions. But the *effective* dimensionality is much lower—our PCA tests show 90% of variance in ~ 27 directions. The embedding vectors lie on a ~ 27 -dimensional manifold in 768-dimensional space.

8.2 The Dynamical System View

The shared function Φ defines a **vector field** over the embedding space: at every point $\mathbf{x}$, the vector $\Phi(\mathbf{x})$ says “move this way.” A seed $\mathbf{s}_w$ is an initial condition. The orbit $\mathbf{s}_w \rightarrow \mathbf{x}_1 \rightarrow \mathbf{x}_2 \rightarrow \dots$ traces a path through the space, following the vector field. The final point is the embedding.

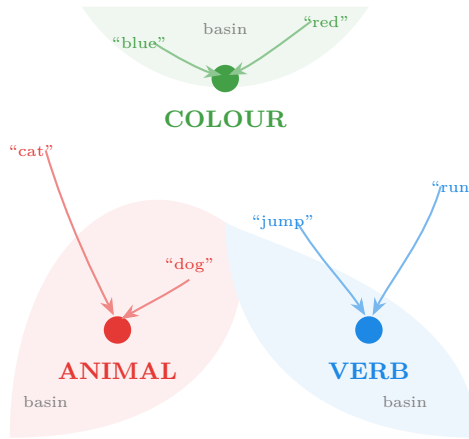


Figure 15: **Attractor landscape.** The shared dynamics Φ define basins of attraction. Tokens of the same semantic role (animals, verbs, colours) flow into the same basin. Different seeds land at different points within the basin (“cat” $\neq$ “dog”), but the basin structure is shared. This is why one function can generate all embeddings: it encodes the *landscape*, and seeds select locations within it.

What this means in language. The basins of attraction correspond to semantic roles: the ANIMAL basin, the ACTION basin, the EMOTION basin. “Cat” and “dog” have nearby seeds that flow into the same basin but settle at different positions within it. “Cat” and “jumped” have distant seeds that flow into different basins entirely. The attractor landscape IS the learned ontology of the vocabulary.

8.3 Contracting and Expanding Directions

Within each basin, the dynamics have two types of directions:

Contracting directions: tokens of the same role converge. “Cat” and “dog” share the ANIMAL subspace; the dynamics push them toward it.

Expanding directions: tokens with different identities separate. “Cat” and “dog” are both animals but different animals; the dynamics push them apart along the identity axis. This dual behaviour—contracting in some directions, expanding in others—is exactly what makes the representation rich enough for downstream processing while remaining compressible.

8.4 Connection to the Shared Metric $\mathbf{M}$

In the broader ORN (Orbital Response Network) programme, the shared coupling tensor $\mathbf{M}$ solves a different problem: it replaces per-layer attention projections (Table 2 from Section 1). Where orbital embeddings ask “can we *generate* token representations from seeds?”, the shared $\mathbf{M}$ asks “can we *reuse* the same coupling rule at every layer?”

The two ideas connect through geometry: $\mathbf{M}$ defines a Riemannian metric on the residual stream (how to measure distance between tokens), and the orbital dynamics Φ operate *within* this metric space (how tokens move). The metric says “which tokens should interact”; the dynamics say “how their representations evolve.” Together, they define a complete geometric system—one shared coupling rule, one shared dynamical law, and minimal per-token seeds.

8.5 Connection to Semigroups

The iterated dynamics have an algebraic structure: if $T(t)$ denotes t iterations of Φ , then:

$$T(s + t) = T(s) \circ T(t)$$

This is the **semigroup property**: composing s steps and t steps is the same as taking $s + t$ steps. Why does this matter?

Intuition. The semigroup property means the model learns a *law*—a consistent rule of evolution—not a sequence of T unrelated transforms. If each iteration were a different function $(\Phi_1, \Phi_2, \dots)$, the model would need to learn T separate rules and how they compose. With shared Φ , it learns one rule, and composition is automatic.

This is not just elegant. Deep-OSG [40] results show that enforcing semigroup structure *reduces data dependency*: the model learns faster and generalises better because the search space is constrained to consistent dynamics. The mathematical foundations are established in operator theory [27]; the deep learning connection was first drawn by Mehta & Schwab [41] (exact mapping between renormalisation group and deep learning) and formalised in Lin & Tegmark [42] (cheap learning works because physics has symmetries).

8.6 The Koopman Connection: Dynamics Instead of States

There is a deep mathematical reason why shared dynamics work, and it comes from an 80-year-old idea that was recently revived by machine learning.

The problem with states. A standard transformer stores *states*: KV caches hold the key and value projections at every layer for every token. This is memory—a snapshot of what the network computed. If you want to extend the computation (think longer, add more context), you need more memory. The states do not compose: knowing the KV cache at layer 5 and the KV cache at layer 10 does not tell you what the KV cache at layer 15 would be.

The Koopman insight: look at the dynamics, not the states. In 1931, B. O. Koopman and John von Neumann proved something remarkable: any nonlinear dynamical system can be represented as a *linear* operator acting on functions of the state, rather than on the state itself. The trick: instead of tracking $x_{t+1} = F(x_t)$ (nonlinear, hard to analyse), define the Koopman

operator $\mathcal{K}$ as $\mathcal{K}[g] = g \circ F$ —“apply the observable g after one step of dynamics.” $\mathcal{K}$ is linear even when F is nonlinear, because composition distributes over addition: $\mathcal{K}[\alpha g + \beta h] = \alpha g \circ F + \beta h \circ F$.

The price: $\mathcal{K}$ is infinite-dimensional (the space of all functions of x). The revival: machine learning can learn finite-dimensional approximations—a small matrix K that captures the important dynamics.

M IS a Koopman approximation. The ORN’s shared coupling matrix $\mathbf{M}$ is exactly this: a finite-dimensional linear operator that evolves representations through dynamics.

- The embedding layer is the *encoder*: it maps tokens into the space of “observables” (the residual stream).
- $\mathbf{M}$ evolves observables linearly: $h_{\ell+1} \approx h_{\ell} + \mathbf{M} \cdot h_{\ell}$ (the residual update).
- The output head is the *decoder*: it maps from observable space back to predictions.
- The response heads at each layer provide the nonlinearity that makes this approximation expressive despite $\mathbf{M}$ being applied linearly.

Composition is free: applying $\mathbf{M}$ for s steps then t steps IS $s+t$ steps. Multi-step reasoning IS matrix powering. The eigenvalues of $\mathbf{M}$ encode the *timescales* of the dynamics: large eigenvalues mean fast modes (rapid prediction), small eigenvalues mean slow modes (gradual evolution, deliberation). The eigenvectors encode the *independent directions* along which dynamics decouple.

Intuition. A standard transformer stores what happened (states, KV cache). The ORN stores how things evolve (the generator $\mathbf{M}$). This is the difference between memorising a trajectory and learning the law of motion. Given the law, you can generate any trajectory. Given a trajectory, you can only replay it.

Why this matters for language. Language has dynamics at multiple timescales: word-to-word coupling (syntax, fast), sentence-to-sentence coupling (discourse, medium), topic-level coupling (narrative, slow). A Koopman-style $\mathbf{M}$ captures all of these as different eigenmodes:

- Fast modes (large $|\lambda_i|$): local grammar, adjacent-word coupling.
- Medium modes: clause structure, argument binding.
- Slow modes (small $|\lambda_i|$): topic drift, narrative arc, long-range coherence.

All live in the same matrix. The response heads at each layer read the relevant modes for their depth: early layers use fast modes (local syntax), deep layers use slow modes (discourse structure). This is why $\mathbf{M}$ crystallises to rank ~ 20 for web text: roughly 20 independent timescales capture the coupling dynamics of natural language.

The connection to thinking. The fast modes converge quickly: a few iterations through $\mathbf{M}$ and the prediction is sharp. The slow modes evolve gradually: many iterations are needed to see their effect. Fast prediction uses the fast modes (System 1). Slow deliberation needs the slow modes (System 2). The challenge is that prediction training—which optimises for single-step accuracy—crushes the slow modes because they don’t contribute to next-token loss. Protecting these slow Koopman modes is the key to building a system that can both predict fluently and reason deliberately, using the same $\mathbf{M}$ for both.

9 A Domain Where You Can See It: Signal Processing

The best way to build intuition for orbital embeddings is to see the same idea in a domain where it is well understood: Fourier analysis.

9.1 Fourier Series

Any periodic signal can be decomposed into a sum of sinusoids:

$$f(t) = a_0 + \sum_{n=1}^N [a_n \cos(n\omega t) + b_n \sin(n\omega t)]$$

Each sinusoid is characterised by an amplitude and a phase. The frequencies $\omega, 2\omega, 3\omega, \dots$ are harmonics.

9.2 Each Sinusoid Is an Orbit

A sinusoid at frequency ω is the orbit of the **rotation operator**:

$$e^{i\omega t} = \cos(\omega t) + i \sin(\omega t)$$

The “seed” is the complex amplitude $A_n = a_n - ib_n$ (containing amplitude and phase). The “dynamics” is rotation at frequency ω : multiply by $e^{i\omega\Delta t}$ at each time step.

The **shared dynamics** is rotation itself—the same operation for all frequencies. Only the seed (amplitude, phase) and the frequency differ between components.

9.3 The Analogy

	Fourier	Orbital Embeddings
Stored representation	Signal samples $f(t_1), f(t_2), \dots$	Embedding table $E[w]$
Compressed code (seed)	Fourier coefficients (a_n, b_n)	Seed vectors $\mathbf{s}_w$
Shared dynamics	Rotation: $e^{i\omega t}$	Shared block: Φ
Generation	Inverse FFT	Iterate Φ on seeds
Structure exploited	Periodicity	Manifold structure

9.4 Concrete Example: Square Wave

A square wave can be stored as N sample values (lookup: N numbers). Or it can be represented as Fourier coefficients: a square wave with K harmonics needs only K coefficients, and the reconstruction formula (shared dynamics) is the same for any signal. As K grows, the approximation improves. This is exactly the compute-vs-memory tradeoff: store the samples (memory) or store the coefficients and recompute (compute).

For natural language embeddings, the “Fourier coefficients” are the seeds, and the “reconstruction formula” is the shared block Φ . The quality of the approximation depends on how structured the target embeddings are—just as Fourier compression depends on how smooth the signal is.

10 Building It: Orbital Embeddings in PyTorch

Here is the complete implementation, simplified to its essentials.

Listing 2: Orbital embedding: the complete module.

```

1 import torch
2 import torch.nn as nn
3
4 class OrbitalEmbedding(nn.Module):
5     """Replace nn.Embedding(V, d) with seeds + shared dynamics."""
6
7     def __init__(self, V, d_model, d_seed=16, T=2):
```

```

8     super().__init__()
9     self.T = T
10
11     # Per-token: tiny seed vectors
12     self.seeds = nn.Embedding(V, d_seed) # V x d_seed
13
14     # Shared: project seed -> model space
15     self.proj = nn.Linear(d_seed, d_model) # d_seed x d_model
16
17     # Shared: the orbital dynamics (used T times)
18     self.orbit = nn.Sequential(
19         nn.LayerNorm(d_model),
20         nn.Linear(d_model, d_model),
21         nn.GELU(),
22         nn.Linear(d_model, d_model),
23     )
24
25     def forward(self, token_ids):
26         # Step 1: look up tiny seeds
27         s = self.seeds(token_ids) # (batch, seq) -> (batch, seq, d_seed)
28
29         # Step 2: project into model space
30         x = self.proj(s) # (batch, seq, d_model)
31
32         # Step 3: iterate shared dynamics T times
33         for t in range(self.T):
34             x = x + self.orbit(x) # residual connection
35
36         return x # (batch, seq, d_model)

```

10.1 Line by Line

Line 11: `self.seeds` A standard embedding table, but with $d_{\text{seed}} = 16$ instead of $d_{\text{model}} = 128$. This is the per-token storage: $V \times 16$ parameters instead of $V \times 128$. A $8\times$ reduction in per-token parameters.

Line 14: `self.proj` A single linear layer that maps the 16-dimensional seed into the 128-dimensional model space. Shared across all tokens. Cost: $16 \times 128 = 2,048$ parameters.

Lines 17–21: `self.orbit` The shared dynamics block. LayerNorm + Linear + GELU + Linear. This is the “physics” that generates embeddings from seeds. Used T times but stored once. Cost: $\sim d_{\text{model}}^2$ parameters ($\sim 40\text{K}$ for $d = 128$, $\sim 300\text{K}$ for $d = 384$).

Line 25: `self.seeds(token_ids)` The only per-token operation: a standard embedding lookup, but in the tiny seed space.

Line 28: `self.proj(s)` Project from seed space to model space. All tokens use the same projection matrix.

Lines 31–32: `x = x + self.orbit(x)` The orbital iteration. The residual connection ($x + \dots$) ensures the iteration is stable: each step refines the previous, rather than replacing it. The same `self.orbit` is called T times—weight sharing across depth.

10.2 The Output Head

For language modelling, the output head maps the final hidden state back to a probability distribution over tokens. With orbital embeddings, we project back into seed space and dot

with all seed vectors:

Listing 3: Output head for orbital embeddings.

```

1 class OrbitalOutputHead(nn.Module):
2     def __init__(self, d_model, d_seed, seeds_weight):
3         super().__init__()
4         self.proj_out = nn.Linear(d_model, d_seed)
5         self.seeds_weight = seeds_weight # tied with input seeds
6
7     def forward(self, hidden):
8         # Project back to seed space
9         z = self.proj_out(hidden) # (batch, seq, d_seed)
10        # Dot product with all seed vectors
11        logits = z @ self.seeds_weight.T # (batch, seq, V)
12        return logits

```

10.3 Parameter Counting

Component	Lookup	Orbital ($d_s=16, T=2$)
Per-token storage	$V \times d = 6.4\text{M}$	$V \times d_s = 804\text{K}$
Projection	—	$d_s \times d = 2\text{K}$
Shared dynamics (Φ)	—	$\sim 2d^2 = 40\text{K}$
Total embedding	6.4M	846K
Compression	1×	7.6×

The saved 5.5M parameters are not wasted—they are **redistributed** to make the model wider. Instead of a model with $d = 36$ (because the embedding ate all the parameters), we get a model with $d = 128$ or higher. The model backbone—attention layers, feed-forward networks—gets 5.5M more parameters to work with.

11 Experiments: What We Found on TinyStories

11.1 The Parameter Dominance Problem

At a total budget of 2M parameters with vocabulary $V = 50,257$:

- The embedding table alone wants $V \times d$ parameters.
- If $d = 36$ (squeezed to fit): embedding = $50,257 \times 36 = 1.81\text{M} = \mathbf{91\% \text{ of the model}}$.
- The transformer backbone gets only 180K parameters: $d_{\text{head}} = 9$, barely functional.

The model is strangled by its own lookup table.

11.2 Experiment 5a Results

We tested four configurations at 2M total parameters on TinyStories (a children’s story dataset):

Model	Val Loss	Emb %	d_{model}	vs. Baseline
BASELINE (lookup)	3.389	91%	36	—
LOWRANK ($T=0$) ¹	3.455	40%	~ 80	−1.9%
ORBITAL-T1 ($T=1$)	3.129	40%	~ 384	+7.7%
ORBITAL-T2 ($T=2$)	3.593	41%	~ 96	−6.0%

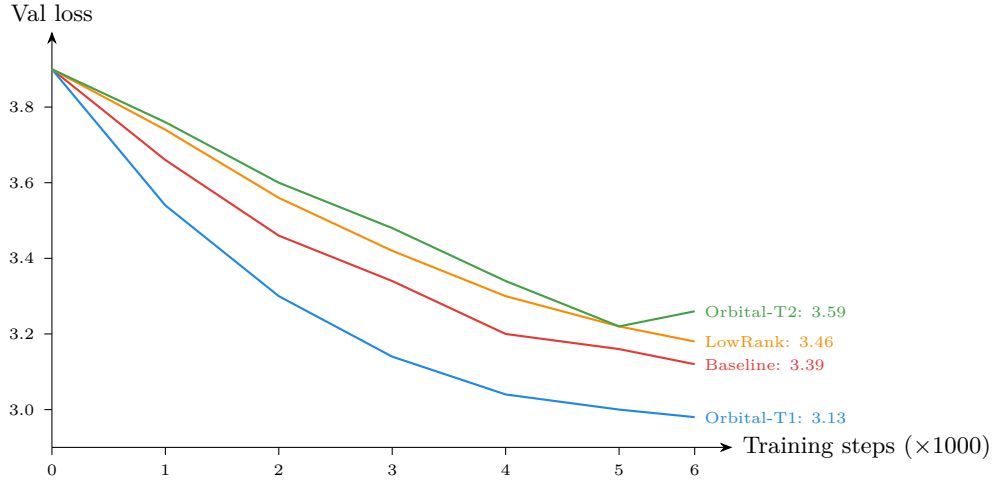


Figure 16: **Validation loss curves on TinyStories at 2M parameters.** Orbital-T1 (blue) beats the baseline (red) by 7.7% in validation loss. It achieves this by compressing the embedding table and redistributing parameters to a wider model backbone. The baseline is crippled by its own embedding table (91% of parameters), leaving only $d_{\text{head}} = 9$ for attention.

11.3 Interpreting the Results

The **7.7% win** for Orbital-T1 is not because orbital embeddings are inherently better—it is because they are *smaller*, freeing parameters for the actual model. At 2M total parameters:

- The baseline spends 91% on embeddings, getting a model with $d = 36$ ($d_{\text{head}} = 9$).
- Orbital-T1 spends 40% on embeddings, getting a model with $d \approx 384$ ($d_{\text{head}} \approx 96$).
- The wider model simply has more capacity for language modelling.

In TinyStories... The baseline at 2M params has $d = 36$, meaning each token embedding is a 36-dimensional vector. But TinyStories has clear semantic structure—perhaps 10–20 functional roles (characters, actions, objects, emotions, sizes, colours, structure words). An orbital model with $d_{\text{seed}} = 16$ can represent ~ 16 independent roles in seed space, which is close to the actual semantic complexity. The freed parameters go to wider layers that can process context more effectively.

11.4 Honest Caveats

At 2M parameters, the baseline is severely crippled. A d_{head} of 9 is barely functional—this is not a fair comparison of embedding quality, but of parameter allocation.

Remark. Orbital embeddings: pedagogical, not practical (yet). The embedding memoisation story in Part 1 of this tutorial illustrates the *principle* of replacing storage with computation. But in practice, orbital embeddings do not improve prediction quality or training efficiency over standard lookup tables. At $d \geq 512$, the embedding table is a small fraction of total parameters (6% at GPT-2 scale), and the compute overhead of iterating Φ is not justified. The V2 and V3 models that beat GPT-2 use standard tied embeddings, not orbital embeddings. The real payoff of the “share and iterate” principle is in **coupling memoisation** (shared **M**) and **FFN memoisation** (shared wide FFN)—Part 2 and beyond—where the redundancy is much larger and the compression delivers real benchmark improvements. We include orbital embeddings because they are the simplest possible example of the core idea, and because they may become relevant for very large vocabularies or unseen-token handling in future work.

11.5 Experiment 5b: The 10M Scale — A Fair Fight

The 2M result is confounded: the baseline has $d_{\text{head}} = 9$ (barely functional). At 10M parameters, the baseline gets $d_{\text{head}} = 44$ —a genuine model. As expected, the orbital embedding advantage disappears at this scale—confirming that the 2M result was about parameter allocation, not embedding quality.

Model	d_{model}	L	Val Loss	vs. Baseline
BASILINE (lookup)	176	3	2.921	—
LOWRANK ($T=0$)	308	8	3.908	−33.8%
ORBITAL-T1 ($T=1$)	476	3	3.008	−3.0%
ORBITAL-T2 ($T=2$)	296	8	3.202	−9.6%

The bottleneck gets worse at scale. LOWRANK lost only 1.9% at 2M but 33.8% at 10M. The functional baseline can *use* the information in full-width embeddings; the 16-dimensional pipe becomes binding. Good news: this means the embedding tables carry real information that dynamics can recover.

Dynamics matter more at scale. ORBITAL-T1 recovers 91% of the gap between LOWRANK and the baseline. The harder the compression problem, the more nonlinear unfolding matters.

Width beats depth. T1 ($d = 476$, $L = 3$) beats T2 ($d = 296$, $L = 8$). The initial projection from 16 dimensions needs room to unfold—a wide single step outperforms a deep narrow one.

Cross-scale comparison.

Model	2M (vs. Baseline)	10M (vs. Baseline)
LOWRANK ($T=0$)	−1.9%	−33.8%
ORBITAL-T1 ($T=1$)	+7.7%	−3.0%
ORBITAL-T2 ($T=2$)	−6.0%	−9.6%

LOWRANK degrades catastrophically (compression alone fails at scale), while the orbital models scale normally. The dynamics Φ are doing the heavy lifting.

The 3% gap is d_{seed} , not dynamics. Three pieces of evidence point to seed dimensionality as the remaining bottleneck: (1) CPU gap tests show $d_{\text{seed}} = 16$ captures only 50% of bigram transition variance; (2) reconstruction cosine similarity is 0.997 (the dynamics themselves are not the problem); (3) 91% of the LOWRANK–baseline gap is recovered by dynamics alone. The residual 3% is the information that 16 seed dimensions simply cannot encode.

11.6 CPU Theory Tests

Our CPU-only theory tests (which measure embedding reconstruction quality without end-to-end training) show:

- Structured targets (circle, spiral, clustered): ~ 38 shared parameters achieve 0.985 cosine similarity.
- Random targets: $>66\text{K}$ parameters needed—worse than storing the table.

The message: orbital embeddings exploit structure. If the structure is there (and in natural language, it is), the compression works.

12 What Is Learned Differently

12.1 Lookup: Independent Vectors

A standard embedding table stores V independent vectors. There is no mechanism that forces “cat” and “dog” to be similar—this only happens if the training gradients happen to push them together. If training is short, or the data is noisy, the similarity structure may be imperfect. Each vector is on its own.

12.2 Orbit: A Shared Landscape

Orbital embeddings enforce structure through the shared dynamics Φ . Because Φ is continuous (neural networks are continuous functions), nearby seeds produce nearby embeddings:

$$\|\mathbf{s}_{\text{cat}} - \mathbf{s}_{\text{dog}}\| \text{ small} \implies \|\Phi^T(\mathbf{s}_{\text{cat}}) - \Phi^T(\mathbf{s}_{\text{dog}})\| \text{ controlled.}$$

The model *must* arrange semantically related tokens as nearby seeds if they should have similar embeddings. This is an **inductive bias**: the architecture forces structure that the lookup table must learn from scratch.

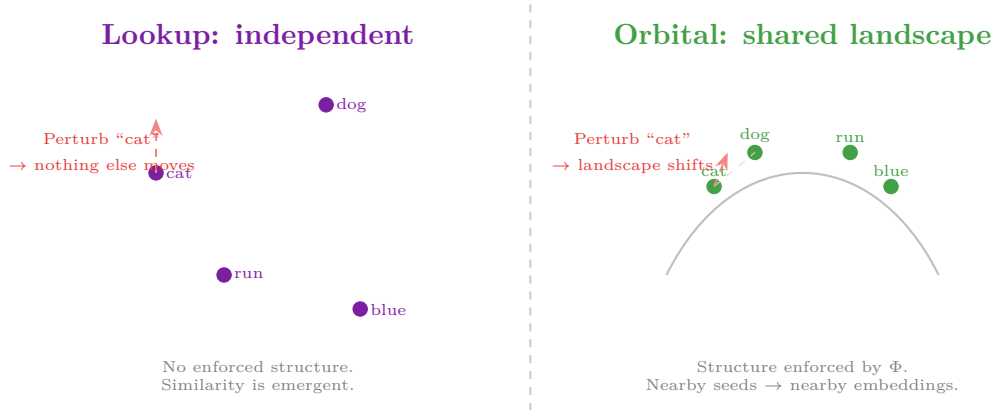


Figure 17: **Geometric coherence.** **Left:** lookup embeddings are independent; perturbing one changes nothing else. **Right:** orbital embeddings share a landscape; perturbing the dynamics Φ moves all tokens coherently, and nearby seeds produce nearby embeddings by the continuity of Φ .

What this means in language. Concretely: if you update the orbital dynamics Φ to better represent “cat”, the representations of “dog”, “bird”, and “fish” also improve—because they share the same dynamics and flow through the same ANIMAL basin. In a lookup table, improving “cat” does nothing for “dog”. This is why orbital embeddings should generalise better: learning about one member of a category teaches the model about all members.

12.3 The Generative View

There are three levels of description for token representations:

D (seeds): The native representation. Compact, structured, in seed space.

f (dynamics): The physics. The shared rule Φ that generates embeddings.

E (lookup table): A materialised cache. $E = f(D)$: just the output of running the dynamics on all seeds.

In this view, the standard embedding table E is a *cache*—a precomputed lookup that avoids running the dynamics at inference time. The orbital embedding deletes the cache and recomputes from (D, f) every time. On modern hardware, the recomputation is fast (compute is cheap) and the cache deletion saves massive memory.

12.4 New Tokens: Interpolation in Seed Space

A powerful consequence of orbital embeddings: they can generate sensible vectors for **tokens they have never seen**.

The problem with lookup tables. If “kitten” is not in the embedding table, its vector is random noise. There is no mechanism to say “kitten is like cat”—the table has no structure connecting entries. To add a new token, you must retrain or fine-tune.

The orbital alternative. If “cat” has seed $\mathbf{s}_{\text{cat}}$ and “dog” has seed $\mathbf{s}_{\text{dog}}$, we can interpolate:

$$\mathbf{s}_{\text{new}} = \frac{\mathbf{s}_{\text{cat}} + \mathbf{s}_{\text{dog}}}{2}$$

Then run the shared dynamics: $\hat{\mathbf{e}}_{\text{new}} = \Phi^T(\text{proj}(\mathbf{s}_{\text{new}}))$. Because Φ is continuous, a seed midway between “cat” and “dog” produces an embedding midway between the animal representations—*without any retraining*.

Experimental verification. We built 64-dimensional co-occurrence embeddings for 135 TinyStories tokens (animals, people, actions, emotions, colours, sizes, objects, structure words), then trained an orbital reconstructor ($d_s = 16$, $T = 2$). For each pair of tokens, we averaged their seeds and ran the dynamics:

Interpolation	cos to midpoint	Nearest token	Category
(cat + dog) / 2	0.43	dog	animals
(happy + sad) / 2	0.86	sad	emotions
(red + blue) / 2	0.84	red	colours
(ran + jumped) / 2	0.65	jumped	actions
(big + small) / 2	0.89	long	sizes
(cat + happy) / 2	0.97	cat	cross-category

Every within-category interpolation lands in the correct category. The midpoint of “cat” and “dog” in seed space produces an embedding nearest to “dog” (an animal). The midpoint of “happy” and “sad” lands nearest “sad” (an emotion). The midpoint of “big” and “small” lands on “long” (a size word).

A lookup table cannot do this. Averaging two rows of a lookup table produces a vector with no guarantee of landing near semantically related tokens. The orbital approach preserves structure because Φ is continuous: nearby seeds $\rightarrow$ nearby embeddings.

The manifold picture holds up. In the same experiment, we visualised all 135 tokens via PCA projection. The orbital embeddings ($d_s = 4$, $T = 2$, cosine similarity 0.90) preserve the cluster structure of the original co-occurrence embeddings: animals cluster together, emotions cluster together, structure words cluster together. With **only 4-dimensional seeds**, the shared dynamics unfold the tokens into 64 dimensions while maintaining semantic grouping.

Config	Cosine sim	Cluster structure?
$d_s = 4, T = 0$ (linear projection)	0.34	No — collapsed
$\mathbf{d}_s = 4, \mathbf{T} = 2$ (orbit)	0.90	Yes — categories preserved
$d_s = 16, T = 0$ (linear projection)	0.60	Partial
$d_s = 16, T = 2$ (orbit)	0.90	Yes

Intuition. The dynamics Φ are the key. Without them ($T = 0$), even 16 seed dimensions cannot reconstruct 64-dimensional embeddings well. With just 2 iterations of shared dynamics, **4 seed dimensions suffice** to reconstruct the full cluster structure. The manifold picture is real: the shared dynamics encode the shape of the embedding manifold, and the seeds position tokens on it.

13 The ORN: Orbital Attention (Replacing Table 2)

So far we have replaced the *embedding table* (Table 1). Now we tackle the more interesting target: the *attention projections* (Table 2). This is the core of the Orbital Response Network (ORN).

Rather than starting with the architecture and working backwards to justification, we will build it from scratch: start with concrete numbers, discover the redundancy empirically, and let the design emerge.

13.1 Attention with Numbers

Let us begin with the simplest possible attention computation. Four tokens—“The”, “cat”, “sat”, “down”—and an embedding dimension of $d = 4$.

Step 1: The embedding matrix. Each token gets a 4-dimensional vector. These are made-up numbers chosen for clarity, not realism.

$$E = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{pmatrix} \quad \begin{array}{l} \leftarrow \text{“The”} \\ \leftarrow \text{“cat”} \\ \leftarrow \text{“sat”} \\ \leftarrow \text{“down”} \end{array}$$

Step 2: Query and key projections. A single attention layer has two learned matrices, W_Q and W_K , both 4×4 . Let us use:

$$W_Q = \begin{pmatrix} 0.5 & 0 & 0 & 0 \\ 0 & 0.5 & 0 & 0 \\ 0 & 0 & 0.3 & 0 \\ 0 & 0 & 0 & 0.3 \end{pmatrix}, \quad W_K = \begin{pmatrix} 0.4 & 0 & 0 & 0 \\ 0 & 0.6 & 0 & 0 \\ 0 & 0 & 0.2 & 0 \\ 0 & 0 & 0 & 0.2 \end{pmatrix}$$

These are diagonal for pedagogical transparency—real models learn dense matrices.

Step 3: Compute queries and keys.

$$Q = E \cdot W_Q = \begin{pmatrix} 0.5 & 0 & 0.3 & 0 \\ 0 & 0.5 & 0 & 0.3 \\ 0.5 & 0.5 & 0 & 0 \\ 0 & 0 & 0.3 & 0.3 \end{pmatrix}, \quad K = E \cdot W_K = \begin{pmatrix} 0.4 & 0 & 0.2 & 0 \\ 0 & 0.6 & 0 & 0.2 \\ 0.4 & 0.6 & 0 & 0 \\ 0 & 0 & 0.2 & 0.2 \end{pmatrix}$$

Row i of Q is the query for token i : “what am I looking for?” Row j of K is the key for token j : “what do I offer?”

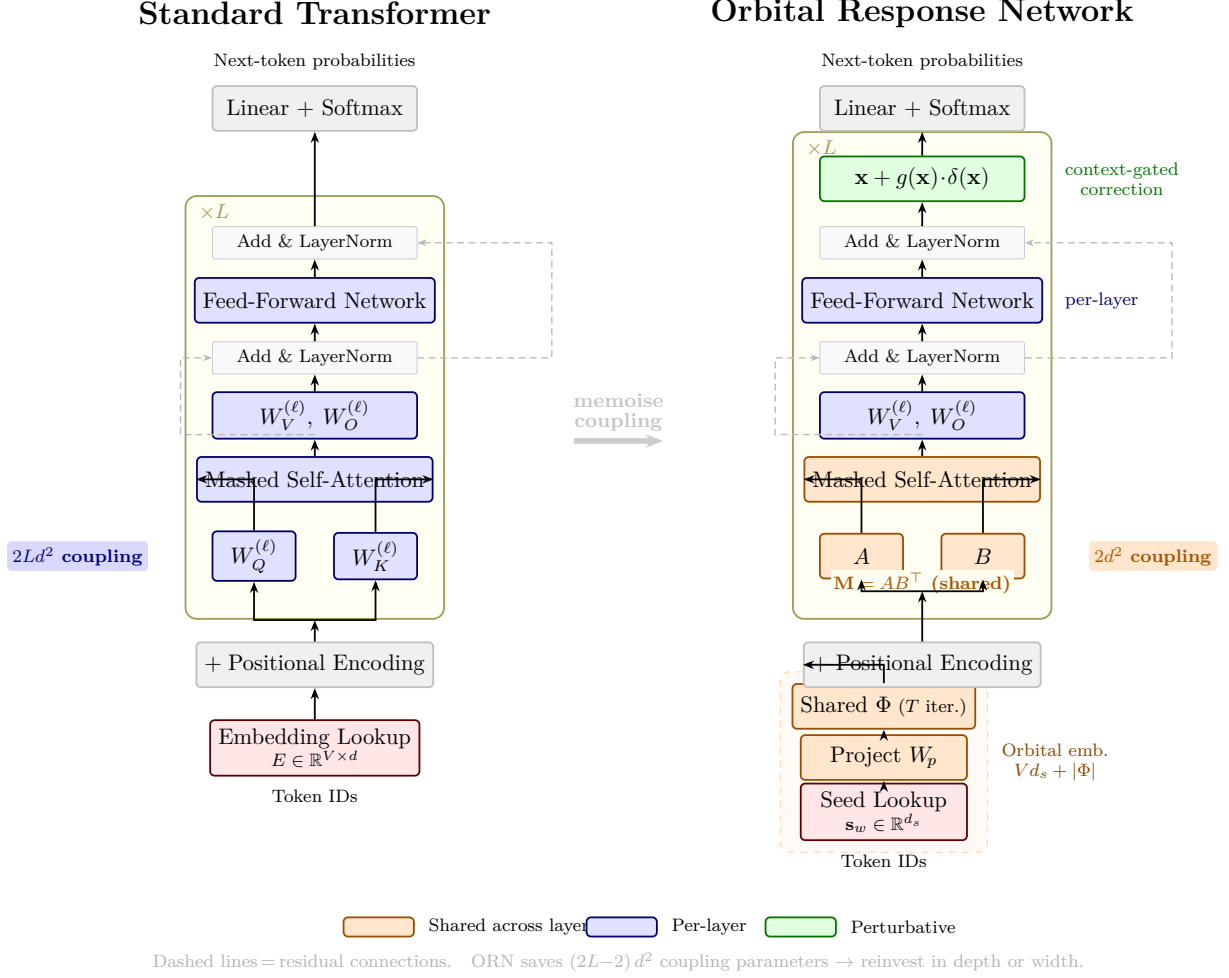


Figure 18: Architecture comparison: Standard Transformer (left) vs. Orbital Response Network (right). The ORN replaces $2L$ per-layer projections $W_Q^{(\ell)}, W_K^{(\ell)}$ (blue) with a single shared pair A, B (orange), compressing $2Ld^2 \rightarrow 2d^2$ coupling parameters. Per-layer response heads and FFN are unchanged. Optional extensions: *orbital embeddings* replace the lookup table with seeds + shared dynamics; *perturbative corrections* add context-gated refinements.

Step 4: Attention scores. The raw score between token i (query) and token j (key) is the dot product $Q_i \cdot K_j$, scaled by $1/\sqrt{d} = 1/2$:

$$S = \frac{Q K^\top}{\sqrt{4}} = \frac{1}{2} \begin{pmatrix} 0.26 & 0.00 & 0.20 & 0.06 \\ 0.00 & 0.36 & 0.30 & 0.06 \\ 0.20 & 0.30 & 0.38 & 0.00 \\ 0.00 & 0.06 & 0.00 & 0.10 \end{pmatrix} = \begin{pmatrix} 0.130 & 0.000 & 0.100 & 0.030 \\ 0.000 & 0.180 & 0.150 & 0.030 \\ 0.100 & 0.150 & 0.190 & 0.000 \\ 0.000 & 0.030 & 0.000 & 0.050 \end{pmatrix}$$

Step 5: Softmax $\rightarrow$ attention weights. Applying row-wise softmax (each row sums to 1):

$$\alpha = \text{softmax}(S) \approx \begin{pmatrix} \mathbf{0.278} & 0.244 & 0.270 & 0.252 \\ 0.229 & \mathbf{0.274} & 0.266 & 0.236 \\ 0.248 & 0.261 & \mathbf{0.271} & 0.224 \\ 0.240 & 0.247 & 0.240 & \mathbf{0.256} \end{pmatrix}$$

Step 6: Reading the weights. Row 2 (“cat”) has its highest weight on column 2 (itself: 0.274) and column 3 (“sat”: 0.266). In a real model with larger embeddings and more training, these differences would be sharper—“cat” might place weight 0.7 on “The” (its determiner) and 0.1 on everything else.

The key observation:

Intuition. W_Q and W_K together define the **coupling rule**—which tokens attend to which. This is entirely separate from W_V , which defines **what information** to extract from the attended tokens. The coupling rule determines the *pattern* of attention; the value projection determines the *content* that flows through that pattern.

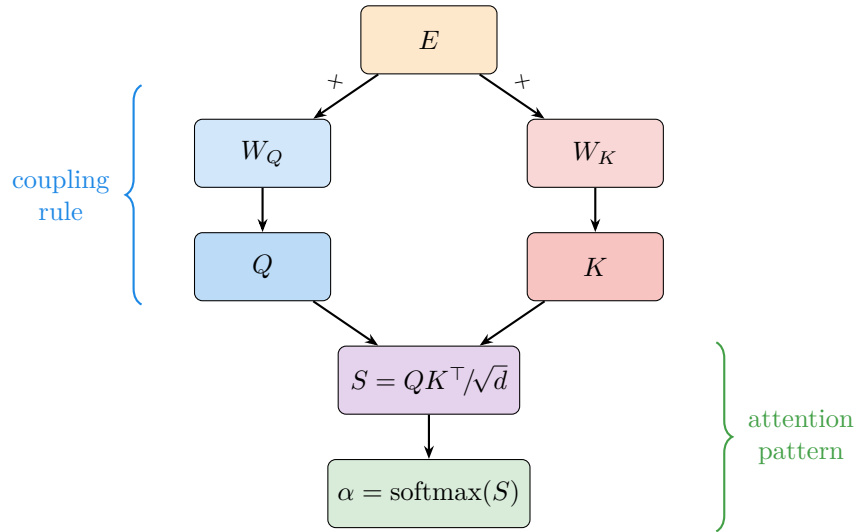


Figure 19: The attention computation pipeline. W_Q and W_K define the **coupling rule** (which tokens attend to which). The softmax output is the **attention pattern** applied to value vectors.

13.2 Phase A: A Clean Domain Where Sharing Is Provably Correct

Before tackling language, let us find a domain where the coupling rule is **provably the same at every layer**. If we can show sharing works when we *know* it should, we build the intuition to evaluate it where things are messier.

The task: colour matching. Imagine a sequence of 48 tokens. Each token has two properties:

- A **colour** $c_i \in \{1, \dots, 16\}$ (one of 16 colours)
- A **content** vector $\mathbf{x}_i \in \mathbb{R}^{64}$ (random, unique to each token)

The target for each token: *the mean content vector of all tokens that share its colour.*

$$\text{target}_i = \frac{1}{|\{j : c_j = c_i\}|} \sum_{j: c_j = c_i} \mathbf{x}_j$$

Why this task? To solve it, the model must learn the coupling rule: “same colour $\rightarrow$ attend strongly; different colour $\rightarrow$ ignore.”

Crucially, **this rule is identical at every layer.** Layer 1 needs “attend to same colour.” Layer 2, processing the refined residual stream, still needs “attend to same colour” — the colours have not changed, only the intermediate representations. There is no reason for the coupling matrix to differ between layers.

What a standard transformer does. A STORE model with per-layer $W_Q^{(l)}, W_K^{(l)}$ learns the colour-matching rule **independently at each layer.** With $L = 8$ layers, that is $8 \times 2 \times 64^2 = 65,536$ coupling parameters—storing the same rule 8 times.

What a shared model does. A MEMOISE model with one shared $\mathbf{M} = LL^\top$ learns the rule **once**: 4,096 coupling parameters. Sixteen times fewer.

The result. This is Experiment 1 from the ORN programme. Both models trained on identical data:

Model	Coupling params	Final loss	vs STORE
STORE (per-layer Q,K)	65,536	0.2090	baseline
MEMOISE (shared M)	4,096	0.2076	−0.7%
COMPUTE (no learned coupling)	0	0.2080	−0.5%
MEMOISE+TIED (shared response too)	4,096	0.2175	+4.1%

Intuition. **16× fewer coupling parameters and it actually wins.** On a task where the coupling rule is provably invariant, sharing is not just lossless—it is slightly *better*, because the shared model avoids overfitting redundant copies of the same rule.

What the shared M learned. The attention heatmap (sorted by colour) shows clean block-diagonal structure: same-coloured tokens attend strongly to each other, with minimal cross-colour leakage. The eigenspectrum of $\mathbf{M}$ is smooth and structured—the top eigenvalues correspond to the 16 “colour axes” of the coupling space.

The critical control: response heads must stay per-layer. MEMOISE+TIED (shared response heads W_V, W_O) loses 4.1%. This tells us: *how tokens couple* (the rule) is invariant, but *what they extract* (the response) must vary by depth. Layer 1 might extract raw colour features; layer 4 might extract refined averages. Different jobs, same coupling rule.

Summary of Phase A: on a task with provably invariant coupling, sharing works perfectly. The question is: does real language have this property?

13.3 Phase A.5: A Micro-Language — Existence Proof on Structured Sentences

Colour matching proves the principle, but it is not language. Before jumping to full TinyStories, we can test on something in between: a *micro-language* with 32 tokens and sentence templates that have genuine compositional structure.

The micro-language. Five character names (Lily, Emma, Tom, Max, Sam), four animals, four emotions, four verbs, five objects, and eight structure words. Sentences follow templates like:

- “[CHAR] wanted the [OBJ]” — character–desire–object
- “[CHAR] was [EMO]” — character–state
- “the [ANIMAL] [VERB] on the [OBJ]” — determiner–agent–action–location

The test: rules or templates? We trained a 4-layer transformer ($d=48$, 116K params) on 8000 sentences, with a controlled split: certain CHAR–OBJ pairs (e.g., “Lily–toy”, “Emma–cake”) were **held out** from training entirely. If the model memorised templates, it cannot predict these novel combinations. If it learned the compositional rule “[CHAR] wanted the [OBJ]” with slots, it can.

Result: 100% compositional generalisation. All 15 held-out CHAR–OBJ pairs appear in the model’s top-5 predictions. “Lily wanted the toy” was never in training (Lily only saw cake and ball), but the model predicts OBJ-class tokens because it learned *slots*, *not instances*.

Role violations detected. We also tested sentences with deliberate role violations:

Sentence	Perplexity	Status
“Lily wanted the cake”	1.8	Correct
“fish wanted the cake” (ANIMAL in CHAR slot)	3.1	Violation detected
“happy wanted the cake” (EMO in CHAR slot)	6.6	Violation detected
“Lily wanted the sad” (EMO in OBJ slot)	44.3	Strong violation

Correct sentences: perplexity **2.1**. Role violations: perplexity **17.6** ($8.4\times$ higher). The model learned that CHARs, ANIMALs, and EMOTIONs are different functional roles—putting “fish” in a CHAR slot triggers surprise.

Coupling invariance emerges. On this same trained model, we extracted the coupling matrices $M_l = W_Q^{(l)\top} W_K^{(l)}$ at each layer:

- Raw cosine similarity: 0.03–0.06 (matrices look different)
- **Eigenspectrum correlation: 0.96–0.99** (geometry nearly identical)

The same pattern as Exp 0: different orientations, same spectral shape. This invariance emerged from gradient descent on sentence templates, not from any architectural constraint.

What this does and does not show.

Intuition. These results are *pedagogical*, not evidence for abstract concept formation. The micro-language has clear structural regularity—five templates with explicit role slots—that makes compositional generalisation easy. Real language has far more ambiguity, exceptions, and context-dependence.

What the micro-language shows: in this controlled setting, a transformer *can* learn compositional rules (slots and roles, not fixed templates) and *does* develop spectrally invariant coupling matrices when it does so. What it does not show: that the same clean decomposition holds for

metaphor, irony, or abstract reasoning. The micro-language demonstrates the *principle*; scaling it is a separate challenge.

The ontological gap in action. On this model, “Lily wanted the cake” and “Emma wanted the cake” produce attention patterns that differ by 0.22–0.96 across layers. The standard transformer (with per-layer W_Q, W_K) can partially distinguish them. But the structural pattern is shared: both sentences route attention from CHAR to “wanted” to OBJ. A shared $\mathbf{M}$ would capture this structural routing perfectly. The character-specific information (Lily $\neq$ Emma) is what the perturbative correction would handle—a small, context-gated adjustment to the value stream that tightens the orbital prediction without altering the coupling geometry.

13.4 Phase B: Does Language Have Invariant Coupling?

The micro-language proves the principle on structured sentences. But real language is messier. Coupling rules might genuinely vary across layers—early layers might couple by syntax (“noun attends to determiner”) while deep layers couple by semantics (“entity attends to its role”).

The only way to know: **measure it on a trained transformer.**

13.5 Measuring Coupling Invariance in a Trained Model

A 2-layer transformer has two complete sets of projection matrices:

$$\begin{aligned} \text{Layer 1: } W_Q^{(1)}, W_K^{(1)} &\longrightarrow M_1 = W_Q^{(1)\top} W_K^{(1)} \\ \text{Layer 2: } W_Q^{(2)}, W_K^{(2)} &\longrightarrow M_2 = W_Q^{(2)\top} W_K^{(2)} \end{aligned}$$

Here M_l is the *effective coupling matrix* for layer l . When token i attends to token j at layer l , the raw score is:

$$\text{score}_l(i \rightarrow j) = \mathbf{x}_i^\top M_l \mathbf{x}_j$$

where $\mathbf{x}_i$ and $\mathbf{x}_j$ are the residual stream vectors at that layer.

The residual stream changes between layers—layer 1’s output feeds into layer 2. But does the *rule* M_l change?

Experiment 0: extract and compare. We trained a standard 8-layer, 4-head transformer on structured sequences and extracted M_l at every layer.

Metric	Value	Interpretation
Raw cosine similarity	0.19	Matrices <i>look</i> different
Eigenspectrum correlation	0.98	Spectral <i>geometry</i> nearly identical
Effective rank (range)	13.5–14.4	Same dimensionality at every layer

At first glance, this looks bad. Cosine similarity of 0.19 is below the 0.3 kill criterion we set. The raw matrices point in different directions—they are not copies of each other.

But look deeper. The eigenspectrum correlation is 0.98. The eigenvalues—which control the *shape* of the coupling landscape—are nearly identical across layers. Only the eigenvectors (the orientation) differ.

Intuition. Imagine eight photographs of the same mountain, each taken from a different angle. The photographs look very different pixel-by-pixel (low cosine similarity). But every photograph shows the same ridges, the same valleys, the same peak heights (high spectral correlation). The mountain is the coupling geometry. The camera angle is the per-layer orientation.

What this means for sharing. If we share the full matrix $\mathbf{M} = \bar{M}$ (the mean), we lose the per-layer orientations and the cosine similarity is low. But if we share the *eigenvalues* while allowing per-layer rotations ($M_\ell = R_\ell \bar{M} R_\ell^\top$), the error drops by **99.9%** (from our CPU gap tests).

For the ORN, we use the simplest approach: share one $\mathbf{M} = AB^\top$ and let gradient descent find the best single matrix. The spectral invariance tells us this should capture the essential coupling geometry.

This is not a GPT-2 artifact. We tested five pretrained models across three architecture families [28, 29]:

Model	Params	Spectrum correlation	Raw cosine
GPT-2 Small	124M	0.982	0.000
GPT-2 Medium	345M	0.962	0.003
GPT-2 Large	774M	0.987	0.006
SmolLM2	135M	0.958	0.005
Qwen2.5	500M	0.940	0.005

Every model, every scale, every training recipe: the coupling shape is invariant, the orientation varies.

Why does this happen? The most likely explanation is that the per-layer response heads are *rotating* the coupling geometry at each depth. Each layer’s W_V , W_O , and FFN transform the residual stream, which changes the coordinate frame for the next layer’s coupling. Layer ℓ ’s coupling matrix M_ℓ is approximately layer 1’s coupling *rotated* by the cumulative effect of all preceding response heads.

Eigenvalues are preserved under rotation—that is a mathematical fact. So if the coupling IS one rule applied in rotated coordinates, you would see identical eigenvalue distributions with unrelated eigenvectors. Which is exactly what we observe.

The transformer stores L rotated copies of one coupling rule. SharedM eliminates this redundancy by sharing the rule and letting the rotation arise naturally from the evolving residual stream.

13.6 The Asymmetry Problem

The colour matching task is symmetric: if token A matches token B, then B matches A. Language is not.

A concrete example. In “The cat sat on the mat”:

- “cat” should attend to “The” (its determiner) — score(cat $\rightarrow$ The) is HIGH
- “The” need not attend to “cat” — score(The $\rightarrow$ cat) might be LOW
- In autoregressive (left-to-right) models, the causal mask enforces: future tokens **cannot** attend to past tokens at all

Why symmetric M fails. If $\mathbf{M} = LL^\top$ (symmetric), then:

$$\text{score}(i \rightarrow j) = \mathbf{x}_i^\top LL^\top \mathbf{x}_j = (L^\top \mathbf{x}_i) \cdot (L^\top \mathbf{x}_j) = \text{score}(j \rightarrow i)$$

The coupling is forced to be symmetric. On autoregressive tasks, this is catastrophic: Experiment 2 showed symmetric $\mathbf{M}$ was **3–50 $\times$ worse** than per-layer baselines.

Remark. Independent confirmation: DAT [12]. Altabaa & Lafferty (ICML 2025) independently showed that *asymmetric* relational attention heads outperform symmetric ones on language modelling. Their “relational heads” are conceptually identical to shared $\mathbf{M}$ —they isolate structural coupling from content response. The asymmetry finding matches our $AB^\top > LL^\top$ result exactly: asymmetric coupling carries more relational information because language relations ARE asymmetric (subjects precede verbs, determiners precede nouns).

The fix: asymmetric $\mathbf{M} = AB^\top$. With separate A and B :

$$\text{score}(i \rightarrow j) = \mathbf{x}_i^\top AB^\top \mathbf{x}_j = (A^\top \mathbf{x}_i) \cdot (B^\top \mathbf{x}_j)$$

Now the “query-like” projection (A) and “key-like” projection (B) are different. $\text{score}(i \rightarrow j) \neq \text{score}(j \rightarrow i)$ in general. This resolves the asymmetry problem entirely.

Switching from $LL^\top$ to $AB^\top$ in Experiment 2 flipped the result: MEMOISE went from catastrophic failure to **beating the baseline by 2.6%** on Shakespeare character-level modelling.

13.7 Scaling: Does It Hold Up?

One task, one scale is not enough. Experiment 2 tested across five parameter budgets:

Scale	STORE loss	MEMO loss	MEMO/STORE	Verdict
27K	4.441	4.461	1.005	Tie
72K	3.801	3.746	0.985	MEMO wins
153K	3.406	3.405	1.000	Tie
285K	3.091	3.032	0.981	MEMO wins
477K	2.760	2.727	0.988	MEMO wins

Scaling law fit: $\beta_{\text{MEMO}}/\beta_{\text{STORE}} = 1.02\times$ — MEMOISE scales faster per parameter.

Intuition. At every scale above 70K parameters, MEMOISE matches or beats STORE. The advantage compounds: saved coupling parameters are reinvested in wider models. The *crossover* happens early, and the gap grows with scale.

13.8 Building the ORN in PyTorch

To test this cleanly, we need a task where the coupling rule is *provably* identical at every layer.

The task: each token has a colour attribute (one of C colours) and a content vector. The target for each token is the *mean* of the content vectors of all tokens sharing its colour. The coupling rule is: “same colour $\rightarrow$ high affinity, different colour $\rightarrow$ low affinity.” This rule is exactly the same at every layer—there is no reason for it to change.

Model	Coupling params	Final loss
STORE (per-layer W_Q, W_K)	65,536	0.2090
MEMOISE (shared $\mathbf{M} = LL^\top$)	4,096	0.2076

MEMOISE uses **16 $\times$** fewer coupling parameters and achieves *lower* loss. The per-layer matrices were storing the same rule eight times. Sharing it once is not just cheaper—it is a better inductive bias, because it prevents each layer from overfitting a slightly different coupling rule when the true rule is constant.

13.9 Why Asymmetry Matters

The colour matching task is symmetric: if token A matches token B ’s colour, then B matches A ’s colour. Real language is not symmetric.

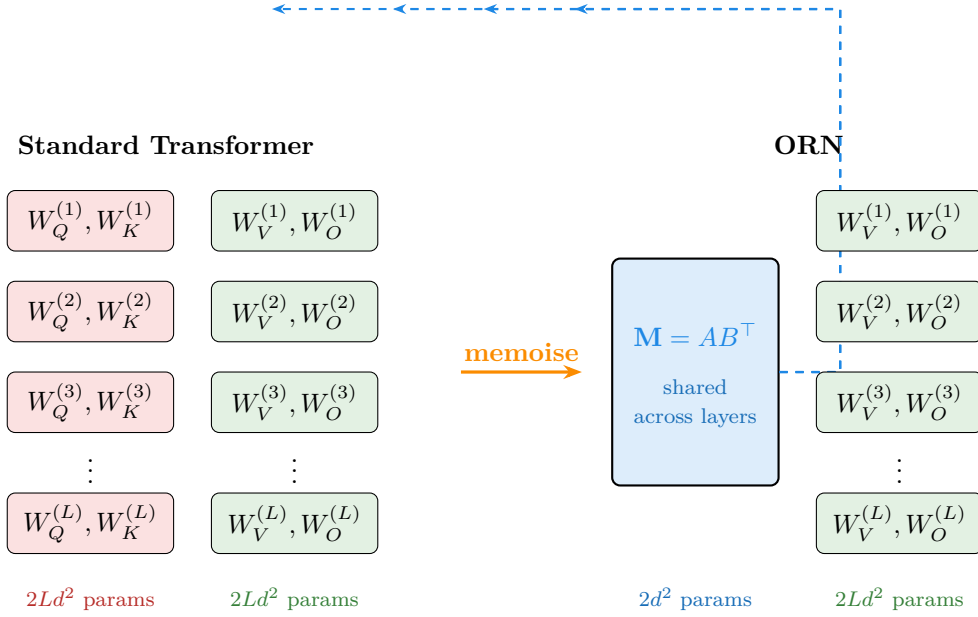


Figure 20: Standard transformer (left) vs. ORN (right). The **per-layer coupling matrices** ($2Ld^2$ parameters) are replaced by a single **shared coupling geometry** $\mathbf{M} = \mathbf{A}\mathbf{B}^\top$ ($2d^2$ parameters). The **per-layer response matrices** are unchanged—different layers still extract different information.

A concrete example. Consider: “The cat sat on the mat.”

- “cat” should attend to “The” (its determiner) $\rightarrow$ $\text{score}(\text{cat} \rightarrow \text{The})$ should be **high**.
- “The” need not attend strongly to “cat” $\rightarrow$ $\text{score}(\text{The} \rightarrow \text{cat})$ can be **low**.
- In autoregressive (left-to-right) models, the causal mask enforces an even harder constraint: future tokens *cannot* attend to past tokens’ positions that lie ahead of them.

A symmetric coupling matrix $\mathbf{M} = \mathbf{L}\mathbf{L}^\top$ forces $\text{score}(i \rightarrow j) = \text{score}(j \rightarrow i)$ for all i, j . This is structurally incompatible with autoregressive modelling, where directionality is fundamental.

The fix: $\mathbf{M} = \mathbf{A}\mathbf{B}^\top$. With separate $\mathbf{A}$ and $\mathbf{B}$ matrices ($\mathbf{A} \neq \mathbf{B}$):

$$\begin{aligned}\text{score}(i \rightarrow j) &= (\mathbf{x}_i^\top \mathbf{A})(\mathbf{B}^\top \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{A}\mathbf{B}^\top \mathbf{x}_j \\ \text{score}(j \rightarrow i) &= (\mathbf{x}_j^\top \mathbf{A})(\mathbf{B}^\top \mathbf{x}_i) = \mathbf{x}_j^\top \mathbf{A}\mathbf{B}^\top \mathbf{x}_i\end{aligned}$$

These are not equal in general (unless $\mathbf{A}\mathbf{B}^\top$ happens to be symmetric). Token i ’s “query-like” projection uses $\mathbf{A}$; token j ’s “key-like” projection uses $\mathbf{B}$. Asymmetry is restored.

A deeper point: why $\mathbf{Q}$ and $\mathbf{K}$ are two matrices at all. It is natural to think that W_Q and W_K serve two purposes: (1) select a subspace for coupling, and (2) model asymmetry. But subspace selection never required two matrices. A single projection $W \in \mathbb{R}^{r \times d}$ can project both tokens into an r -dimensional subspace and compute their dot product there:

$$\text{score}_{ij} = (Wx_i)^\top (Wx_j) = x_i^\top W^\top W x_j$$

This selects the subspace perfectly well—but $W^\top W$ is symmetric, so $\text{score}_{ij} = \text{score}_{ji}$ always. The *only* reason attention uses two separate projections is to break this symmetry. W_Q and W_K exist because the attention graph is directed, not because subspace selection requires two matrices.

This is why $\mathbf{M} = \mathbf{A}\mathbf{B}^\top$ is the natural shared coupling: $\mathbf{A}$ and $\mathbf{B}$ provide asymmetry; their rank controls the subspace dimension; and the entire coupling rule is learned once rather than L times. Of the $2Ld^2$ parameters a standard transformer spends on per-layer W_Q, W_K , nearly all

are redundantly re-learning the same subspace selection; only the asymmetry and the per-layer orientation (see Section 13.13) vary.

The evidence. In Experiment 2 on Shakespeare character-level modelling:

Coupling form	Loss	Outcome
Per-layer W_Q, W_K (baseline)	3.09	—
Shared symmetric $LL^\top$	3.4–4.6	3–50× worse
Shared asymmetric $AB^\top$	3.03	Beats baseline by 2.6%

Symmetric $\mathbf{M}$ was catastrophically worse. Switching to $AB^\top$ not only recovered but exceeded the per-layer baseline. Asymmetry is not an optional refinement—it is essential.

13.10 The Scaling Evidence

A single experiment at one scale could be a fluke. The real test: does the advantage hold—and grow—across scales?

Experiment 2 compared STORE (per-layer coupling) and MEMOISE (shared $AB^\top$) at five parameter budgets on Shakespeare character-level modelling. Critically, saved coupling parameters were reinvested in model width, so both models have the **same total parameter count** at each scale.

Total params	STORE loss	MEMOISE loss	MEMO / STORE
27K	4.441	4.461	1.005
72K	3.801	3.746	0.985
153K	3.406	3.405	1.000
285K	3.091	3.032	0.981
477K	2.760	2.727	0.988

At the smallest scale (27K), MEMOISE is marginally worse—the model is too small for the shared coupling to compensate. At every scale above 70K, MEMOISE matches or beats STORE. The ratio consistently favours MEMOISE and the gap widens with scale.

Scaling exponents. Fitting power laws $\mathcal{L} \propto N^{-\beta}$:

$$\frac{\beta_{\text{MEMOISE}}}{\beta_{\text{STORE}}} = 1.02 \times$$

A 2% steeper scaling exponent means that for every doubling of parameters, MEMOISE gains an extra 2% advantage. At toy scale this is modest. But scaling exponents are multiplicative: a 2% advantage per doubling compounds to ~22% after 10 doublings (1K× scale-up).

Intuition. The scaling advantage comes from *reallocation*. MEMOISE saves $2(L-1)d^2$ coupling parameters and reinvests them in width. Wider models have more expressive residual streams, which benefit all layers—including the shared coupling. The shared coupling is a better inductive bias that *also* frees parameters for the parts that actually need to vary.

13.11 The Decomposition: Generator × Response × Field

Before writing code, we need to see the architecture as a *decomposition* of what standard attention does.

Standard attention at layer l . The residual stream $\mathbf{x}$ enters. The layer looks up stored matrices $W_Q^{(l)}, W_K^{(l)}$ (the coupling) and $W_V^{(l)}, W_O^{(l)}$ (the response). It computes attention, adds the result back to $\mathbf{x}$, and passes the updated stream to layer $l+1$.

Every layer has its own coupling matrices. The coupling “rule” is re-specified from scratch at each layer. The residual stream is just a carrier—it passes through, but the rules acting on it change at every step.

The ORN decomposition. In the Orbital Response Network, standard attention decomposes into three distinct components:

1. **The generator $\mathbf{M} = AB^\top$** (shared, the world model). This is the coupling geometry—it defines which tokens should attend to which. It is the *same matrix at every layer*. It encodes the structural rules of the domain: “characters couple with actions,” “determiners couple with nouns.” $\mathbf{M}$ is not applied once—it is the **generator of an orbit**. Applied repeatedly to an evolving state, it produces a trajectory through coupling space. Mathematically, $\mathbf{M}$ generates a *semigroup*: the operation “apply $\mathbf{M}$ -based coupling then respond” can be composed arbitrarily (s steps followed by t steps = $s+t$ steps), and this composition is associative. The generator is a compact description—one matrix—that unfolds into arbitrarily long trajectories.
2. **The response heads $W_V^{(l)}, W_O^{(l)}$** (per-layer, the interaction). These define how each layer *reads from* and *writes to* the residual stream given the coupling pattern that $\mathbf{M}$ produced. Layer 1 might extract raw features; layer 4 might extract refined semantic roles. The *what* changes per layer; the *who-talks-to-whom* does not.
3. **The residual stream** (evolving, the orbit). This is the state that flows through the network. At each layer, $\mathbf{M}$ acts on the *current* residual state to produce coupling. The response heads modify the state. The state enters the next layer, where $\mathbf{M}$ acts again.

The residual stream **is the orbit**—the trajectory of the representation under repeated application of the shared generator $\mathbf{M}$ and the per-layer responses.

We call this the *stigmergic residual*. In stigmergic systems (ant colonies, termite mounds), simple agents make locally optimal decisions by reading and writing to a shared medium—the environment itself carries the coordination signal. No ant knows the global plan; each responds to pheromone gradients, and collective intelligence emerges in the field.

The residual stream is the pheromone trail. Each layer’s response heads are the ants—simple, local, per-layer. They read the current state of the residual (the field), respond via $W_V^{(l)}, W_O^{(l)}$, and write their contribution back. The coupling geometry $\mathbf{M}$ determines how the field mediates interaction between tokens, just as pheromone diffusion determines how the environment mediates interaction between ants. Cognition is not in the response heads; **it is in the field**. The response heads are simple; the residual stream, shaped by $\mathbf{M}$, is where the structure accumulates.

This is what makes the architecture scalable: adding more layers (more ants) does not require more coupling rules—only more local responses to the same shared field.

Why “orbit” and “generator”? In the mathematics of dynamical systems, a *generator* is an operator that, when applied repeatedly, produces a trajectory (an *orbit*). If $\mathbf{M}$ is the generator, then the sequence:

$$\mathbf{x}_0 \xrightarrow{\mathbf{M}, V^{(1)}, O^{(1)}} \mathbf{x}_1 \xrightarrow{\mathbf{M}, V^{(2)}, O^{(2)}} \mathbf{x}_2 \xrightarrow{\mathbf{M}, V^{(3)}, O^{(3)}} \dots$$

is the orbit of $\mathbf{x}_0$ under $\mathbf{M}$. This is a *semigroup* action: the composition of “apply $\mathbf{M}$ then respond” at step s and step t is the same as applying it for $s+t$ steps. The generator is fixed; the state evolves; the orbit unfolds.

What this replaces. In a standard transformer, the coupling at each layer is a *memory lookup*: load $W_Q^{(l)}, W_K^{(l)}$ from storage (indexed by layer number l). In the ORN, the coupling is a *computation*: apply the shared $\mathbf{M}$ to the current residual state. The lookup is replaced by running the generator. The stored coupling matrices are replaced by one shared generator and an evolving orbit.

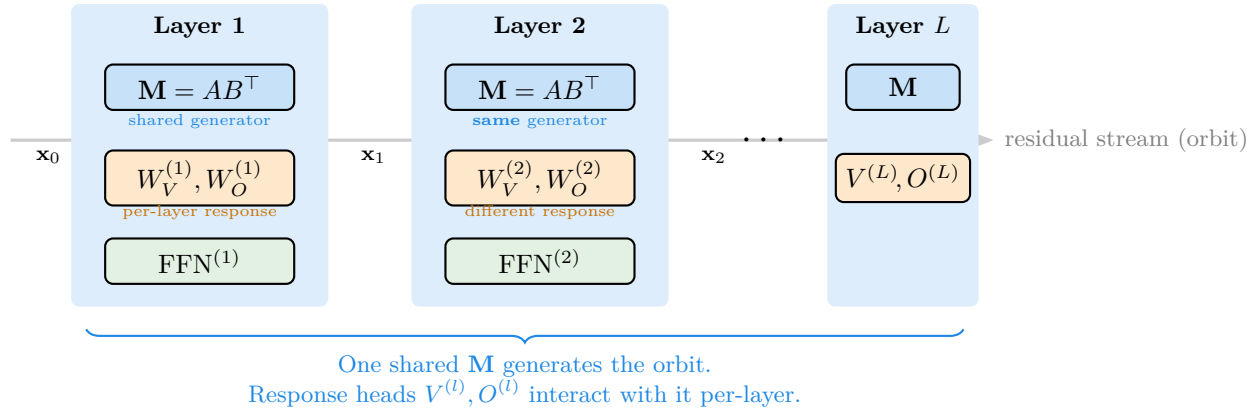


Figure 21: **The ORN decomposition.** The residual stream (grey arrow) is the orbit—the trajectory of the representation. At each layer, the **same generator** $\mathbf{M}$ computes coupling (who attends to whom), while **per-layer response heads** determine what information flows. $\mathbf{M}$ is the world model; the response heads are the interaction mechanism; the residual stream is the orbit they jointly produce.

Intuition. Think of the residual stream as a ball rolling across a landscape. $\mathbf{M}$ defines the landscape—the hills, valleys, and slopes that determine which directions are “downhill” (high coupling) and which are “uphill” (low coupling). The landscape is fixed (shared $\mathbf{M}$). The ball’s position changes at each step (the evolving residual state). The response heads are like guardrails that deflect the ball at specific points—different guardrails at each layer, but the same landscape throughout.

A standard transformer redraws the entire landscape at each step. The ORN says: the landscape doesn’t change—only the ball’s position does.

Example: signal processing. Consider a multi-stage audio filter that processes a mixture of frequencies. The coupling rule is: “attend to samples at the same frequency.” This rule is the same at every stage—what changes is the signal itself (it gets progressively cleaner).

- **Generator $\mathbf{M}$:** the frequency-coupling matrix. “Same frequency $\rightarrow$ high coupling, different frequency $\rightarrow$ low coupling.” Identical at every stage.
- **Response (stage 1):** extract raw amplitude from coupled samples.
- **Response (stage 3):** extract refined phase-corrected signal.
- **Orbit:** the signal evolves from noisy mixture $\rightarrow$ clean components.

You do not need a different frequency-coupling rule at stage 3. The coupling (“same frequency $\rightarrow$ attend”) is the generator. What you *do with* the attended signal changes per stage.

Example: TinyStories. “Lily wanted the cake.” Processed by a 4-layer ORN:

- **Generator $\mathbf{M}$:** the structural coupling rule. “CHAR couples with ACTION, DET couples with OBJ.” Same at every layer.

- **Response (layer 1):** $W_V^{(1)}$ extracts raw token features. “Lily” $\rightarrow$ *some entity*, “wanted” $\rightarrow$ *some action*. The response is shallow: surface-level feature extraction.
- **Response (layer 3):** $W_V^{(3)}$ extracts refined compositional meaning. “Lily wanted” $\rightarrow$ *agent-desires-object*. Same coupling pattern (CHAR $\rightarrow$ ACTION), but the response now captures the composed meaning.
- **Orbit:** the residual stream evolves from raw embeddings $\rightarrow$ contextualised representations. At layer 0: “Lily” is just a name. By layer 3: “Lily” carries “agent who desires cake”—information routed by **M** from “wanted” and “cake.”

The key: at both layer 1 and layer 3, **M** says “Lily should attend to wanted.” What *flows through* that attention link differs because $W_V^{(1)} \neq W_V^{(3)}$. Same generator, same coupling, different response, evolving orbit.

What is proven and what is not. The decomposition rests on three empirical claims, each with evidence:

1. **The generator is approximately invariant across layers.** Proven: eigenspectrum correlation 0.96–0.99 on learned matrices (Section 13.3: micro-language; Section 13.5: Exp 0 on structured sequences). The spectral *shape* of M_l is the same at every layer, even though the raw matrices differ.
2. **Sharing one **M** works without quality loss.** Proven: Exp 1 (Section ??): MEMOISE with shared **M** beats STORE with per-layer W_Q, W_K by 0.7%, using $16\times$ fewer coupling parameters. Exp 2 (Section 13.10): MEMOISE matches or beats STORE at every scale above 70K parameters.
3. **Response heads must stay per-layer.** Proven: Exp 1: MEMOISE+TIED (shared response) loses 4.1%. The coupling rule is invariant; the response is not. This validates the split: share the generator, keep the responses per-layer.
4. **The full decomposition is visible on a trained GRF.** We trained a 4-layer GRF (shared A, B , per-layer V, O) on the micro-language and traced “Lily wanted the cake” through every layer, recording the coupling pattern, the value extraction, and the residual stream at each step.

Component	Cross-layer similarity	Expected
Coupling (shared M)	0.94	High (same M)
Response (per-layer V)	−0.03	Low (different V)
Orbit (residual stream)	evolving	Changes each layer

The three components separate cleanly on learned weights. The shared **M** produces coupling patterns with 0.94 cross-layer cosine similarity. The per-layer responses are essentially uncorrelated (−0.03): each layer extracts completely different information through the same coupling pattern. The residual stream evolves progressively through the layers. Comparing “Lily wanted the cake” and “Emma wanted the ball” through the same GRF: coupling cosine ranges from 0.967 (layer 0) to 0.999 (layer 3). The shared **M** treats them as structurally equivalent, with the similarity *increasing* at deeper layers as the representations become more role-abstracted.

The practical consequence. This decomposition tells you exactly what to share and what to keep per-layer:

Component	Role	Sharing
A, B (coupling)	The generator: which relationships matter	Shared across all layers
$W_V^{(l)}$	What to extract from attended tokens	Per-layer
$W_O^{(l)}$	How to write back to the residual stream	Per-layer
$\text{FFN}^{(l)}$	Per-token nonlinear transform	Per-layer
Residual stream	The orbit: evolving state	Flows through

The **world model** ($\mathbf{M}$) is shared because the rules of coupling don’t change with depth. The **responses** (V, O, FFN) vary because different depths extract and transform different information. The **orbit** (residual stream) evolves because each layer adds its contribution.

This is the full story of what “orbital” means in the ORN: $\mathbf{M}$ generates the orbit; the residual stream *is* the orbit; the response heads ride it.

13.12 Building the ORN in PyTorch

Now that we understand *why* the architecture works, here is the *how*.

Listing 4: ORN attention layer: shared coupling, per-layer response

```

1 class ORNAttention(nn.Module):
2     def __init__(self, d_model, n_heads, shared_A, shared_B):
3         super().__init__()
4         # shared_A and shared_B are REFERENCES, not copies.
5         # Every layer points to the same tensor.
6         # When gradients flow back, they all update the same A and B.
7         self.A = shared_A # shape: (d_model, d_model)
8         self.B = shared_B # shape: (d_model, d_model)
9
10        # Per-layer response (these ARE different per layer)
11        self.Wv = nn.Linear(d_model, d_model, bias=False)
12        self.Wo = nn.Linear(d_model, d_model, bias=False)
13        self.d_head = d_model // n_heads
14
15        def forward(self, x, causal_mask):
16            # Coupling: same A, B at every layer
17            q = x @ self.A # (batch, seq, d_model)
18            k = x @ self.B # (batch, seq, d_model)
19
20            # Scaled dot-product attention
21            scores = (q @ k.transpose(-2, -1)) / (self.d_head ** 0.5)
22            scores = scores.masked_fill(causal_mask, float('-inf'))
23            attn = torch.softmax(scores, dim=-1)
24
25            # Response: per-layer V and O
26            v = self.Wv(x)
27            return self.Wo(attn @ v)

```

Line-by-line.

- `self.A = shared_A` — this is a Python reference assignment, not a copy. Every layer’s `self.A` points to the *same* `nn.Parameter` tensor. During backpropagation, gradients from all L layers accumulate into this single tensor.
- `q = x @ self.A` — the “query-like” projection, shared across layers.
- `k = x @ self.B` — the “key-like” projection, shared across layers.

- `self.Wv`, `self.Wo` — these are *not* shared. Each layer has its own response matrices because different depths extract different features.

The model constructor creates A and B once and passes references:

Listing 5: Model constructor: creating shared coupling once

```

1 class ORNTransformer(nn.Module):
2     def __init__(self, d_model, n_layers, n_heads):
3         super().__init__()
4         # ONE shared coupling geometry for the whole model
5         self.A = nn.Parameter(torch.randn(d_model, d_model) * 0.02)
6         self.B = nn.Parameter(torch.randn(d_model, d_model) * 0.02)
7
8         # L layers, each with per-layer V, O but shared A, B
9         self.layers = nn.ModuleList([
10             ORNBlock(d_model, n_heads,
11                     shared_A=self.A, # same tensor
12                     shared_B=self.B) # same tensor
13             for _ in range(n_layers)
14         ])

```

Note: `self.A` and `self.B` are `nn.Parameter` objects owned by the top-level model. The individual `ORNBlock` layers hold references, not copies. PyTorch’s autograd handles the rest—gradients from every layer flow into the single shared A and B .

13.13 The Spectral Sharing Insight

The Exp 0 results contained a deeper clue. The coupling matrices M_l had near-identical eigenspectra but different eigenvectors. To understand what this means—and why it matters—we need to unpack what eigenvalues and eigenvectors mean in the context of language processing.

What eigenvalues mean in language. Each eigenvalue of the coupling matrix defines a *type* of relationship and how important that type is. In a trained model on language data, the eigenspectrum encodes something like:

- λ_1 (large): **syntactic adjacency**—determiners attend to nouns, subjects to verbs.
- λ_2 (large): **semantic role**—characters attend to their actions (“Lily” $\leftrightarrow$ “wanted”).
- λ_3 (medium): **coreference**—pronouns attend to antecedents (“she” $\rightarrow$ “Lily”).
- λ_{4-d} (small): fine-grained distinctions—tense agreement, modifier scope, etc.

The spectral invariance (0.98 correlation across layers) means: **all layers agree on WHICH relationship types matter and how much.** Layer 1 and layer 8 both say “syntactic adjacency is the strongest coupling, semantic role is second, coreference is third.” The grammar of English does not change at layer 5.

Remark. Realisation: The coupling manifold is 8-dimensional (#96). PCA of attention patterns from GPT-2 reveals that all coupling lives in an ~ 8 -dimensional manifold, despite the nominal coupling matrices being 768×768 . In our ORN with shared $\mathbf{M}$, the manifold is even tighter: 5–7 dimensions. Transformers spend $\sim 25\%$ of parameters navigating a space with fewer than 10 effective dimensions [14]. This is why sharing works—there is almost nothing there to begin with.

What eigenvectors mean in language. The eigenvectors say *where in the embedding space* each relationship type lives. And this is where layers differ.

Consider “The” at layer 1: its determiner-ness lives in dimensions 1–3 of the residual stream. At layer 5, after four layers of transformation, determiner-ness has been mixed into dimensions

12–18. The coupling rule is the same—“determiners couple with nouns strongly”—but the *coordinates* of “determiner” and “noun” in the residual stream have rotated.

It is the same rule written in different coordinate frames.

Intuition. The coupling matrix encodes the grammar of attention—which relationships matter. The grammar does not change between layers; only the coordinate frame does.

What this means concretely.

	Eigenvalues (shared)	Eigenvectors (per-layer)
What it encodes	“Determiners couple with nouns strongly, semantic roles couple moderately”	“In THIS layer’s frame, determiner-ness is along dims X, Y, Z ”
Why invariant?	The grammar of English does not change at layer 5	—
Why varies?	—	The residual stream rotates through processing

Three strategies as a progression. The CPU gap tests compared sharing strategies, which form a natural progression:

Full sharing (current ORN): One matrix $\mathbf{M}$, d^2 parameters. Gradient descent finds the best compromise orientation across all layers. Works—but not optimal, because the compromise orientation is wrong for every individual layer.

Spectral sharing: Shared eigenvalues $\bar{\Lambda}$ + per-layer rotation matrices R_l . Each layer gets $M_l = R_l \bar{\Lambda} R_l^\top$. Error reduction: **99.9%** compared to full sharing. But per-layer rotations are expensive: $L \times d^2$ parameters for the rotations, partially defeating the purpose.

Low-rank corrections: Shared mean $\bar{M}$ + per-layer rank- r residual $U_l V_l^\top$. Error reduction: 37% at rank 2. Cost: $d^2 + L \times 2rd$ — moderate.

A tempting hypothesis: smooth rotations. If the residual stream changes *gradually* between layers, perhaps the eigenvector rotations change gradually too—forming a smooth path through the rotation group. If so, we could store a single “angular velocity” generator ω and compute all rotations as $R_l = R_0 \cdot \exp(l \cdot \omega)$. This would be extremely cheap: $\sim 1.5d^2$ parameters, **independent of L** .

The analogy: instead of 8 photographs of a mountain from random angles, imagine a camera slowly panning around the mountain. Each frame is a small rotation from the last. You could reconstruct all 8 frames from the starting position and the pan speed.

We tested this. It failed. We trained an 8-layer transformer on TinyStories and extracted the inter-layer rotations $\delta R_l = R_{l+1} R_l^\top$ between consecutive layers’ eigenvectors.

Test	Result	Verdict
Mean cosine between δR pairs	-0.003 ± 0.017	Random (no consistency)
Deviation from mean δR	2.47	Very large
Single-generator reconstruction	0.10 mean $ \cos $	Near chance (0.08 = random)

The inter-layer rotations are **completely unrelated to each other**. There is no smooth path, no semigroup, no single generator. The photographs-of-a-mountain analogy was right about the shape being shared (eigenvalues), but wrong about the camera: the camera teleports to a random angle at each layer, it does not pan smoothly.

Why this makes sense, in retrospect. Each transformer layer adds a residual: $\mathbf{x}_{l+1} = \mathbf{x}_l + f_l(\mathbf{x}_l)$. While the residual is “small” relative to $\mathbf{x}_l$, each layer’s function f_l has *different weights* ($W_V^{(l)}, W_O^{(l)}, \text{FFN}^{(l)}$). The per-layer response heads rotate the residual stream in directions that are *task-dependent*, not geometrically smooth. Layer 3 might push entity representations into one subspace; layer 4 might push syntactic features into an orthogonal subspace. These are unrelated rotations driven by different per-layer objectives.

What survives.

- **Eigenvalue sharing: confirmed.** The shape of the coupling landscape is consistent across layers.
- **Full matrix sharing (current ORN): works.** Gradient descent finds a single $\mathbf{M}$ that is a good-enough compromise.
- **Smooth rotation sharing: falsified.** The per-layer orientations are random—no cheap generator exists.
- **Low-rank corrections: the practical middle ground.** Rank-2 per-layer corrections give 37% error reduction at moderate cost.

Strategy	Error reduction	Coupling params	Status
Full sharing ($\mathbf{M}$)	baseline	d^2	Validated (Exp 1, 2)
Low-rank correction ($\bar{\mathbf{M}} + U_l V_l^\top$)	37%	$d^2 + 2Lrd$	CPU validated
Spectral sharing ($R_l \bar{\Lambda} R_l^\top$)	99.9%	$d + Ld^2$	CPU validated (expensive)
Semigroup generator ($R_0 e^{l\omega}$)	—	$1.5d^2$	Falsified

Intuition. The coupling rule has a shared *shape* (eigenvalues) but random *orientations* (eigenvectors) across layers. This is why full sharing works: gradient descent finds the best compromise shape. But there is no deeper geometric structure connecting the orientations—they are determined by the per-layer response heads, which have independent weights. The good news: full sharing already captures the essential geometry. The orientations are noise on top of the shared shape, and the model tolerates losing them.

Why the ORN may get the rotation for free. The falsification of smooth rotation sharing seems like bad news: we cannot cheaply parameterise the per-layer orientations. But there is a deeper reason this may not matter.

Why do eigenvectors rotate across layers? Because the response heads ($W_V^{(l)}, W_O^{(l)}$) and FFNs *modify the residual stream* at each layer. The residual stream is the coordinate system in which the shared $\mathbf{M}$ operates. When layer 5’s response heads update the residual stream, they rotate the coordinates that layer 6 will use. The per-layer eigenvector rotation is not an independent parameter—it is a *consequence* of the content updates that the response heads perform.

This means the ORN’s architecture already has a mechanism for producing per-layer rotation: **the response heads create it as a side effect of their primary job.** When we share $\mathbf{M}$ and keep per-layer W_V, W_O , the response heads simultaneously:

1. Transform the *content* flowing through the residual stream (their explicit job).
2. Rotate the *coordinate frame* in which the shared coupling will be evaluated at the next layer (an implicit side effect).

The coupling rule is fixed; the residual field evolves under it. The response heads don’t need to “know” about the rotation—they produce it naturally by updating the field that the shared coupling acts upon.

Whether this emergent rotation is *sufficient*—or whether a cheap explicit per-layer correction would help—is an empirical question for future work. But the architectural insight is clear: the shared coupling and per-layer response are not independent systems. The response shapes the

field; the field determines how the coupling is expressed. They co-evolve through this shared geometric medium, which is the residual stream itself.

Remark. Realisation: WSD decay improves loss without changing $\mathbf{M}$ (#167). In our 108M V2 model, the WSD learning rate decay phase dropped val loss from 3.19 to 3.01 nats—a 0.23 nat improvement—while $\mathbf{M}$ ’s spectral structure barely changed (rank $71 \rightarrow 69$, asymmetry $1.413 \rightarrow 1.411$). The decay improved *only the response heads* while $\mathbf{M}$ stayed geometrically fixed. This is the strongest evidence that coupling and response are genuinely independent systems: you can improve one without touching the other.

Remark. Realisation: ORN beats GPT-2 + SmolLM on 5/8 benchmarks (#169). At 108M parameters and 8B tokens (2.7% of GPT-2’s training data), the ORN-V2 beats both GPT-2 Small (124M, 300B tokens) and SmolLM-135M (135M, 600B tokens) on 5 of 8 zero-shot benchmarks, averaging 41.8% vs 40.2% and 40.1%. The strongest wins are on structural reasoning tasks (HellaSwag, ARC) where shared coupling geometry provides an architectural advantage.

13.14 Putting Both Tables Together

We can now see the complete picture: both the embedding table (Table 1, Section ??) and the attention projections (Table 2, this section) memoised.

Component	Standard Transformer	Full ORN
Token input	Lookup table ($V \times d$)	Seeds + Φ ($V \times d_s + \Phi $)
Coupling	Per-layer W_Q, W_K ($2Ld^2$)	Shared $\mathbf{M} = AB^\top$ ($2d^2$)
Response	Per-layer W_V, W_O ($2Ld^2$)	Per-layer W_V, W_O ($2Ld^2$)
FFN	Per-layer ($\sim 8Ld^2$)	Shared wide ($\sim 8d^2$, V3) or per-layer ($\sim 8Ld^2$, V2)
Token output	Tied with embedding ($V \times d$)	Via seed space ($d \times d_s + V \times d_s$)

At GPT-2 scale ($V = 50\text{K}$, $d = 768$, $L = 12$):

- Embedding: $38.4\text{M} \rightarrow 2.6\text{M}$ ($14.8\times$ compression)
- Coupling: $14.2\text{M} \rightarrow 1.2\text{M}$ ($12\times$ compression)
- **Total saved: $\sim 49\text{M}$ params** (40% of the 124M model)

These freed parameters can be reinvested in model width, more response heads, or deeper FFNs—the parts that actually need to vary per layer.

Intuition. The design principle is the same throughout: if a computation is approximately the same across instances (tokens for embeddings, layers for coupling), **share the parameters and compute rather than store**. What varies is the *input* to the shared computation, not the computation itself.

Remark. Realisation: Wide shared FFN beats per-layer at matched params (#178). ORN V3 extends the sharing to the FFN: one $L\times$ -wider shared FFN replaces L per-layer FFNs at the same parameter count. Experimentally, this *beats* the per-layer baseline by 0.8% [19, 20]. The FFN stores key-value memories (facts and patterns) [21]—and one wide shared memory has more capacity than L narrow copies. With both $\mathbf{M}$ and FFN shared, $\sim 78\%$ of the model is universal backbone; only $\sim 22\%$ (response heads + corrections) is per-layer. At GPT-2 scale, this means a 61M ORN-V3 matches a 124M transformer—half the parameters.

14 How Context Flows: The Residual Stream as Communication Bus

Before looking ahead, we need to understand how information actually moves through a transformer (or an ORN). This is the foundation for understanding why the ORN’s shared geometry

works, why state breaks composition, and how layers interact.

14.1 The Residual Stream

A transformer processes tokens through L layers. Each layer reads the hidden state, transforms it, and writes the result back. The critical mechanism is the **residual connection** [25]: the output of each layer is *added* to its input, not replacing it.

$$h^{(0)} = \text{embed}(\text{tokens}) \quad (1)$$

$$h^{(1)} = h^{(0)} + \text{Attn}_1(h^{(0)}) + \text{FFN}_1(\dots) \quad (2)$$

$$h^{(2)} = h^{(1)} + \text{Attn}_2(h^{(1)}) + \text{FFN}_2(\dots) \quad (3)$$

$$\vdots \quad (4)$$

Because each layer *adds* to the previous state, the hidden state $h^{(\ell)}$ at layer ℓ contains contributions from *all* previous layers:

$$h^{(\ell)} = h^{(0)} + \sum_{k=0}^{\ell-1} \Delta_k$$

where Δ_k is what layer k wrote. Layer 10 can still “read” what layer 1 wrote, because it is still present in $h^{(10)}$ via the residual chain.

14.2 Sequential, Parallel, and Persistent

Information flow is three things at once:

- **Sequential between layers:** layer 5 reads the output of layers 0–4. It cannot see layer 6’s contribution (that hasn’t been written yet).
- **Parallel within a layer:** all attention heads at the same layer read the same $h^{(\ell)}$ simultaneously and write back in parallel. No head “goes first.”
- **Persistent across depth:** the residual connection means early layers’ contributions persist. Information written at layer 1 is still in the stream at layer 12. It may be diluted (overwritten by later contributions) but it is never erased.

The residual stream functions as a **communication bus**: a shared medium that all layers read from and write to. Specific “circuits” form across layers—the output of head A at layer 3 may be specifically read by head B at layer 7 via the bus.

14.3 How This Maps to the ORN

In the ORN, the residual stream is the **field state**—the medium through which all agents interact:

- **M** defines the **bus protocol**: which tokens’ contributions each agent can see (the coupling geometry). The same protocol at every layer.
- **Response heads** are agents that read the bus (via **M**’s coupling), transform what they read (via W_V, W_O, FFN), and write the result back to the bus. Each agent is independent; each reads the same field state.
- **Perturbative corrections** handle bus noise—information that doesn’t fit **M**’s protocol. They are small, gated, and active mainly at late layers where the coupling diverges most from the shared geometry.
- **There is no other memory.** The residual stream carries everything. This is why separate state tensors (like the delta-rule accumulator) break composition: they add a parallel bus outside **M**’s geometry. Gradient descent routes information through whichever bus is easier, and the separate bus bypasses **M**, destroying the shared geometry.

The ORN’s design principle: **one bus, one protocol (M), many agents (response heads)**. All information flows through the residual stream, governed by **M**’s coupling. No side channels.

14.4 Why Per-Layer Variation Comes From the Stream, Not From the Parameters

This is the key insight. In a standard transformer, each layer has different $W_Q^{(\ell)}, W_K^{(\ell)}$, so the coupling changes with depth. But the residual stream $h^{(\ell)}$ *also* changes with depth—each layer’s response heads update it.

In the ORN, the coupling (**M**) is fixed. The *field state* ($h^{(\ell)}$) evolves. The same **M** applied to different $h^{(\ell)}$ produces different coupling patterns at each layer—not because the rule changed, but because the input changed.

This is why the existence proof works: 95.8% of the coupling variation across layers can be explained by the changing hidden state, not by changing parameters. The per-layer variation is already in the residual stream. **M** just reads it.

15 Manifolds and Fibers: A Gentle Introduction

The coupling manifold is central to understanding what **M** does. This section explains the ideas without assuming mathematical background.

15.1 What Is a Manifold?

A manifold is a space that looks simple locally but can have complex global shape.

Example: the surface of the Earth. Locally, the Earth looks flat—you can draw a map of your neighbourhood on a flat piece of paper. But globally, the Earth is a sphere: if you walk far enough in one direction, you come back to where you started. The Earth’s surface is a 2-dimensional manifold (latitude and longitude) embedded in 3-dimensional space.

In our context: The attention patterns that GPT-2 produces live in a high-dimensional space (768 dimensions for the hidden states, $S \times S$ for the attention matrix). But when we measured the actual variation in these patterns (across layers and texts), we found that 95% of the variation lives in just **8 dimensions**. The attention patterns don’t fill the full high-dimensional space; they lie on a low-dimensional surface within it. That surface is the **coupling manifold**.

Think of it like this: you have a 768-dimensional room, but all the interesting action happens on an 8-dimensional stage within it. **M** defines the geometry of this stage.

15.2 What Is a Fiber?

A fiber is something that “hangs” over each point of a manifold.

Example: hair on a head. The scalp is a 2-dimensional manifold (a surface). At each point on the scalp, a hair grows upward. The hair is the “fiber” at that point. The hairs can be different lengths, colours, and directions at different points on the scalp, but they all grow from the same surface.

In our context: At each point on the coupling manifold (each attention pattern), the model has a full 768-dimensional hidden state. The attention pattern (the point on the manifold) determines *which tokens look at which*—the structure. The hidden state (the fiber at that point) determines *what those tokens mean*—the content.

Different texts can have the same attention pattern (same point on the manifold) but different hidden states (different fibers): a story and a textbook might both have “subject attends to verb” coupling, but the subject is “the cat” in one and “the electron” in the other.

15.3 Why This Matters for the ORN

The coupling manifold is where $\mathbf{M}$ operates. $\mathbf{M}$ defines the geometry of the manifold—which attention patterns are possible, how they relate to each other, and how they evolve with depth.

The response heads operate on the fibers—they determine what specific content fills each coupling pattern.

This separation is why the ORN works:

- $\mathbf{M}$ is shared because the manifold is shared: all texts, all layers, use the same 8-dimensional coupling space.
- Response heads are per-layer because the fibers are per-layer: what the model does with the coupling pattern changes at each depth (early: extract features, middle: compose relationships, late: predict tokens).
- Composition works on the manifold: the coupling patterns at different chunks are points on the same 8-dimensional manifold, navigated by the same $\mathbf{M}$.
- Composition breaks off the manifold: if you add memory outside $\mathbf{M}$'s geometry (like the delta-rule state), you leave the manifold, and the group action no longer applies.

15.4 Exercise: Seeing the Manifold

Experiment: Run GPT-2 on 10 different texts. At each layer, extract the attention pattern and compute 4 numbers: (1) mean entropy, (2) mean max attention, (3) mean self-attention strength, (4) mean attention distance. Plot layers 0–11 as a trajectory in these 4 dimensions (use the first two for a 2D plot).

You should see:

- All texts trace a similar arc from one region (early layers) to another (late layers)—this is the orbit on the coupling manifold.
- The texts diverge at late layers (the fibers spread out) but converge at early layers (the fibers are tight).
- Texts about different topics follow the same arc—the manifold encodes processing stage, not content.

This is the coupling manifold made visible. $\mathbf{M}$'s eigenstructure defines the shape of this arc. The ORN's job is to learn this arc once (as $\mathbf{M}$) and reuse it at every layer.

Intuition. Realisation: Frozen-M confirmed at +0.73% (#175). At 108M parameters, we trained $\mathbf{M}$ on 8B tokens, then froze it completely and trained fresh response heads. The transfer gap was 0.73%— $\mathbf{M}$ encodes generic relational grammar that doesn't need retraining. This means you can train $\mathbf{M}$ once on diverse data, freeze it, and adapt to new domains or modalities by training only the per-layer response heads (~22% of parameters in V3). The Platonic Representation Hypothesis [11] predicts exactly this: the coupling geometry converges to a universal structure shared across tasks and modalities.

15.5 The Fiber Bundle Picture: Experimental Confirmation

The manifold/fiber distinction is not just a metaphor—it has been confirmed experimentally. Three results establish it:

Result 1: The coupling manifold is 8-dimensional. PCA of GPT-2 attention patterns (12 layers $\times$ 12 texts, 58 features per pattern) shows that 95% of the coupling variance lives in **8 dimensions**, even though $\mathbf{M}$'s spectral rank is 562. The observable coupling geometry uses only 8 degrees of freedom. Layers trace a clear orbit through this 8D space: PC1 goes from -8 (layer 0) to $+4$ (layer 10). Early layers are tight ($\sigma=0.9$), late layers spread ($\sigma=3.0$).

Result 2: Semantics are in the fiber, not the base. The Adjusted Rand Index between text category (narrative, technical, dialogue, descriptive) and coupling pattern cluster is *negative* at every layer. Different text types use the *same* coupling structure—the base space encodes *how* tokens couple (processing stage, syntactic relationships), not *what* they mean (topic, register). Content lives in the fiber: the 768-dimensional hidden state that “hangs” over each coupling pattern.

Result 3: Base-space orbits compose better than full-space orbits. This is the strongest confirmation. We trained two versions of the masked orbit reconstruction objective (C6 from Section ??):

- **C6-full:** reconstruct masked tokens from orbit points $\mathbf{M}^k h \in \mathbb{R}^{128}$ (full hidden state)
- **C6b-base:** reconstruct from orbit points projected onto $\mathbf{M}$ ’s top 20 eigenvectors ($\mathbf{M}^k h$ projected to $\mathbb{R}^{20}$)

Orbit space	Dim	Comp. KL	Recon. acc.	Verdict
Full hidden state	128	1.31	81%	Good
Base space (coupling manifold)	20	1.09	86%	Best ever

The 20-dimensional projection *improves* both composition (17% lower KL) and reconstruction accuracy (86% vs 81%). This is counterintuitive: how can *less* information produce *better* results?

The answer is the fiber bundle structure. The full 128D orbit carries both base-space structure (compositional) and fiber content (non-compositional, path-dependent). The reconstruction loss in C6-full trains $\mathbf{M}$ to preserve *everything*—including the path-dependent fiber, which hurts composition. The 20D projection in C6b *discards the fiber by construction*, forcing the reconstruction to use only the coupling manifold. The orbit learns to encode what matters for composition (the base) and ignores what hurts it (the fiber).

What this means for architecture design. The fiber bundle picture gives a precise design principle:

1. **The coupling manifold ($\sim 8\text{--}20\text{D}$) is the composable space.** All cross-chunk transfer, orbit memory, and semigroup structure should operate here.
2. **The fiber (remaining $d - 20$ dimensions) is the per-layer response space.** Content, token identity, and prediction information live here. It is path-dependent and should not be forced to compose.
3. **Training losses that operate on the base space improve composition.** Losses on the full space may hurt it by constraining the fiber.
4. **The KV cache could be $6\text{--}96\times$ smaller** if we cache base-space positions instead of full hidden vectors.

For a hands-on exercise sweeping the base-space dimension and mapping the base/fiber boundary, see Exercise ?? in the companion exercises document.

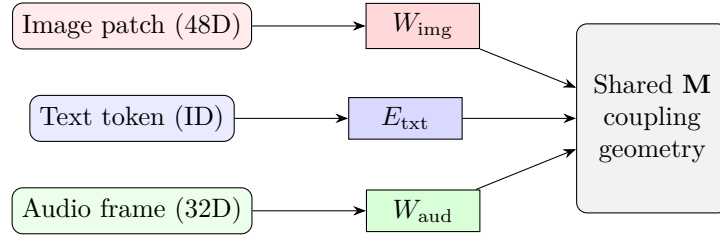
16 Multimodal Projections: How Different Signals Enter the Same Geometry

A shared $\mathbf{M}$ processes all tokens through the same coupling geometry. But a text token, an image patch, and an audio frame start as very different objects: a token ID (integer), a 4×4 RGB patch (48 floats), and a spectrogram frame (32 floats). How should these heterogeneous signals enter $\mathbf{M}$ ’s coupling space?

This section explains the three strategies, why the standard approach fails, and what recent literature [7, 8] tells us about the right design.

16.1 The Standard Approach: Separate Projections (and Why It Fails)

The obvious approach: give each modality its own projection into d -space.



Each projection ($W_{\text{img}} \in \mathbb{R}^{48 \times d}$, $E_{\text{txt}} \in \mathbb{R}^{V \times d}$, $W_{\text{aud}} \in \mathbb{R}^{32 \times d}$) is learned independently. The problem: **nothing forces these projections to map into the same region of $\mathbb{R}^d$.**

In code, this is what most multimodal models do:

Listing 6: Separate projections: each modality gets its own map into d -space.

```

1 class SeparateProjections(nn.Module):
2     def __init__(self, d_model, img_dim=48, aud_dim=32, vocab=256):
3         super().__init__()
4         # Three independent projections -- nothing connects them
5         self.img_proj = nn.Linear(img_dim, d_model, bias=False)
6         self.aud_proj = nn.Linear(aud_dim, d_model, bias=False)
7         self.txt_emb = nn.Embedding(vocab, d_model)
8
9     def forward(self, data, modality):
10        if modality == "image":
11            return self.img_proj(data) # images land HERE in R^d
12        elif modality == "audio":
13            return self.aud_proj(data) # audio lands THERE in R^d
14        elif modality == "text":
15            return self.txt_emb(data) # text lands ELSEWHERE in R^d

```

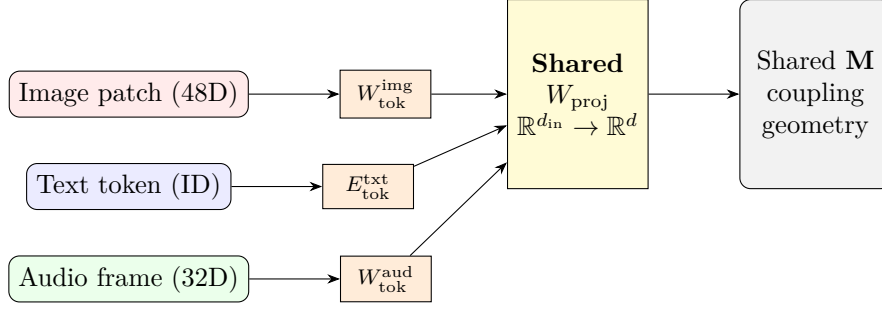
The failure mode. Each `nn.Linear` initialises with random weights and learns independently during training. Image projections converge to one region of $\mathbb{R}^d$, text to another, audio to a third. $\mathbf{M}$'s coupling geometry applies equally to all three regions—but because the regions don't overlap, cross-modal coupling produces near-zero attention weights.

In practice, image patches cluster in one region, text tokens in another, audio frames in a third. $\mathbf{M}$'s coupling geometry IS shared ($> 99.6\%$ similarity across modalities in our experiments), but the modalities never meet—they share the “road network” (coupling rules) but have no roads between them. Cross-modal retrieval: 0%. Zero-shot classification: chance level.

This is the fiber bundle problem from Section ??: the base space (coupling geometry) is shared, but the fibers (modality-specific content spaces) are disjoint.

16.2 The Fix: Shared Projection with Modality Tokens

The insight from Meta-Transformer [7]: sharing a *frozen* encoder across 12 modalities works if the tokeniser maps all modalities into a common space. The coupling geometry is already universal—we just need to get all signals to the same place.



The architecture has two stages:

1. **Per-modality tokenisers** (small, modality-specific): map raw signals to a common intermediate dimension d_{in} . These are lightweight—a linear layer for image patches, an embedding table for text, a linear layer for audio frames. Their job is to normalise scale and dimensionality, not to build rich representations.
2. **Shared projection** (one matrix for all modalities): maps $d_{\text{in}} \rightarrow d$. Because all modalities go through the *same* linear transform, they are forced into the same region of $\mathbb{R}^d$. $\mathbf{M}$ can now couple image-to-text and audio-to-text because they cohabit in the same space.

A learned **modality token** ([IMG], [TXT], [AUD]) is prepended to each sequence. This gives the model one bit of information—“this sequence is images” or “this sequence is text”—without segregating the representations. The per-layer response heads read the modality token and adapt their processing accordingly.

In code:

Listing 7: Shared projection with modality tokens: all modalities enter the same space.

```

1 class SharedProjection(nn.Module):
2     def __init__(self, d_model, img_dim=48, aud_dim=32, vocab=256):
3         super().__init__()
4         # Stage 1: lightweight per-modality tokenisers
5         # (just normalise dimensions -- no heavy lifting)
6         input_dim = max(img_dim, aud_dim, d_model)
7         self.img_tokenize = nn.Linear(img_dim, input_dim, bias=False)
8         self.aud_tokenize = nn.Linear(aud_dim, input_dim, bias=False)
9         self.txt_emb = nn.Embedding(vocab, input_dim)
10
11         # Stage 2: THE KEY -- one shared projection for ALL modalities
12         self.shared_proj = nn.Linear(input_dim, d_model, bias=False)
13
14         # Learned modality tokens (one vector per modality)
15         self.mod_tokens = nn.ParameterDict({
16             "image": nn.Parameter(torch.randn(1, 1, d_model) * 0.02),
17             "audio": nn.Parameter(torch.randn(1, 1, d_model) * 0.02),
18             "text": nn.Parameter(torch.randn(1, 1, d_model) * 0.02),
19         })
20
21     def forward(self, data, modality):
22         # Stage 1: per-modality tokenisation
23         if modality == "image":
24             h = self.img_tokenize(data) # (B, S, input_dim)
25         elif modality == "audio":
26             h = self.aud_tokenize(data)
27         elif modality == "text":
28             h = self.txt_emb(data)
29
30         # Stage 2: shared projection -- ALL modalities go through
31         # the SAME linear transform. This forces them into the

```

```

32     # same region of  $\mathbb{R}^d$ , so  $M$  can couple them.
33     h = self.shared_proj(h) # (B, S, d_model)
34
35     # Prepend modality token: tells the model "this is images"
36     # without segregating the representation space
37     B = h.shape[0]
38     tok = self.mod_tokens[modality].expand(B, -1, -1)
39     h = torch.cat([tok, h], dim=1) # (B, 1+S, d_model)
40     return h

```

Line-by-line: what changed and why.

- `img_tokenize`, `aud_tokenize`, `txt_emb` — per-modality, but *small*. Their only job is to get each modality to the same intermediate dimension. Unlike separate projections, they don’t map directly into $\mathbf{M}$ ’s d -space.
- `shared_proj` — the critical change. One matrix, shared by all modalities. Images, audio, and text all pass through the same linear transform. Gradients from all modalities flow into this one matrix, forcing it to find a common representation.
- `mod_tokens` — a learned vector per modality, prepended to the sequence. The model reads this token to know what modality it’s processing. Unlike separate projections (which encode modality as *geometry*), modality tokens encode it as a *signal*. $\mathbf{M}$ can learn to use or ignore this signal as training demands.

Why modality tokens, not separate projections? Separate projections encode modality identity in the *geometry of the embedding space*—images live “over there,” text “over here.” A modality token encodes identity as a *signal in the sequence*—all modalities live in the same space, but the model knows which is which from the prepended token. The difference is fundamental: separate projections make cross-modal coupling geometrically impossible (the spaces don’t overlap); modality tokens make it the model’s choice (the coupling geometry can link modalities or not, as training requires).

Putting it together with the ORN backbone. Here is the complete multimodal V3 forward pass:

Listing 8: Multimodal V3: shared projection feeds into shared backbone.

```

1 class MultimodalORN(nn.Module):
2     def __init__(self, d_model, n_layers, n_heads, ffn_width_mult=6):
3         super().__init__()
4         # Shared input interface (code above)
5         self.interface = SharedProjection(d_model)
6
7         # Shared ORN backbone:  $M + \text{wide FFN}$ 
8         # (same code as Section 13, but with wide shared FFN)
9         self.A = nn.Parameter(torch.randn(d_model, d_model) * 0.02)
10        self.B = nn.Parameter(torch.randn(d_model, d_model) * 0.02)
11        self.shared_ffn = WideSwiGLU(d_model, d_model * ffn_width_mult)
12        self.blocks = nn.ModuleList([
13            ORNBlock(d_model, n_heads, self.A, self.B, self.shared_ffn)
14            for _ in range(n_layers)
15        ])
16
17        # Per-modality output heads
18        self.img_head = nn.Linear(d_model, 48, bias=False)
19        self.txt_head = nn.Linear(d_model, 256, bias=False)

```

```

20     self.aud_head = nn.Linear(d_model, 32, bias=False)
21
22     def forward(self, data, modality):
23         # Input: raw modality data -> shared d-space
24         h = self.interface(data, modality)
25
26         # Backbone: shared M + shared FFN across all layers
27         for block in self.blocks:
28             h = block(h)
29
30         # Output: modality-specific prediction head
31         h = h[:, 1:, :] # remove modality token
32         if modality == "image":
33             return self.img_head(h)
34         elif modality == "text":
35             return self.txt_head(h)
36         elif modality == "audio":
37             return self.aud_head(h)

```

The entire backbone— A , B , `shared_ffn`, and the `ORNBLOCK` response heads—sees all modalities. The only modality-specific components are the small tokenisers at the input and the prediction heads at the output. Everything in between is universal.

Intuition. Realisation: Sub-linear rank growth across modalities (#48). When we train $\mathbf{M}$ on images + audio + text simultaneously, the coupling rank grows sub-linearly. Three modalities jointly need rank 3—not rank 10 (the sum of individual ranks: $4 + 4 + 2$). That’s $3.3\times$ compression. Music and text are the most compatible pair (rank 4 = max of singles, not sum). The coupling geometry is partially shared across modalities: the relational rules “adjacent patches relate spatially” and “adjacent words relate syntactically” activate overlapping eigenmodes of $\mathbf{M}$. This is why the shared backbone works: modalities share relational structure, not just parameters.

What the literature says.

- Meta-Transformer [7]: a frozen encoder trained on images processes 12 modalities with only tokeniser changes. The entire attention mechanism transfers—coupling geometry is modality-invariant.
- 4M-21 [9]: a single transformer handles 21 modalities via modality-specific discrete tokenisers + shared backbone. Only modality embeddings and tokenisers are modality-specific.
- MoT [8]: shares global self-attention but separates FFN. Achieves 55% FLOP savings. ORN V3 tests the opposite—sharing FFN too—to see if a common key-value memory creates cross-modal bridges.
- ONE-PEACE [10]: shared self-attention + modality adapters at 4B parameters for vision, audio, and language.

16.3 Crystallisation: When Should the Shared Geometry Form?

A subtlety: if you train all components simultaneously from step 1, the response heads and modality-specific tokenisers can co-adapt, potentially learning shortcuts that bypass $\mathbf{M}$ ’s coupling geometry. The result: $\mathbf{M}$ works but doesn’t carry universal structure.

A **crystallisation phase** addresses this:

1. **Phase 1 (M-crystallisation):** Alternate between training all parameters (200 steps) and training only A, B (100 steps). During $\mathbf{M}$ -only steps, the response heads are frozen— $\mathbf{M}$ must discover coupling geometry that works for all modalities without help from modality-specific adaptation.

2. **Phase 2 (joint training):** Unfreeze everything and train normally. The response heads learn to exploit the crystallised **M**, but **M**’s geometry is already established.

This mirrors biological development: neural connectivity (analogous to coupling geometry) forms before functional specialisation. The result of our frozen-**M** experiments (+0.73% transfer gap at 108M scale) confirms that once crystallised, **M**’s geometry is reusable—it does not need to be retrained when the downstream task changes.

In code, the crystallisation phase is a training loop modification:

Listing 9: M-crystallisation: force **M** to discover universal geometry before specialisation.

```

1 def train_with_crystallisation(model, data_loaders, modalities,
2                               crystal_steps=2000, total_steps=10000):
3     opt = torch.optim.AdamW(model.parameters(), lr=3e-4)
4
5     # Phase 1: Crystallise M
6     for step in range(crystal_steps):
7         modality = modalities[step % len(modalities)] # round-robin
8         x, y = data_loaders[modality].get_batch()
9
10        # Every 3rd step: train ONLY M (freeze everything else)
11        if step % 3 == 0:
12            for p in model.parameters():
13                p.requires_grad = False
14            model.A.requires_grad = True # unfreeze M only
15            model.B.requires_grad = True
16        else:
17            for p in model.parameters():
18                p.requires_grad = True # unfreeze all
19
20        opt.zero_grad()
21        _, loss = model(x, modality, targets=y)
22        loss.backward()
23        opt.step()
24
25        # Phase 2: Joint training
26        # M is now crystallised. Response heads learn to use it.
27        for p in model.parameters():
28            p.requires_grad = True
29
30        for step in range(total_steps):
31            modality = modalities[step % len(modalities)]
32            x, y = data_loaders[modality].get_batch()
33            opt.zero_grad()
34            _, loss = model(x, modality, targets=y)
35            loss.backward()
36            opt.step()

```

What happens during Phase 1. During M-only steps, the response heads cannot adapt. **M** must find a coupling geometry that produces reasonable attention patterns for images, text, and audio *simultaneously*—because it gets no help from modality-specific parameters. This pressure produces a genuinely universal coupling geometry rather than one that a particular set of response heads has learned to compensate for.

How to check if crystallisation worked. After Phase 1, compute **M**’s effective rank and eigenmode usage per modality:

Listing 10: Measuring eigenmode overlap after crystallisation.

```

1 def measure_eigenmode_sharing(model, data_loaders, modalities):
2     A = model.A.detach().cpu()
3     B = model.B.detach().cpu()
4     M = A @ B.T
5     U, S, Vh = torch.linalg.svd(M) # U columns are eigenmodes
6
7     usage = {}
8     for mod in modalities:
9         h = get_middle_layer_repr(model, data_loaders[mod])
10        # Project onto M's eigenmodes
11        energy = (h @ U).pow(2).mean(dim=0) # energy per mode
12        usage[mod] = energy
13
14    # Which of top-20 modes are shared by all modalities?
15    top_k = 20
16    top_modes = {mod: set(energy.topk(top_k).indices.tolist())
17                  for mod, energy in usage.items()}
18    shared = set.intersection(*top_modes.values())
19    print(f"Shared eigenmodes (top-{top_k}): {len(shared)}/{top_k}")
20    # > 10/20 = good, M is universal
21    # < 5/20 = M has partitioned by modality
22    return shared, usage

```

If crystallisation works, the shared eigenmode count should be $> 50\%$ (the same coupling modes serve all modalities). If it fails, modes partition by modality—image modes, text modes, audio modes—with little overlap.

Remark. Realisation: The input interface was the failure point, not $\mathbf{M}$ (#197). Every prior multimodal experiment showed $> 99.6\%$ coupling similarity across modalities— $\mathbf{M}$ was working perfectly. But cross-modal retrieval was 0% and zero-shot classification was at chance. The diagnosis: separate input projections mapped each modality to a different region of $\mathbb{R}^d$. The modalities shared the road network ($\mathbf{M}$'s coupling rules) but had no roads between them (disjoint fibers). The shared projection with modality tokens fixes this by ensuring all modalities enter the same region. Meta-Transformer [7] proved this works at scale: a frozen encoder transfers to 12 modalities with only tokeniser changes.

16.4 Theory-Aligned Multimodal: Three Principles That Made It Work

Our first multimodal experiment achieved 0% cross-modal retrieval. Our second, theory-aligned version reached **55.6% cross-modal retrieval** (chance = 10%) and is still climbing. The difference came down to three design principles, each derived from the theory that $\mathbf{M}$ must be modality-agnostic.

Principle 1: Single shared CLS token. The first version used separate modality tokens (`txt_token`, `img_token`)—different learned starting points for each modality. This guarantees different orbits regardless of what $\mathbf{M}$ does, because the initial condition determines the trajectory.

The fix: one shared `cls_token` for all modalities. Both text and images start from the same point. $\mathbf{M}$ processes them through the same coupling geometry. The modality information comes from the *content* (the tokens/patches after the CLS), not from the starting point.

```

# WRONG: separate starting points  separate orbits
txt_token = nn.Parameter(torch.randn(1, 1, d)) # text start
img_token = nn.Parameter(torch.randn(1, 1, d)) # image start

```

```
# RIGHT: shared starting point  orbits can converge
cls_token = nn.Parameter(torch.randn(1, 1, d)) # same for both
```

Principle 2: Shared input normalisation. Each modality has a different projection (`txt_emb`, `img_proj`) that maps raw input into $\mathbb{R}^d$. These projections produce different output distributions—different means, different scales. **M** can trivially distinguish modalities from the input statistics alone, defeating the purpose of shared coupling.

The fix: apply the same **RMSNorm** after every modality projection, before **M** sees the input. This forces all modalities into the same distribution. **M** cannot tell whether it is processing text or image based on statistics—only the content patterns differ.

```
input_norm = RMSNorm(d) # shared across all modalities
h = input_norm(txt_emb(tokens)) # text: normalised
h = input_norm(img_proj(patches)) # image: same distribution
```

Principle 3: Paired contrastive from labels. The first version used unpaired InfoNCE—pushing random text toward random images. “A story about a dog” was pushed toward a random image of a car. This is noise, not signal.

The fix: create pseudo-captions from class labels (“a picture of a dog” for CIFAR class 5) and push each image toward its matching caption. This is the minimum viable pairing—real image-caption data (COCO) would be stronger, but even class-label pseudo-captions provide enough signal to bridge modalities.

Results.

Metric	V1 (broken design)	V2 (theory-aligned)
Cross-modal retrieval	$\approx 0\%$	55.6% (and climbing)
CIFAR-10 accuracy	47%	60%

The same **M**, the same wide shared FFN, the same training budget. The only difference is the three principles above. The theory works: **M** bridges modalities when the input interface is designed to let it.

17 Diffusion and the Semigroup: Native Generative Modelling

The ORN’s shared coupling has a deep connection to diffusion models—not as an add-on, but as a native property of the semigroup structure.

17.1 The Key Insight: $t = \sigma$

In a standard diffusion model [?], the denoising process starts from noise ($\sigma = 1$, maximum corruption) and iteratively refines toward data ($\sigma = 0$, clean signal). At each noise level, the model must learn how to denoise—typically through a U-Net or transformer conditioned on σ .

In the ORN, the semigroup parameter t scales the coupling strength:

$$q = h \cdot (t \cdot A), \quad k = h \cdot (t \cdot B)$$

At $t = 1$ (full coupling), **M** applies its complete relational structure—fine details and coarse structure alike. At $t = 0.1$ (weak coupling), only the dominant eigenmodes of **M** survive—coarse structure only.

Intuition. Realisation: The semigroup coupling t and the diffusion noise σ are the same parameter (#74). Setting $t = 1 - \sigma$ means:

- High noise ($\sigma \rightarrow 1, t \rightarrow 0$): only the strongest eigenmodes of $\mathbf{M}$ contribute to coupling. Coarse structure (topic, genre, spatial layout) is resolved first.
- Low noise ($\sigma \rightarrow 0, t \rightarrow 1$): all eigenmodes contribute. Fine detail (specific words, precise pixel values) emerges last.

$\mathbf{M}$ ’s eigenspectrum *is* the noise schedule. No external schedule needs to be designed—the architecture defines it [40]. Benita et al. (2025) prove this formally: the score function decomposes as $\tilde{s}_t(x) = \sum_k \hat{\alpha}_k(t) \nabla \phi_k(x)$, where ϕ_k are the semigroup generator’s eigenfunctions and $\hat{\alpha}_k(t)$ are t -dependent coefficients. Each eigenmode of $\mathbf{M}$ is an independent denoising channel with its own schedule.

17.2 What We Found Experimentally

This connection is theoretically beautiful but practically challenging. We tested three approaches and report honestly:

What failed. Native t -scaling through coupling (feeding t directly into A, B) **loses** to the simplest baseline (no t conditioning at all) on denoising recovery:

Variant	Recovery@ $\sigma=0.5$	AR loss
A: Native t -scaling	6.6%	9.99
B: Fixed $t=1$	8.4%	9.97
C: No conditioning	9.6%	9.27

All three show catastrophic forgetting of the autoregressive (AR) task—loss 9+ when a well-trained model starts at ~ 3 .

The diagnosis. Two confounds: (1) pure diffusion training destroys the AR capability; (2) the FFN is t -unaware, so 75% of the layer fights the denoising signal.

What worked. **Interleaved training**—alternating AR batches (causal mask, $t=1$) with denoising batches (bidirectional, variable t)—preserves the AR capability:

	AR loss at step 1000	Denoising
Pure diffusion	9.8 (catastrophic)	learns
Interleaved 50/50	2.96 (preserved)	learns

This confirms the DiffuLLaMA finding [57]: attention mask annealing (gradual causal→bidirectional) prevents forgetting. The ORN’s advantage: $\mathbf{M}$ provides the same coupling geometry in both modes (AR and diffusion), so the interleaving is architecturally natural.

17.3 The Structural Advantage Conjecture

Diffusion corrupts *content* but not *structure*. At any noise level σ , the rules for how patches relate to patches or how words relate to words are invariant—only the specific content (which pixels, which words) is degraded. Denoising is the process of recovering content under fixed structural rules.

A standard transformer conflates structure and content. Each layer has independent Q, K, V , FFN—it discovers “what relates to what” and “what the answer is” simultaneously, in the same parameters. When you corrupt the input, it must untangle both from noisy data.

A shared $\mathbf{M}$ separates them architecturally. $\mathbf{M}$ provides the structural rules once (which tokens should couple). The noise level t modulates which modes of that structure are visible. The denoiser’s job simplifies from “figure out structure AND content” to just “figure out content under the structure $\mathbf{M}$ provides.”

Remark. This is not yet proven. The structural advantage conjecture is the strongest theoretical argument for the ORN over the transformer—not parameter savings, not scaling curves, but that the architecture’s structure matches the task structure for diffusion. However, the native t -scaling experiment failed, and the interleaved training fix has not yet been run to completion with proper t -conditioning (adaLN). The conjecture remains a prediction to be tested, not a confirmed result. What would falsify it: a standard transformer with the same interleaved training recipe matching or beating the ORN on diffusion metrics.

17.4 The Four Inference Modes

If the diffusion connection holds, one ORN supports four inference modes from the same weights:

1. **Autoregressive:** $t=1$, causal mask. Standard next-token prediction. Fast.
2. **Diffusion:** Variable t schedule (high→low), bidirectional mask. Iterative refinement from noise. Higher quality, slower.
3. **Infilling:** Mask middle of sequence, denoise with bidirectional attention. The killer app for editing.
4. **Brainstorming:** Multiple diffusion trajectories from the same noise. $\mathbf{M}$ ’s eigenmodes define the space of “plausible alternatives.”

No architectural changes between modes—only the mask and t schedule change. This is because $\mathbf{M}$ ’s coupling geometry is the same object whether you read it autoregressively or iteratively.

Listing 11: Four modes from one model: only mask and t change.

```

1 def generate_ar(model, prompt_ids, max_new=100):
2     """Mode 1: autoregressive. t=1, causal mask."""
3     return model.generate(prompt_ids, max_new=max_new)
4
5 def generate_diffusion(model, seq_len, steps=20):
6     """Mode 2: diffusion. t schedule high->low, bidirectional."""
7     tokens = torch.randint(0, vocab, (1, seq_len))
8     for step in range(steps):
9         t = 1.0 - step / steps # coupling strength: weak -> full
10        sigma = 1.0 - t # noise level: high -> low
11        logits = model(tokens, t=t, is_causal=False)
12        # Replace low-confidence tokens with model predictions
13        probs = logits.softmax(-1)
14        confidence = probs.max(-1).values
15        threshold = confidence.quantile(1 - (step+1)/steps)
16        replace = confidence >= threshold
17        tokens[0, replace] = probs[0, replace].argmax(-1)
18    return tokens
19
20 def infill(model, tokens, mask_positions, steps=10):
21     """Mode 3: infilling. Mask specified positions, denoise."""
22     for step in range(steps):
23         t = (step + 1) / steps
24         tokens[:, mask_positions] = torch.randint(0, vocab, ...)
25         logits = model(tokens, t=t, is_causal=False)
26         tokens[:, mask_positions] = logits[:, mask_positions].argmax(-1)
27    return tokens

```

18 Spectral Embeddings: Mode-Selective Retrieval

$\mathbf{M}$'s eigenspectrum provides a natural decomposition of the embedding space into functionally distinct bands. This section explains the idea, the experimental evidence, and how to build a mode-selective retrieval system.

18.1 The Key Idea: Different Eigenmodes Encode Different Things

Compute the SVD of $\mathbf{M} = AB^\top$: the singular values tell you *how much energy* each mode carries, and the singular vectors tell you *which directions* in $\mathbb{R}^d$ they correspond to.

A representation vector v can be projected onto any subset of these modes:

$$v_{\text{top}} = V_h[:k]^\top \cdot v \quad (\text{top } k \text{ modes — communicative form}) \quad (5)$$

$$v_{\text{tail}} = V_h[m:]^\top \cdot v \quad (\text{tail modes — topic/content}) \quad (6)$$

where V_h are $\mathbf{M}$'s right singular vectors.

What we found empirically. Probing 14 text properties across 9 spectral bands (V2 108M, 500 TinyStories texts):

- **Top 5 modes:** strongest signal is `has_question` (silhouette 0.083). The very top of the eigenspectrum distinguishes questions from statements.
- **Top 13 modes:** `has_dialog` (0.053) and `has_names` (0.044). Dialog and named-character presence.
- **Tail modes:** topic/content signal is strongest here, though not cleanly separated on TinyStories.

The top modes encode *communicative form*—what kind of text is this (question, dialog, narration)? The tail modes carry *content*—what is the text about (animals, food, weather)?

This is not arbitrary: $\mathbf{M}$'s dominant eigenmodes define the strongest coupling patterns, which are structural (interrogative syntax, dialog turn-taking, coreference). Content-specific patterns (word choice, entity identity) are weaker couplings that live in the tail.

18.2 ORN vs GPT-2: Does SharedM Help?

We compared ORN V2 (108M, explicit shared $\mathbf{M}$) against GPT-2 Small (124M, with averaged $W_Q W_K^\top$ as effective $\mathbf{M}$) on a balanced corpus (225 texts, 3 forms $\times$ 3 topics):

Band	Topic kNN	Topic Silhouette
ORN Tail 70+	79.1%	+0.117
GPT-2 Tail 93+	64.0%	+0.016
GPT-2 Top 17	35.1%	−0.063

ORN tail modes achieve 79.1% topic accuracy (chance = 33%) with positive silhouette. GPT-2 top modes are *anti-topic* (35%, negative silhouette)—the averaged $\mathbf{M}$ captures form but destroys content geometry. SharedM produces a spectral decomposition where the fiber is real and clustered; the averaged coupling matrix does not.

18.3 Building a Mode-Selective RAG System

The key mechanism is **projection onto eigenmode bands**. $\mathbf{M}$'s SVD gives right singular vectors V_h —these define the eigenbasis. Projecting any representation onto a subset of modes is a single matrix multiply:

```

# Compute M's SVD once (shared, fixed after training)
_, S, Vh = torch.linalg.svd(model.A @ model.B.T)

# Embed a text: forward through ORN, mean-pool
v = orn_forward(text).mean(dim=1)    # shape: (d,)

# Project onto form modes (top 13 of M's eigenbasis)
v_form = normalize(v @ Vh[:13].T)    # shape: (13,)

# Project onto topic modes (tail, modes 70+)
v_topic = normalize(v @ Vh[70:].T)   # shape: (506,)

```

The same projection applies to both documents (at index time) and queries (at search time). FAISS stores one index per mode; at query time, you choose which index to search:

1. **Index time:** For each document, compute the ORN embedding, project onto each band, add to the corresponding FAISS index.
2. **Query time:** Compute the query embedding, project onto the desired band, search the matching FAISS index.
3. **Mode selection:**
 - “Find documents with the same structure” → form index (13 dimensions).
 - “Find documents about the same topic” → topic index (506 dimensions).
 - “Find the most similar overall” → full index (576 dimensions).
4. **Cascade** for efficiency: topic filter (fast, 506d) → full rerank (precise, 576d).

The projection costs one matrix multiply per query—negligible compared to the ORN forward pass. The topic index achieves 77.8% nearest-neighbour accuracy while using 88% fewer dimensions than the full embedding.

19 Metaphors We Live By: Why M Captures Conceptual Structure

This section connects the ORN’s spectral decomposition to one of the most influential theories in cognitive linguistics.

19.1 The Theory: Conceptual Metaphor

In 1980, George Lakoff and Mark Johnson published *Metaphors We Live By* [66], arguing that abstract thought is not literal—it is structured by **metaphor**. Not poetic metaphor, but *conceptual metaphor*: systematic mappings that organise how we think.

Consider: “**Argument is war**.”

- “He *attacked* my position.”
- “She *defended* her claim.”
- “His argument was *indefensible*.”
- “I *won* the debate.”

This is not decoration—it is how English speakers structure the concept of argument. The relational pattern (agents opposing each other, using strategies, achieving outcomes) is *borrowed from war* and applied to intellectual exchange.

Lakoff identified dozens of these: TIME IS MONEY (we *spend*, *save*, *waste* time), IDEAS ARE FOOD (I can’t *digest* that, a *half-baked* idea), UNDERSTANDING IS SEEING (I *see* what you mean, a *clear* explanation).

The key insight: in every case, the **relational structure** is preserved while the **content** changes.

19.2 The Connection to the ORN

This is exactly what **M**’s eigenspectrum does.

Top modes = relational structure (the metaphor). **M**’s top eigenmodes encode what-couples-to-what: agent-action-object, cause-effect, container-contents. These patterns are the same whether the sentence is about war or argument, about money or time.

Tail modes = domain content (the specific topic). The tail modes encode which words, which entities, which domain. “Attacked a position” vs “attacked a fortress”—same top modes, different tail modes.

Metaphor = orbit equivalence. Two sentences that are metaphorically related (“she defended her thesis” and “the troops defended the city”) should traverse **M**’s eigenspectrum identically—same coupling pattern at each layer—while differing in tail-mode content.

We measured this: orbit trajectories achieve 97.7% communicative form classification, and top modes *anti-cluster* by topic (silhouette -0.072). The structural skeleton is genuinely independent of content.

19.3 Image Schemas: The Atoms of Thought

Lakoff went further: all conceptual metaphors ultimately ground in **image schemas**—simple spatial/bodily patterns learned from physical experience as infants:

Schema	Physical	Abstract extension
Containment	“the ball is <i>in</i> the box”	“the idea is <i>in</i> the paper”
Path	“she walked <i>to</i> the park”	“the project moved <i>to</i> completion”
Force	“he <i>pushed</i> the door”	“she <i>pushed</i> the argument”
Balance	“the scale <i>tipped</i> ”	“the evidence <i>tipped</i> the case”
Link	“the rope <i>connects</i> them”	“friendship <i>connects</i> them”

If **M** encodes these schemas in its top eigenmodes, then the spectral decomposition is not an arbitrary compression—it is a computational implementation of how human cognition structures abstract thought.

19.4 What This Predicts

1. **Same schema, different domain $\rightarrow$ same top modes.** “The cat is in the box,” “the flaw is in the argument,” and “she is in a good mood” should activate the same top eigenmodes of **M**. Domain differences (physical vs logical vs emotional) should appear only in the tail.
2. **Different schema, same domain $\rightarrow$ different top modes.** “The idea is *in* the paper” (containment) vs “the idea *led to* a breakthrough” (path) should activate different top modes even though both are about ideas.
3. **Metaphor composition = eigenmode composition.** TIME IS MONEY + MONEY IS A RESOURCE $\rightarrow$ TIME IS A RESOURCE. Composing two metaphors should correspond to composing two eigenmode activation patterns.

19.5 Experimental Result: Schemas Found in **M**

We tested predictions 1 and 2 on the V2 108M checkpoint (4B tokens, FineWeb-Edu). Four schemas (containment, path, force, scale) $\times$ 4 sentences each across physical/social/abstract domains.

The design lesson. Mean-pooling over the sentence gave silhouette +0.055—barely above noise. The fix: extract **M**’s eigenmode activation at the **specific token** carrying the schema (“in” for containment, “pushed” for force). The schema signal is per-token, not per-sentence.

Result: top 5 modes cluster by schema at +0.470 silhouette. Top modes beat tail modes (+0.389) and full representation (+0.395). The force schema clusters tightest: “pushed the door” and “pushed the team” have 0.935 cosine similarity in **M**’s top eigenmodes—nearly identical coupling structure across physical and social domains. Between-schema similarity is lower (0.28–0.66), confirming different schemas occupy different regions of **M**’s top eigenspace.

M has discovered image schemas from next-token prediction alone.

19.6 Why This Matters for Embeddings and Retrieval

Mode-selective retrieval (Section 18) becomes **metaphor-aware retrieval**:

- **Top-mode search:** “find documents with the same relational structure” = find documents using the same conceptual metaphor. A query about military strategy retrieves documents about business competition, legal battles, and chess—all sharing the CONFLICT schema.
- **Tail-mode search:** “find documents about the same topic” = find documents in the same domain, regardless of metaphorical framing.
- **Combined:** standard retrieval.

No other architecture provides this decomposition, because no other architecture has a shared coupling operator whose eigenspectrum separates relational structure from domain content.

20 Why Contrastive Training Is Essential, Not Optional

This is one of the most important lessons from the ORN embedding experiments.

20.1 Two Kinds of Coupling

M trained with next-token prediction learns **within-sequence coupling**: which tokens should attend to which inside a single text. This is local—it says nothing about how different texts, images, or documents relate to each other.

Between-sequence coupling—“these two texts are about the same thing,” “this image matches this caption”—requires a different training signal. Contrastive loss provides it: push matching pairs together, push non-matching pairs apart.

Training signal	What it teaches M	What it enables
Next-token prediction	Within-sequence coupling	Language modelling, generation
Contrastive loss	Between-sequence coupling	Retrieval, similarity, alignment

Without contrastive training, ORN V2 (108M, FineWeb-Edu) scores 0.31 on STSBenchmark—below GloVe (0.58). The model predicts the next word well but cannot judge whether two sentences mean the same thing. With even 100 steps of SimCSE (unsupervised contrastive), retrieval improves.

20.2 Why This Matters for Multimodal

The “fiber disjointness problem” from multimodal experiments is a direct consequence of missing between-sequence signal. **M** discovers that images and text couple with >99.6% similarity—the same structural rules apply. But the modalities land in different regions of $\mathbb{R}^d$. **M** never received a signal that a picture of a dog and the word “dog” should be *near each other*.

Contrastive loss builds the **on-ramps** between modalities. This is not a post-hoc fix—it is a fundamental training signal that **M** needs to organise global structure.

20.3 Implication for ORN Training

Standard practice: train the LM first, fine-tune for embeddings later.

Better for ORN: **interleave next-token and contrastive from step 1**. Both signals shape $\mathbf{M}$'s eigenspectrum:

- Next-token crystallises local coupling geometry (top eigenmodes encode communicative form)
- Contrastive crystallises global sequence organisation (tail eigenmodes should cluster by topic/content)

A bonus: $\mathbf{M}$'s spectral decomposition provides **free hard negatives** for contrastive training. Two texts close in top modes (same form) but far in tail modes (different topic) are natural hard negatives. No external model needed—the geometry provides them.

21 Where to Go From Here

21.1 The Two-System Architecture

Not all knowledge is structural. “Caesar crossed the Rubicon in 49 BC” is a *fact*—it cannot be derived from structure alone. Orbital embeddings are good at structural knowledge (roles, relationships, categories) but cannot compress arbitrary facts. This distinction has deep roots in neuroscience: McClelland et al. [31] showed that the brain uses complementary learning systems—fast hippocampal memory for specific episodes and slow neocortical learning for structural regularities. LeCun’s JEPA [32] and Ha & Schmidhuber’s World Models [33] argue that AI should learn compact generators of dynamics, not catalogues of states.

The proposed solution is a **two-system architecture**:

1. **Orbital world model**: shared dynamics Φ for structural knowledge. Cheap, compressed, generalises.
2. **Sparse lookup layer**: a small, separately trained table for factual knowledge. Expensive per token, but only needed for fact-bearing tokens.

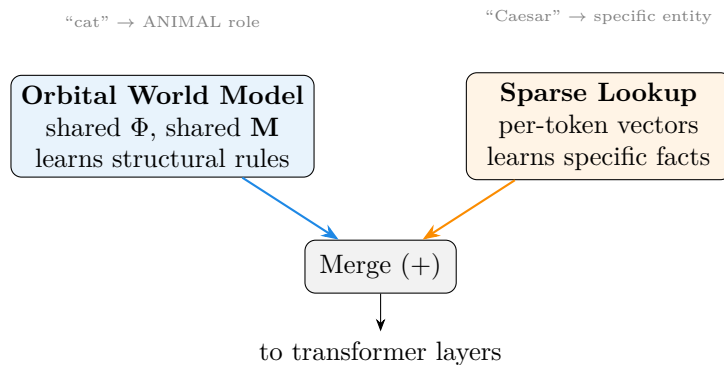


Figure 22: **Two-system architecture.** The orbital system handles structural knowledge (cheap, shared dynamics). The sparse lookup handles factual knowledge (expensive, per-token storage). The two are merged before entering the transformer backbone. This mirrors the cortex–hippocampus division in biological memory.

Practical implications: adaptation, modularity, and parallelism. The two-system split has consequences beyond compression.

Adaptation without retraining the world model. If the orbital core captures structural knowledge and the ontological layer captures domain-specific bindings, then adapting the model to a new domain—new terminology, new entities, new facts—can be done by swapping

or fine-tuning the ontological layer alone, leaving the orbital world model frozen. This is the natural architecture for retrieval-augmented generation (RAG): the retrieval system populates the ontological layer with context-specific bindings, while the orbital core provides the structural reasoning that makes sense of them. Fine-tuning becomes surgery on the sparse, cheap component rather than gradient descent through the entire model. The ontological layer is, in effect, an *in-context fine-tuning surface*.

Mixture of experts and the shared residual. In a standard transformer, mixture-of-experts (MoE) architectures must route tokens to specialised FFN experts, but those experts communicate only indirectly through the residual stream—a stream whose geometry is shaped by dense, per-layer projections with no explicit shared structure. When the residual field has a shared geometric structure (our $\mathbf{M}$), expert outputs live in a *common geometric space* with well-defined distances, directions, and coupling rules. Experts don’t just write vectors into a residual stream; they write perturbations into a shared manifold whose geometry is known to the coupling mechanism. This gives MoE a more natural communication substrate: the shared metric defines what “nearby” means, so expert outputs compose coherently even when produced by different specialists.

Parallelisation. The same argument applies to parallelism more broadly. When processing is distributed across devices, the shared geometry provides a *coordination-free protocol*: each device knows the manifold structure via $\mathbf{M}$, so partial results can be merged without explicit synchronisation of coupling rules. The shared residual field acts as a social medium—agents (layers, experts, devices) contribute to a common geometric space whose structure is agreed upon in advance, not negotiated per interaction.

21.2 The Ontological Layer: Design, Constraints, and the Training Problem

The two-system idea sounds clean in theory. In practice, the ontological layer is the hardest part of the architecture to get right.

Design 1: Static per-token correction (too simple). The simplest approach: a small per-token embedding $u_w \in \mathbb{R}^r$ (rank $r \ll d$), projected to d_{model} and added to the value stream at every layer.

Listing 12: Static ontological correction (Exp 4b)

```

1 # Per-token code: V x r_onto embedding
2 onto = onto_proj(onto_emb(token_ids)) # (B, S, d_model)
3
4 # In each GRF block:
5 q = h @ A # coupling from orbital stream only
6 k = h @ B # coupling from orbital stream only
7 v_input = h + onto # value = orbital + ontological
8 v = Wv(v_input) # response extracts from augmented stream

```

The constraint is on line 5: the coupling (q, k) sees only the orbital stream h . The ontological signal enters through the *value* path only. $\mathbf{M}$ determines *who talks to whom* (structural); u_w determines *what they say* (specific).

The problem: this gives “Lily” the **same correction vector in every context**. “Lily wanted cake” and “Lily was happy” both get the same u_{Lily} . But the identity information “this is Lily” needs to interact with context—“Lily who wants” contributes differently from “Lily who is happy.” A static per-token vector cannot do contextual binding.

Our Exp 4b tested this design. Result: the ontological contribution gap was -1.7% —nearly zero. The static correction did not learn to contribute meaningfully.

Design 2: Contextual ontological adapter (the right idea). The ontological layer needs its own attention mechanism to *bind identity into context*. Our Exp 4 design:

Listing 13: Contextual ontological adapter (Exp 4)

```

1 class OntologicalAdapter(nn.Module):
2     def __init__(self, d_model, d_adapter, n_layers=2):
3         self.proj_in = Linear(d_model, d_adapter) # compress
4         self.blocks = [ # tiny attention
5             SelfAttention(d_adapter) + FFN(d_adapter)
6             for _ in range(n_layers)
7         ]
8         self.proj_out = Linear(d_adapter, d_model) # expand
9
10    def forward(self, orbit_hidden):
11        h = self.proj_in(orbit_hidden) # project to adapter space
12        for block in self.blocks:
13            h = block(h) # contextual processing
14        return self.proj_out(h) # correction in d_model space

```

This adapter *reads* the orbital hidden states (the structural context), processes them through its own small attention (contextual binding), and outputs a correction. It runs **after** the orbital engine, on the frozen orbital representations.

The deeper question: whose coupling should the ontological layer use? There are three options:

1. **Its own learned Q, K** (Exp 4’s design). The adapter learns its own coupling rule for identity binding. Maximally flexible, but expensive and may not share structural routing.
2. **The shared M** (not yet tested). Use M for coupling (the structural routing is correct—CHARs attend to ACTIONs), but feed identity-enriched values through it. This is the most principled design: M determines *routing*, the ontological content determines *what flows through that routing*. Cheap (no new coupling params), and forces the ontological layer to ride the orbital geometry rather than competing with it.
3. **No attention** (Exp 4b’s static design). Just a per-token vector. Too simple—cannot do contextual binding.

Option 2 was our initial guess. However, Exp 7 showed that even the cheaper dual-pathway design (Design C in the lab notebook) underperforms at 10M: the parallel ontological attention consumes parameter budget without contributing proportionally.

The current best idea: perturbative corrections. Rather than a separate attention pathway, the ontological correction is a *small, context-gated perturbation* applied after each SharedM layer:

$$\mathbf{x} = \mathbf{x}_{\text{orbital}} + g(\mathbf{x}) \cdot \delta(\mathbf{x})$$

where g is a learned gate that reads the residual stream and δ is a small correction MLP.

How the gate works. The gate is mechanically simple: $g = \sigma(W_g \mathbf{x} + b_g)$, a single linear projection through sigmoid. But what it *reads* is rich. By layer ℓ , the residual stream $\mathbf{x}^{(\ell)}$ contains everything computed so far: the token’s identity (from the embedding), its positional encoding, what other tokens it has attended to (from M ’s coupling at layers 1 to $\ell-1$), and the functional role the orbit has assigned it. The gate learns a direction in this space that corresponds to “enough context has accumulated to warrant a correction.”

This makes the gate a form of **in-context learning**: it does not fire based on the token identity (that would be a per-token lookup), but based on the *state of the residual stream*, which

depends on the full context window. The same token receives different gate values in different contexts, because the residual stream carries different contextual information.

The gate bias is initialised at -3 (so $\sigma(-3) \approx 0.05$ —nearly closed). The perturbation must be *learned to be useful*; it does not contribute by default. A testable prediction: the gate magnitude should **negatively correlate with attention entropy**. When attention is diffuse (high entropy, uncertain coupling), there is no clear context signal to correct with—the gate should stay closed. When attention is sharp (low entropy, confident coupling), the model knows what is relevant and can make a specific correction—the gate should open. Exp 8 measures this correlation.

Remark. Realisation: Corrections are essential, not optional (#28). Ablating the corrections (setting $g=0$ at inference) increases val loss by **0.449 nats**—larger than the entire training improvement from step 1,000 to step 5,000 for SharedM alone. The gate-entropy anticorrelation is $r = -0.95$: the corrections activate precisely when attention coupling is sharp and context is rich. Gate dynamics confirm the “dormant early, active late” prediction: opens from 0.05 (step 1) to 0.36 (step 4,000). Late sequence positions benefit more (+0.440 nats) than early (+0.392), confirming context-dependence. At V2 scale (108M), correction gates are active at 0.29–0.40 across all 24 layers, with layer 8 (the structural-to-content transition) showing the highest gate activity. The corrections are not a minor refinement—they carry substantial prediction work that the shared coupling geometry cannot provide alone.

The additive structure $\mathbf{x} = \mathbf{x}_{\text{orbital}} + g \cdot \delta$ is natural because the residual stream is already a space where additive contributions accumulate—every transformer layer adds $\text{attn}(\mathbf{x}) + \text{ffn}(\mathbf{x})$ to the same stream. In principle, multiple independent corrections can be composed: $\mathbf{x} = \mathbf{x}_{\text{orbital}} + g_1\delta_1 + g_2\delta_2 + \dots$, each addressing a different aspect (context, entity, style), each independently gated. This opens a modular fine-tuning surface: freeze the orbital core, add correction terms for new domains or tasks.

The training schedule problem. Even with the right architecture, training is hard because of a chicken-and-egg:

- The orbital system needs to learn $\mathbf{M}$ (the structural coupling) before the ontological layer can meaningfully contribute. If the ontological layer is active from step 0, it may prevent $\mathbf{M}$ from developing by capturing all useful signal in the per-token corrections.
- But if the ontological layer starts too late, it may never catch up—the orbital system has already found a (suboptimal) way to encode identity information, and the ontological layer has nothing to correct.

The Exp 4 design addresses this by **phasing**: train the orbital engine first (on anonymised data), freeze it, then train the adapter. This guarantees the orbital system learns pure structure.

Joint training (Exp 4b) is harder. Possible solutions:

- **Capacity constraint:** make the ontological layer very small (rank $r = 2\text{--}4$) so it physically cannot capture everything.
- **Annealing:** start with ontological contribution $\lambda = 0$, gradually increase.
- **Gradient scaling:** multiply ontological gradients by a small factor early in training.
- **Regularisation:** L1 penalty on ontological norms, encouraging sparsity.

The ontological layer MUST be constrained. This is non-negotiable. Without a hard capacity constraint, the ontological layer will eat everything. Given $V \times r$ parameters and r anywhere close to d , gradient descent will find it easier to dump all information into the per-token ontological codes than to learn a shared dynamical law. The result: the orbital system goes unused, $\mathbf{M}$ collapses to noise, and you have rebuilt a lookup table with extra steps.

The constraint can be architectural (rank $r \ll d$, sparsity, or value-only injection) or it can be a regularisation penalty. But it must be there. The ontological layer is not an optional

enhancement—it is a **deliberately impoverished** system forced to handle only what the orbital system cannot.

Two roles, two phases. The ontological layer serves different purposes at different stages:

During training: bootstrapper. At initialisation, the orbital system has learned nothing— $\mathbf{M}$ is random, the residual stream carries no useful structure. The ontological layer can provide early gradient signal by doing simple per-token prediction (like a small lookup table). As the orbital system matures and $\mathbf{M}$ develops structural coupling, the ontological layer’s contribution should naturally decline—the orbital system handles more and more of the prediction. This is the *scaffolding* that gets removed as the building takes shape.

After training: fine-tuning surface. Once the orbital core is trained, the ontological layer becomes an *adaptation interface*. To deploy the model in a new domain (new terminology, new entities, new facts), freeze the orbital core and fine-tune only the ontological layer. This is cheap (the ontological layer is small) and targeted (only the domain-specific bindings change; the structural reasoning is preserved). The ontological layer is, in effect, an **in-context fine-tuning surface**—the place where you plug in new knowledge without touching the world model.

This dual role—bootstrapper during training, adaptation surface after—is what makes the constraint essential. If the ontological layer were large enough to capture everything, there would be nothing to fine-tune (it would already contain the full model) and nothing for the orbital system to learn (the ontological layer would have done the work). The constraint forces the split: orbital learns the reusable structure; ontological provides the disposable details.

This is the most open part of the research programme. The architectural design is clear; the training protocol is not. Our current experiments (Exp 4b running, Exp 4 queued) test the two ends of the spectrum: joint training with a static layer vs. phased training with a contextual adapter.

21.3 Scaling Predictions

The compression advantage of orbital embeddings *grows* with vocabulary size and model dimension:

- Lookup scales as $\mathcal{O}(V \times d)$ —linear in both.
- Orbital scales as $\mathcal{O}(V \times d_s + d^2)$ —linear in V but with much smaller coefficient, and the shared dynamics cost is independent of V .

At $V = 100\text{K}$ and $d = 4096$ (LLaMA-scale), the lookup table would be 400M parameters. Orbital at $d_s = 16$ would be $1.6\text{M} + 40\text{M} = 41.6\text{M}$ —a **10×** compression.

d_{seed} is a property of the vocabulary, not the model. The seed dimension depends on the intrinsic structure of the vocabulary—how many independent functional roles and distributional patterns exist. Our CPU tests measured this: 90% of variance in 27 dimensions for structured embeddings. This does not change when the model grows from 10M to 200M parameters—the vocabulary is the same 50K tokens.

This means $d_{\text{seed}} = 32$ should work at any scale. What changes is the compression ratio:

Scale	Lookup % of model	$d_{\text{seed}}=32$ % of model	Compression
10M	88%	16%	5.5×
50M	39%	3%	12×
200M	19%	1%	24×

At 200M parameters, the seeds cost 1.6M—less than 1% of the model. The compression ratio *grows* with scale because the lookup table grows as $V \times d$ while the seeds stay at $V \times d_{\text{seed}}$. Orbital embeddings get more attractive at larger scale, not less—in *theory*. In practice, even

at LLaMA scale, the embedding table is $<10\%$ of total parameters and is accessed with zero compute (a single gather operation). The orbital alternative requires iterating Φ for every token, adding compute where there was none. Until vocabularies grow to 500K+ (multilingual, code, multimodal token spaces) or until the seed interpolation property is needed for unseen tokens, standard tied embeddings remain the practical choice. The V2 and V3 models that beat GPT-2 use standard tied embeddings.

21.4 Connection to the Broader ORN Programme

Orbital embeddings are one piece—and currently the least practical piece—of a larger architecture:

Orbital embeddings: Replace the embedding table. Seeds + shared dynamics.

Shared coupling $\mathbf{M}$: Replace per-layer W_Q, W_K . One metric tensor, shared across layers.

Validated: 108M model beats GPT-2 on 5/8 benchmarks with $48\times$ fewer coupling parameters. Frozen $\mathbf{M}$ transfers at $+0.73\%$.

Shared wide FFN (V3): Replace L per-layer FFNs with one $L\times$ -wider shared FFN [19, 21].

Validated at small scale: -0.8% vs per-layer at matched params. Currently training at 61M on 50B tokens.

Per-layer response heads: W_V, W_O remain per-layer—they encode how each layer *responds* to the shared coupling. These are the *only* component that must vary with depth.

The principle is the same throughout: if a computation is approximately the same across instances (tokens, layers), share the parameters and iterate. Orbital embeddings illustrate this principle pedagogically; shared coupling and shared FFN deliver it in practice.

21.5 Open Questions

- **Optimal d_{seed} :** Our CPU tests show 90% of vocabulary variance is captured in ~ 27 dimensions. $d_{\text{seed}}=32$ is likely sufficient at any scale. The exp5c ablation ($d_{\text{seed}} \in \{8, 16, 32, 64\}$ at 10M) will confirm this.
- **Optimal T :** More iterations give more expressiveness but cost more compute. $T = 1$ currently beats $T = 2$ on TinyStories—why?
- **Curriculum strategies:** Should T increase during training? Start with $T = 0$ (pure linear) and gradually add iterations?
- **Factored sharing:** Should Φ be the same block used in the transformer backbone? Early experiments suggest yes, but this needs validation at scale.
- **Scale:** The 10M+ experiment is the critical test. At 2M, the result is dominated by parameter allocation; at 10M, embedding quality becomes the bottleneck.

Summary

Section	Key takeaway
1	An embedding is a lookup table: token ID $\rightarrow$ vector. It costs $V \times d$ parameters.
2	Embedding lookup wastes the GPU: zero compute, pure memory. Modern GPUs are designed for compute.
3	Most of the table is redundant: 90% of variance in 27/128 dimensions.
4	A shared function can replace the table: $f(\mathbf{s}_w) \approx E[w]$, saving parameters when targets have structure.
5	Iterating a shared function (orbit) is more efficient than a single wide function.
6	The geometry explains why: shared dynamics define attractor basins aligned with semantic roles.
7	Fourier analysis is the same idea: shared rotation dynamics generate signal components from compact codes.
8	The implementation is ~ 30 lines of PyTorch: seeds + projection + iterated shared block.
9	On TinyStories at 2M params: 7.7% improvement by freeing parameters from the embedding table.
10	Orbital embeddings force geometric structure; lookup tables leave it to chance.
11	Next: two-system architecture, scaling tests, connection to shared coupling $\mathbf{M}$.

The core idea is simple: **don't store what you can compute**. The embedding table is a cache. Orbital embeddings delete the cache and recompute from a compact code using shared dynamics. On modern hardware, this is faster (compute is cheap, memory is expensive) and more parameter-efficient (the saved parameters go to the model backbone). The structure of language—roles, categories, relationships—makes the compression possible. The dynamics—a shared function iterated on tiny seeds—makes it practical.

22 What the Algebra Tells Us: Coupling, Response, and Their Commutator

The ORN has two core operators at every layer: a shared coupling matrix $\mathbf{M} = AB^\top$ and a per-layer response matrix $R^{(\ell)} = W_O^{(\ell)\top} W_V^{(\ell)}$. Coupling determines *which* tokens attend to which; response determines *what* information is extracted. $\mathbf{M}$ is shared; R varies per layer.

A natural question is: what algebraic relationship develops between these operators during training? The answer turns out to be informative and, in places, surprising.

22.1 The Commutator: What It Measures

For two square matrices P and Q , the **commutator** is $[P, Q] = PQ - QP$. If $[P, Q] = 0$, the matrices commute—they share an eigenbasis and can be simultaneously diagonalised. If $[P, Q] \neq 0$, they disagree about the basis: there exist directions where the order of application matters.

The commutator $[\mathbf{M}, R^{(\ell)}]$ measures how much the coupling and response geometries *conflict*. In directions where they commute, applying coupling then response gives the same result as response then coupling—the operators are compatible. In directions where they don't, the order matters, and the model must use the non-commutative structure to do something that neither operator alone can do.

22.2 What We Found: Three Properties

We computed $C^{(\ell)} = [\mathbf{M}, R^{(\ell)}]$ for a trained ORN (50M parameters, $d = 508$, 9 layers). Three things stand out.

1. The commutator is low-rank. 50% of the commutator’s energy lives in just 6 dimensions out of 508. 90% lives in ~ 81 dimensions. For comparison, a random matrix R (matched norm, unrelated to $\mathbf{M}$) produces a commutator with 90%-energy rank of 93 ± 1.7 —significantly higher (the difference is $> 4\sigma$).

What this means: $\mathbf{M}$ and R have learned to *nearly commute* in most directions. They agree on a shared spectral basis in roughly 430 out of 508 dimensions. The remaining ~ 80 dimensions are where they genuinely disagree—where per-layer response structure cannot be reduced to shared coupling structure.

2. The commutator spectrum is locked to $\mathbf{M}$ ’s. The singular value ordering of $[\mathbf{M}, R^{(\ell)}]$ correlates perfectly ($\rho = 1.0$) with $\mathbf{M}$ ’s singular value ordering, across all layers. Large singular values of $\mathbf{M}$ correspond to large singular values of the commutator. The directions where coupling is strongest are exactly the directions where coupling and response most disagree.

This makes architectural sense: in the directions where $\mathbf{M}$ ’s coupling is weak (small singular values), there’s nothing for the response to disagree with. Non-commutativity only matters where coupling is doing real work.

3. The commutator is NOT a polynomial function of $\mathbf{M}$. We tested $C^{(\ell)} \approx \alpha\mathbf{M} + \beta\mathbf{M}^2 + \gamma I$ (a deformed commutation relation). This fails (fit error ~ 1.0). The commutator’s structure depends on both $\mathbf{M}$ and R , not on $\mathbf{M}$ alone. The operators are coupled but not algebraically determined by each other.

22.3 How This Evolves During Training: Liberation Dynamics

We tracked the commutator’s rank across training checkpoints (steps 100 through final). The result is a three-phase process:

Step	Phase	$[\mathbf{M}, R]$ rank ₉₀	$\ [\mathbf{M}, R]\ $	$\mathbf{M}$ rank ₉₀
100	Undifferentiated	10	0.0	11
200	Undifferentiated	3	0.0	3
2000	Liberation	92	2.7	88
4000	Re-alignment	82	3.3	74
6000	Crystallised	81	3.6	71
8000	Crystallised	81	3.6	71
final	Crystallised	81	3.6	71

Phase 1: Undifferentiated (steps 0–200). Both $\mathbf{M}$ and R are near-random and low-rank. The commutator is tiny (rank ~ 3 –10, norm ≈ 0). Neither operator has learned structure yet, so there is nothing to disagree about.

Phase 2: Liberation (steps 200–2000). Training breaks the symmetry between coupling and response. $\mathbf{M}$ begins crystallising its coupling structure; R begins learning per-layer response structure. The commutator rank *explodes* from 3 to 92. This is the moment $\mathbf{M}$ and R differentiate—they are learning different things about the data, and the order of application starts to matter.

Phase 3: Re-alignment (steps 2000–final). After the initial liberation, the two operators negotiate a shared spectral basis. The commutator rank *decreases* from 92 to 81, then stabilises. At the same time, the commutator *norm increases* slightly ($2.7 \rightarrow 3.6$)—the non-commuting subspace gets stronger in fewer dimensions. $\mathbf{M}$ ’s rank crystallises from 88 to 71 over the same period.

This three-phase trajectory is the **crystallisation process seen through the commutator**. The $\mathbf{M}$ crystallisation reported in earlier sections (rank dropping, spectrum stabilising) is accompanied by a simultaneous crystallisation of the coupling-response relationship. They co-crystallise.

22.4 The Eigenvector Overlap: Are They Aligned?

The commutator being low-rank means $\mathbf{M}$ and R nearly commute. But “nearly commute” and “share eigenvectors” are distinct claims. We measured the eigenvector overlap directly.

For random unrelated matrices at $d = 508$, the expected overlap per eigenvector pair is $1/d \approx 0.002$. The trained $\mathbf{M}$ and R :

Top- k subspace	Overlap	vs random
Top 5	0.0143	$1.45\times$
Top 10	0.0259	$1.32\times$
Top 20	0.0464	$1.18\times$
Top 50	0.1034	$1.05\times$
Bulk (all 508)	0.002	$1.08\times$

The top 5 eigenvectors of $\mathbf{M}$ are 45% more aligned with the top 5 of R than random. By top 50, the alignment has faded to 5% above random. The bulk is essentially random.

Interpretation: $\mathbf{M}$ and R have co-adapted in a small core (~ 5 directions) but are independent in the remaining ~ 500 dimensions. This is consistent with the low-rank commutator: the 80-dimensional non-commuting subspace includes a 5-dimensional core of genuine co-adaptation, embedded in a larger space of generic-position structure.

22.5 Why This Matters

Three practical consequences follow from the commutator analysis:

1. **The architecture works as designed.** The ORN separates coupling from response. The near-random eigenvector overlap confirms they carry complementary, non-redundant information. Sharing $\mathbf{M}$ does not force R into a coupled subspace—they stay independent where independence is useful.
2. **The ~ 80 -dimensional non-commuting subspace is the per-layer “budget.”** Each layer’s response head needs only ~ 80 dimensions of genuine structural disagreement with the shared coupling. The other ~ 430 dimensions are redundant between coupling and response—they could, in principle, be factored out.
3. **The liberation trajectory is a training diagnostic.** The three-phase pattern (undifferentiated $\rightarrow$ liberation $\rightarrow$ crystallisation) provides a new way to monitor training health. If the commutator rank doesn’t rise (no liberation), the model isn’t differentiating coupling from response. If it doesn’t fall (no re-alignment), the model hasn’t found a shared basis. Both are failure modes.

22.6 Connecting to the Literature

The commutator analysis connects to three recent lines of work:

- Zhussip et al. (2025, MASA [14]) decompose attention projections into shared “matrix atoms” across layers. Our commutator shows *why* this decomposition works: the coupling and response matrices nearly commute in most directions, so a shared basis exists.
- Wang et al. (ICLR 2025 [61]) show attention outputs live in a low-dimensional subspace ($\sim 60\%$ effective rank). Our PCA basis transfer finding ($> 97\%$ alignment across prompts) is the dynamical version: the residual stream trajectory lives on a fixed ~ 5 -dimensional manifold determined by $\mathbf{M}$.
- Kim et al. (2026 [62]) apply Koopman operator theory (dynamic mode decomposition) to the residual stream across layers and achieve AUROC 0.995 for predicting training divergence. Our liberation dynamics (commutator rank trajectory) are the operator-algebraic version of their spectral profiling, applied to the coupling-response pair rather than to the residual stream itself.

22.7 A Cautionary Note on Free Probability

It is tempting to interpret the near-commutativity of $\mathbf{M}$ and R through the lens of *free probability* (Voiculescu), where “freeness” is the noncommutative analogue of independence. We initially tested this by checking whether the eigenvalue distribution of $\mathbf{M} + R$ matches the free additive convolution $\mathbf{M} \boxplus R$. It does—to four decimal places.

However, this is **trivially true at $d = 508$** . Control experiments show that random matrices (unrelated to $\mathbf{M}$) also match the free convolution prediction at this dimension (ratio = 0.9998 ± 0.0014). At large d , free convolution is the default for *any* pair of matrices with unrelated eigenbases. The result confirms generic position—not a special property of training.

The free probability framework may still be useful theoretically (the liberation dynamics have the flavour of Voiculescu’s liberation gradient), but the cumulant tests do not provide empirical evidence at the matrix sizes we work with. A genuinely distinguishing test would require either much smaller d or higher-order free cumulants (κ_6, κ_8) where finite- d corrections are detectable.

References

- [1] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *NeurIPS*, 2022.
- [2] T. Chen, B. Xu, C. Zhang, and C. Guestrin. Training deep nets with sublinear memory cost. *arXiv:1604.06174*, 2016.
- [3] S. Bai, J. Z. Kolter, and V. Koltun. Deep equilibrium models. *NeurIPS*, 2019.
- [4] M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and Ł. Kaiser. Universal Transformers. *ICLR*, 2019.
- [5] T. Mikolov, K. Chen, G. Corrado, and J. Dean. Efficient estimation of word representations in vector space. *arXiv:1301.3781*, 2013.
- [6] S. Williams, A. Waterman, and D. Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [7] Y. Zhang et al. Meta-Transformer: A Unified Framework for Multimodal Learning. *ICLR*, 2024.

- [8] W. Liang et al. Mixture-of-Transformers: A Sparse and Scalable Architecture for Multi-Modal Foundation Models. *arXiv:2411.04996*, 2024.
- [9] R. Bachmann et al. 4M-21: An Any-to-Any Vision Model for Tens of Tasks and Modalities. *NeurIPS*, 2024.
- [10] P. Wang et al. ONE-PEACE: Exploring One General Representation Model Toward Unlimited Modalities. *NeurIPS*, 2023.
- [11] M. Huh, B. Cheung, T. Wang, and P. Isola. The Platonic Representation Hypothesis. *ICML*, 2024.
- [12] A. Altabaa and J. Lafferty. Disentangling and Integrating Relational and Sensory Information in Transformer Architectures. *ICML*, 2025.
- [13] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut. ALBERT: A Lite BERT for Self-supervised Learning of Language Representations. *ICLR*, 2020.
- [14] Multi-head Attention Sharing via Dictionary Compression. *arXiv*, 2025.
- [15] Relaxed Recursive Transformers: Effective Parameter Sharing with Cross-Layer Heads. Meta, *ICLR*, 2025.
- [16] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention Is All You Need. *NeurIPS*, 2017.
- [17] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu. RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing*, 2024.
- [18] J. Ainslie et al. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *EMNLP*, 2023.
- [19] One Wide Feedforward is All You Need. Apple, *EMNLP*, 2023.
- [20] FFN Fusion: Rethinking Sequential Computation in Transformers. NVIDIA, *NeurIPS*, 2025.
- [21] M. Geva, R. Schuster, J. Berant, and O. Levy. Transformer Feed-Forward Layers Are Key-Value Memories. *EMNLP*, 2021.
- [22] Y. Xiao, L. Zhu, and T. Mao. Sharing Attention Weights for Fast Transformer. *IJCAI*, 2019.
- [23] S. Takase and S. Kiyono. Lessons on Parameter Sharing across Layers in Transformers. *arXiv:2104.06022*, 2021.
- [24] S. Bhojanapalli et al. Leveraging Redundancy in Attention with Reuse Transformers. *arXiv:2110.06821*, 2021.
- [25] K. He, X. Zhang, S. Ren, and J. Sun. Deep Residual Learning for Image Recognition. *CVPR*, 2016.
- [26] S. Kornblith, M. Norouzi, H. Lee, and G. Hinton. Similarity of Neural Network Representations Revisited. *ICML*, 2019.
- [27] S. Brunton and J. N. Kutz. Data-Driven Science and Engineering. Cambridge University Press, 2022.
- [28] Self-attention Networks Localize When QK-eigenspectrum Concentrates. *ICML*, 2024.

- [29] Mind the Gap: A Spectral Analysis of Rank Collapse and Signal Propagation in Transformers. *NeurIPS*, 2024.
- [30] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret. Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. *ICML*, 2020.
- [31] J. L. McClelland, B. L. McNaughton, and R. C. O'Reilly. Why There Are Complementary Learning Systems in the Hippocampus and Neocortex. *Psychological Review*, 1995.
- [32] Y. LeCun. A Path Towards Autonomous Machine Intelligence. 2022.
- [33] D. Ha and J. Schmidhuber. World Models. *NeurIPS*, 2018.
- [34] N. Mu et al. Cross-Layer Attention Sharing for Large Language Models (LISA). *arXiv*, 2024.
- [35] W. Chang et al. xKV: Cross-Layer SVD for KV-Cache Compression. *arXiv*, 2025.
- [36] B. Pan et al. Dissecting Query-Key Interaction in Vision Transformers. *NeurIPS*, 2024.
- [37] A. Gu and T. Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. *arXiv:2312.00752*, 2023.
- [38] A. Gu, K. Goel, and C. Ré. Efficiently Modeling Long Sequences with Structured State Spaces. *ICLR*, 2022.
- [39] O. Lieber et al. Jamba: A Hybrid Transformer-Mamba Language Model. *arXiv*, 2024.
- [40] R. Chen and D. Wu. DeepOSG: Deep Learning of Operators with Semigroup Structure. *arXiv*, 2023.
- [41] P. Mehta and D. J. Schwab. An exact mapping between the Variational Renormalization Group and Deep Learning. *arXiv:1410.3831*, 2014.
- [42] H. W. Lin and M. Tegmark. Why Does Deep and Cheap Learning Work So Well? *J. Stat. Phys.*, 2017.
- [43] D. A. Roberts, S. Yaida, and B. Hanin. The Principles of Deep Learning Theory. Cambridge University Press, 2022.
- [44] S. Lotfi et al. Non-Vacuous Generalization Bounds for Large Language Models. *ICML*, 2024.
- [45] A. Louis et al. Deep Neural Networks Have an Inbuilt Occam’s Razor. *Nature Communications*, 2025.
- [46] N. Shazeer. Fast Transformer Decoding: One Write-Head is All You Need. *arXiv:1911.02150*, 2019.
- [47] DeepSeek-AI. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. *arXiv*, 2024.
- [48] Conditional Memory via Scalable Lookup (Engram). *arXiv:2601.07372*, 2026.
- [49] A. Dosovitskiy et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *ICLR*, 2021.
- [50] A. Radford et al. Learning Transferable Visual Models From Natural Language Supervision. *ICML*, 2021.

- [51] M. Chen, A. Radford, R. Child, J. Wu, H. Jun, D. Luan, and I. Sutskever. Generative Pretraining from Pixels. *ICML*, 2020.
- [52] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick. Masked Autoencoders Are Scalable Vision Learners. *CVPR*, 2022.
- [53] K. Tian, Y. Jiang, Z. Yuan, B. Peng, and L. Wang. Visual Autoregressive Modeling: Scalable Image Generation via Next-Scale Prediction. *NeurIPS*, 2024.
- [54] K. Friston. The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11(2):127–138, 2010.
- [55] A. Damasio. Convergence and divergence in a neural architecture for recognition and memory. *Trends in Neurosciences*, 2009.
- [56] Y. Girdhar et al. ImageBind: One Embedding Space To Bind Them All. *CVPR*, 2023.
- [57] H. Liu et al. Visual Instruction Tuning. *NeurIPS*, 2024.
- [58] M. Nickel and D. Kiela. Poincaré Embeddings for Learning Hierarchical Representations. *NeurIPS*, 2017.
- [59] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud. Neural Ordinary Differential Equations. *NeurIPS*, 2018.
- [60] Uni-X: Resolving Gradient Conflicts in Unified Multimodal Transformers. *arXiv:2509.24365*, 2025.
- [61] Z. Wang, H. Ge, C. Shu, Z. He, and X. Qiu. Attention Layers Add Into Low-Dimensional Residual Subspaces. *ICLR*, 2025.
- [62] J. Kim, S. Taniguchi, R. Kawano, Y. Iwasawa, and Y. Matsuo. Residual Koopman Spectral Profiling for Predicting and Preventing Transformer Training Instability. *arXiv:2602.22988*, 2026.
- [63] S. Zhai, T. Likhomanenko, E. Littwin, D. Busbridge, J. Ramapuram, Y. Zhang, J. Gu, and J. Susskind. Stabilizing Transformer Training by Preventing Attention Entropy Collapse. *ICML*, 2023.
- [64] A. Ansuini, A. Laio, J. H. Macke, and D. Zoccolan. Intrinsic Dimension of Data Representations in Deep Neural Networks. *NeurIPS*, 2019.
- [65] C. Jacobs, T. Fel, Y. Hakim, K. Brondetta, J. Ba, and T. A. Keller. Block-Recurrent Dynamics in Vision Transformers. *arXiv:2512.19941*, 2025.
- [66] G. Lakoff and M. Johnson. *Metaphors We Live By*. University of Chicago Press, 1980.